

 Devin AI Export - PDF Version

Generated on: 11/4/2025, 10:06:02 PM

Source: Devin AI Platform

Overview

Relevant source files

- [

 .devcontainer/devcontainer.json](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json>))
- [

 .jhipster/Broker.json](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json>))
- [

 .jhipster/ChatRoom.json](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json>))
- [

 .jhipster/CopyPortfolio.json](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json>))
- [

 .jhipster/CopySubscriber.json](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json>))
- [

 .jhipster/CopyTradingOrder.json](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json>))

Purpose and Scope

This document provides a high-level introduction to the **nhsv-admin** repository, a JHipster-based administrative platform designed to manage a copy trading system with integrated broker management and chat room features. The system enables market leaders to maintain portfolios that subscribers can automatically replicate through algorithmic trade execution.

This overview covers the system's architecture, core domain subsystems, technology stack, and development environment. For detailed information about specific subsystems, refer to:

- User and access management: [User and Access Management](#)
 - Chat room features: [Chat Room System](#)
 - Copy trading functionality: [Copy Trading Platform](#)
 - Development environment setup: [Development Environment](#)
-

System Architecture

The nhsv-admin repository implements a multi-layered architecture with four primary domain subsystems supported by a robust development infrastructure. The system is built on JHipster, which generates standardized Spring Boot backend components and manages database schema evolution through Liquibase.

High-Level Component Architecture



Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L1-L47>.

[.devcontainer/devcontainer.json1-47](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L1-L47) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L1-L47>)

[.jhipster/Broker.json1-70](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json#L1-L70) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json#L1-L70>) ([%5B](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json#L1-L70))

[.jhipster/ChatRoom.json1-86](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L1-L86) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L1-L86>) ([%5B](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L1-L86))

[.jhipster/CopyPortfolio.json1-36](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L1-L36) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L1-L36>) ([%5B](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L1-L36))

[.jhipster/CopySubscriber.json1-73](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json#L1-L73) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json#L1-L73>) ([%5B](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json#L1-L73))

[.jhipster/CopyTradingOrder.json1-93](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json#L1-L93) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json#L1-L93>) ([%5B](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json#L1-L93))

Technology Stack

The system is built on a modern Java-based technology stack with containerized development support.

Component	Technology	Configuration Reference
Runtime Environment	Java 11	[
.devcontainer/devcontainer.json11](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L11-L11 (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L11-L11))		
Application Framework	JHipster (Spring Boot)	Entity definitions in <code>.jhipster/*.json</code>
Build Tool	Maven	[

.devcontainer/devcontainer.json13](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L13-L13> (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L13-L13>)) | | **Frontend Runtime** | Node.js LTS | [

.devcontainer/devcontainer.json15](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L15-L15> (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L15-L15>)) |

Database	PostgreSQL	Entity table prefix: <code>t_*</code>
Schema Migration	Liquibase	<code>incrementalChangelog: true</code> in entity definitions
DTO Mapping	MapStruct	[

.jhipster/CopyPortfolio.json4](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L4-L4>)([%5B](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L4-L4)).

.jhipster/CopySubscriber.json3](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json#L3-L3>)([%5B](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json#L3-L3)) | | **API Features** | Pagination, JPA Metamodel Filtering | [

.jhipster/ChatRoom.json74](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L74-L74>)([%5B](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L74-L74)).

.jhipster/CopyPortfolio.json16](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L16-L16>)([%5B](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L16-L16)) | | **Development Container** | Docker with VS Code | [

.devcontainer/devcontainer.json1-47](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L1-L47> (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L1-L47>)) |

Sources: (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L1-L47>).

.devcontainer/devcontainer.json1-47 (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L1-L47>)[

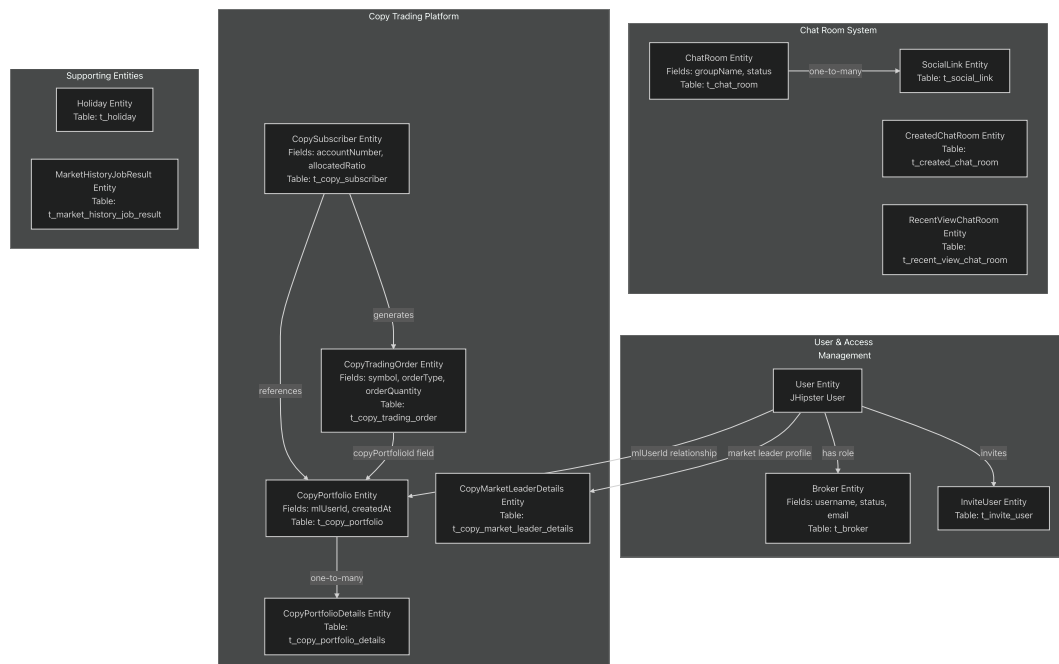
.jhipster/Broker.json62-68](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json#L62-L68>)([%5B](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json#L62-L68)).

.jhipster/CopyPortfolio.json#L4-L17](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L4-L17>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L4-L17>)

Domain Subsystems

The application is organized into four primary domain subsystems, each responsible for distinct business capabilities.

Domain Subsystem Map



Sources: (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json#L1-L70>)

.jhipster/Broker.json#1-70] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json#L1-L70>)

.jhipster/ChatRoom.json#1-86](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L1-L86>) [[%5B](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L1-L86)]([%5B](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L1-L86))

.jhipster/CopyPortfolio.json#1-36](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L1-L36>) [[%5B](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L1-L36)]([%5B](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L1-L36))

.jhipster/CopySubscriber.json#1-73](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json#L1-L73>) [[%5B](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json#L1-L73)]([%5B](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json#L1-L73))

.jhipster/CopyTradingOrder.json#1-93](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json#L1-L93>) [[%5B](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json#L1-L93)]([%5B](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json#L1-L93))

Subsystem Responsibilities

Subsystem	Primary Entities	Core Functionality
User & Access Management	Broker, InviteUser, User	Manages broker accounts with status tracking, email validation, invitation workflow, and user authentication
Chat Room System	ChatRoom, SocialLink, CreatedChatRoom, RecentViewChatRoom	Handles chat room lifecycle with approval workflow (status: PENDING, APPROVED, REJECTED), social media links, and view tracking
Copy Trading Platform	CopyPortfolio, CopyPortfolioDetails, CopySubscriber, CopyTradingOrder, CopyMarketLeaderDetails	Core business domain for portfolio management, subscriber allocation ratios, and automated trade execution
Supporting Services	Holiday, MarketHistoryJobResult	Provides market calendar and job tracking for data processing operations

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json#L4-L60>

[.jhipster/Broker.json#L4-L60](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json#L4-L60) <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json#L4-L60>

[.jhipster/ChatRoom.json#L4-L69](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L4-L69) <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L4-L69>

[.jhipster/CopyPortfolio.json#L8-L33](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L8-L33) <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L8-L33>

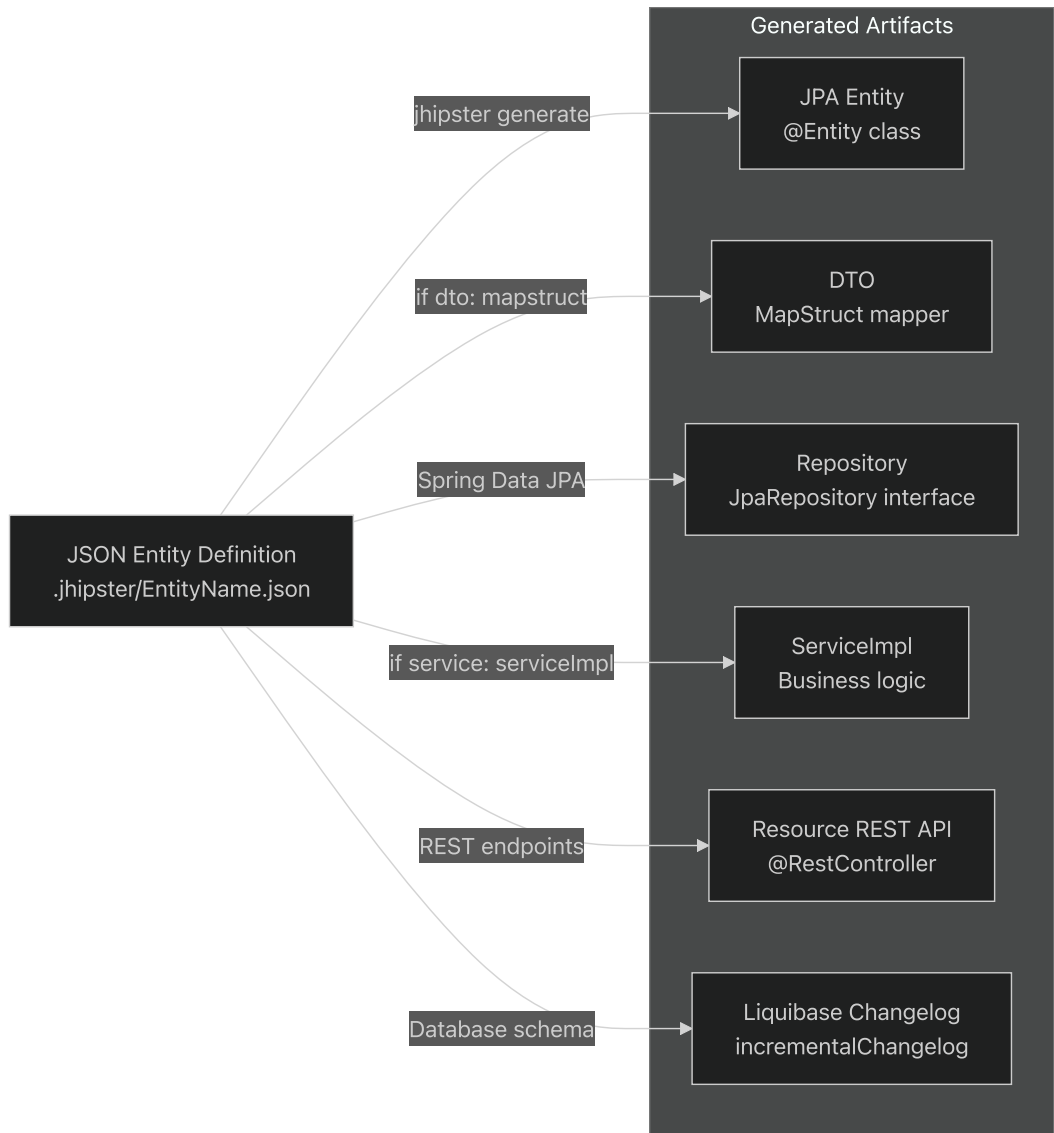
[.jhipster/CopySubscriber.json#L6-L69](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json#L6-L69) <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json#L6-L69>

[.jhipster/CopyTradingOrder.json#L6-L83](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json#L6-L83) <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json#L6-L83>

JHipster Entity Generation Pattern

The system leverages JHipster's code generation capabilities to maintain consistency across all entities. Each entity is defined by a JSON configuration file in the `.jhipster/` directory, which drives the generation of all backend layers.

Entity-to-Code Generation Flow



Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json#L62-L68>)
[.jhipster/Broker.json62-68](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json#L62-L68) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json#L62-L68>) [[.jhipster/CopyPortfolio.json4-17](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L4-L17)](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L4-L17>) [[%5B](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L4-L17)](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json#L2-L5>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json#L2-L5>)

Key Entity Configuration Fields

Each JSON entity definition contains standardized configuration:

Configuration Field	Purpose	Example Values
changelogDate	Liquibase migration timestamp	"20230703044953"
dto	DTO generation strategy	"no", "mapstruct"
entityTableName	Database table name	"t_copy_portfolio"
fields[]	Entity properties with validation	Field types, constraints
relationships[]	JPA relationships	one-to-many, many-to-one
service	Service layer generation	"serviceClass", "serviceImpl"
pagination	REST API pagination	"pagination", "no"
jpaMetamodelFiltering	Dynamic query filtering	true, false
incrementalChangelog	Liquibase incremental updates	true

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json#L2-L68>
[.jhipster/Broker.json#2-68](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json#L2-L68) <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json#L2-L68> [
[.jhipster/CopyPortfolio.json#2-18](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L2-L18)](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L2-L18>)[[%5B](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L2-L18)]([%5B](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L2-L18))
[.jhipster/CopyTradingOrder.json#2-17](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json#L2-L17)](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json#L2-L17> <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json#L2-L17>))

Development Environment

The repository includes a fully containerized development environment configured through VS Code Dev Containers. This ensures consistent development experiences across different platforms.

Development Container Configuration

The development environment is defined in <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L1-L47>
[.devcontainer/devcontainer.json#1-47](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L1-L47) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L1-L47>) with the following key features:

Feature	Configuration	Purpose
Base Image	Java 11 variant	[
.devcontainer/devcontainer.json#11](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L11-L11 (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L11-L11))		
Maven	Installed	[

.devcontainer/devcontainer.json13](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L13-L13>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L13-L13>)) | | **Node.js** | LTS version | [

.devcontainer/devcontainer.json15](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L15-L15>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L15-L15>)) | | **Docker-in-Docker** | Enabled | [

.devcontainer/devcontainer.json44](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L44-L44>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L44-L44>)) | | **Forwarded Ports** | 9060, 3001, 9000, 8080 | [

.devcontainer/devcontainer.json36](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L36-L36>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L36-L36>)) |

Installed VS Code Extensions

The container automatically installs development productivity extensions:

- `vscjava.vscode-java-pack` - Java development tools
- `pivotal.vscode-boot-dev-pack` - Spring Boot support
- `dbaeumer.vscode-eslint` - JavaScript linting
- `esbenp.prettier-vscode` - Code formatting
- `firsttris.vscode-jest-runner` - Test runner
- `christian-kohler.npm-intellisense` - NPM package IntelliSense

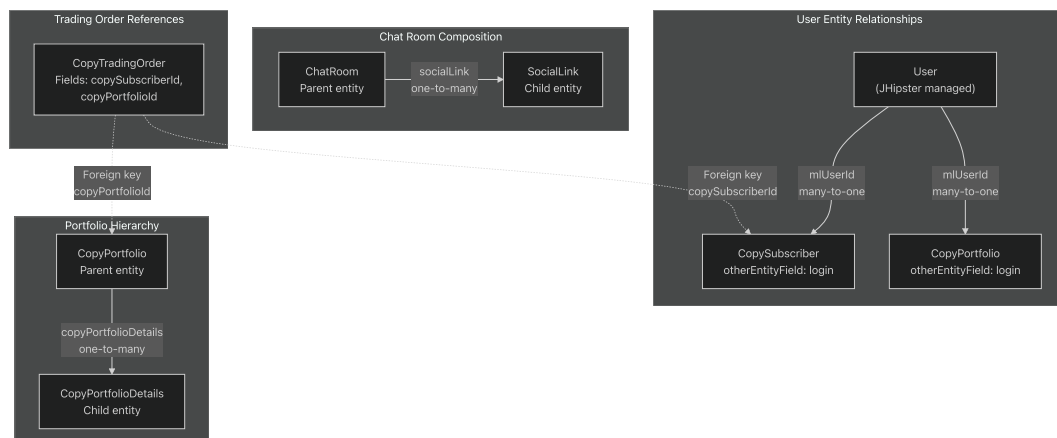
Sources: (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L25-L33>)

[.devcontainer/devcontainer.json25-33](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L25-L33) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L25-L33>)

Entity Relationship Patterns

The system follows consistent entity relationship patterns established by JHipster conventions.

Core Relationship Types



Sources: (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L20-L33>)

[.jhipster/CopyPortfolio.json20-33](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L20-L33) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L20-L33>) [

[.jhipster/CopySubscriber.json61-70](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json#L61-L70)] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json#L61-L70>) [(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json#L61-L70>)]

[admin/blob/2cf90647/.jhipster/CopySubscriber.json#L61-L70\)%5B\)](#)

[.jhipster/ChatRoom.json#76-83\]\(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L76-L83\)\[](#)[\(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L76-L83\)%5B\)](#)

[.jhipster/CopyTradingOrder.json#75-83\]\(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json#L75-L83](#) [\(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json#L75-L83\)\)](#)

Relationship Configuration Examples

Many-to-One with User Entity:

```
{
  "otherEntityField": "login",
  "otherEntityName": "user",
  "relationshipName": "mlUserId",
  "relationshipType": "many-to-one"
}
```

Source: [\(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L28-L32\)](#)

[.jhipster/CopyPortfolio.json#28-32](#) [\(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L28-L32\)\[](#)

[.jhipster/CopySubscriber.json#63-68\]\(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json#L63-L68](#) [\(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json#L63-L68\)\)](#)

One-to-Many with Child Entities:

```
{
  "otherEntityName": "copyPortfolioDetails",
  "otherEntityRelationshipName": "copyPortfolioId",
  "relationshipName": "copyPortfolioDetails",
  "relationshipType": "one-to-many"
}
```

Source: [\(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L21-L25\)](#)

[.jhipster/CopyPortfolio.json#21-25](#) [\(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L21-L25\)](#)

Foreign Key References:

```
{
  "fieldName": "copySubscriberId",
  "fieldType": "Long",
  "fieldValidateRules": ["required"]
},
{
  "fieldName": "copyPortfolioId",
  "fieldType": "Long",
  "fieldValidateRules": ["required"]
}
```

Source: [\(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json#L75-L83\)](#)

[.jhipster/CopyTradingOrder.json#75-83](#) [\(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json#L75-L83\)](#)

Key Entity Field Patterns

Status and Enumeration Fields

The system uses typed enumerations for state management and categorization:

Entity	Enum Field	Values	Purpose
ChatRoom	status	PENDING , APPROVED , REJECTED , CREATED , UPDATED	Approval workflow tracking
ChatRoom	action	CREATE , UPDATE , DELETE	Change tracking
CopySubscriber	orderSetType	QUICK_MATCH , SAFE_MATCH	Order execution strategy
CopyTradingOrder	sellBuyType	BUY , SELL	Trade direction
CopyTradingOrder	exchangeType	HOSE , HNX , UPCOM	Vietnamese stock exchanges
CopyTradingOrder	orderType	ATO , ATC , LO , MP , MAK , MOK , MTL , PLO	Order execution types

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L29-L31>
[.jhipster/ChatRoom.json#L29-L31](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L29-L31) <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L29-L31> [
.jhipster/ChatRoom.json#L66-L68]([https://github.com/nhsv-tradex/nhsv-](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L66-L68)
[admin/blob/2cf90647/.jhipster/ChatRoom.json#L66-L68](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L66-L68))([%5B](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L66-L68))
.jhipster/CopySubscriber.json#L30-L32]([https://github.com/nhsv-tradex/nhsv-](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json#L30-L32)
[admin/blob/2cf90647/.jhipster/CopySubscriber.json#L30-L32](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json#L30-L32))([%5B](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json#L30-L32))
.jhipster/CopyTradingOrder.json#L31-L43]([https://github.com/nhsv-tradex/nhsv-](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json#L31-L43)
[admin/blob/2cf90647/.jhipster/CopyTradingOrder.json#L31-L43](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json#L31-L43))([https://github.com/nhsv-tradex/nhsv-](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json#L31-L43)
[admin/blob/2cf90647/.jhipster/CopyTradingOrder.json#L31-L43](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json#L31-L43)))

Temporal Tracking Fields

All entities maintain comprehensive temporal tracking:

Field Pattern	Type	Usage
createdAt	ZonedDateTime	Entity creation timestamp (often required)
updatedAt	ZonedDateTime	Last modification timestamp
approvedAt	ZonedDateTime	Workflow approval timestamp
rejectedAt	ZonedDateTime	Workflow rejection timestamp
deactivatedAt	ZonedDateTime	Deactivation timestamp

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json#L42-L51>

[.jhipster/Broker.json#42-51](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json#L42-L51) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json#L42-L51)[

[.jhipster/ChatRoom.json#38-52](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L38-L52)](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L38-L52)[(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L38-L52)%5B)

[.jhipster/CopyPortfolio.json#9-11](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L9-L11)](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L9-L11)[(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L9-L11)%5B)

[.jhipster/CopySubscriber.json#35-41](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json#L35-L41)](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json#L35-L41)[(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json#L35-L41)%5B)

[.jhipster/CopyTradingOrder.json#66-72](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json#L66-L72)](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json#L66-L72 (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json#L66-L72))

Audit Trail Fields

User action tracking is implemented through dedicated audit fields:

Field	Purpose	Example Entities
<code>createdBy</code>	User who created the record	<code>ChatRoom</code>
<code>approvedBy</code>	User who approved the record	<code>ChatRoom</code>
<code>rejectedBy</code>	User who rejected the record	<code>ChatRoom</code>
<code>deactivatedBy</code>	User who deactivated the record	<code>Broker</code>
<code>invitedBy</code>	User who initiated invitation	<code>Broker</code>

Sources: (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L34-L63)

[.jhipster/ChatRoom.json#34-63](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L34-L63) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L34-L63)[

[.jhipster/Broker.json#54-59](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json#L54-L59)](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json#L54-L59 (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json#L54-L59))

Copy Trading Entity Hierarchy

The copy trading platform follows a hierarchical entity structure with clear separation between market leader portfolios and subscriber accounts.

Layer	Entities	Relationships	Purpose
Market Leader Profile	CopyMarketLeaderDetails , CopyMarketLeaderProfitLoss , CopyMarketLeaderProfitLossDetails	Linked to User entity	Stores market leader metadata, performance metrics
Portfolio Management	CopyPortfolio , CopyPortfolioDetails	CopyPortfolio → CopyPortfolioDetails (one-to-many)	Current portfolio state and asset allocations
Portfolio History	CopyPortfolioHistory , CopyPortfolioDetailHistory	Historical snapshots	Time-series portfolio composition tracking
Subscriber Management	CopySubscriber , CopySubscriberDetails , CopySubscriberHistory	CopySubscriber references User (mlUserId)	Subscription records, account details, allocation ratios
Trade Execution	CopyTradingOrder , CopyTradingRegister	References copySubscriberId and copyPortfolioId	Order generation and registration tracking

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L1-L36>

[.jhipster/CopyPortfolio.json#L1-L36](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L1-L36) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L1-L36>)

[.jhipster/CopySubscriber.json#L1-L73](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json#L1-L73) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json#L1-L73>) ([%5B](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json#L1-L73))

[.jhipster/CopyTradingOrder.json#L1-L93](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json#L1-L93) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json#L1-L93>)

Validation and Data Integrity

The system enforces data integrity through field-level validation rules defined in entity configurations:

Common Validation Rules

Rule	Purpose	Application Example
required	Non-null constraint	accountNumber, userName, allocatedRatio
unique	Uniqueness constraint	Broker.email, Broker.id
maxlength	String length limit	accountNumber (255), customerName (500)

Example from CopySubscriber:

```
{
  "fieldName": "accountNumber",
  "fieldType": "String",
  "fieldValidateRules": ["required", "maxlength"],
  "fieldValidateRulesMaxlength": "255"
}
```

Source: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json#L7-L10>

[.jhipster/CopySubscriber.json7-10](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json#L7-L10) <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json#L7-L10>

Example from Broker:

```
{
  "fieldName": "email",
  "fieldType": "String",
  "fieldValidateRules": ["unique"]
}
```

Source: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json#L37-L40>

[.jhipster/Broker.json37-40](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json#L37-L40) <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json#L37-L40>

Repository Structure Overview

The codebase follows JHipster's standard directory organization:

Directory	Contents	Purpose
.devcontainer/	Docker and VS Code configuration	Containerized development environment
.jhipster/	Entity JSON definitions	JHipster entity metadata for code generation
src/main/java/	Java source code	Spring Boot application backend
src/main/resources/	Application resources	Configuration files, Liquibase changelogs
src/main/webapp/	Frontend application	React/Angular web application
target/	Maven build output	Compiled artifacts (gitignored)
node_modules/	NPM dependencies	Frontend dependencies (gitignored)

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L1-L47>

[.devcontainer/devcontainer.json#1-47](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L1-L47) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L1-L47>) [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L1-L47>]

[.jhipster/Broker.json#1-70](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L1-L70)] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L1-L70>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L1-L70>)

This overview establishes the foundational understanding of the nhsv-admin system architecture. For detailed exploration of specific subsystems, refer to the dedicated pages listed in the table of contents.

Development Environment

Overview

Relevant source files

- [[.devcontainer/Dockerfile](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/Dockerfile)] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/Dockerfile>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/Dockerfile>)
- [[.devcontainer/devcontainer.json](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json)] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json>)
- [[.editorconfig](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig)] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig>)
- [[.eslintrc.json](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json)] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json>)

Purpose and Scope

This document provides an overview of the development environment architecture for the nhsv-admin project. It explains how the containerized development setup, code quality tooling, and integrated development workflow work together to ensure consistent development practices across the team.

The development environment is built around a Docker-based VS Code Dev Container that provides Java 11, Node.js, and Maven in a consistent, reproducible environment. Code quality is enforced through multiple layers: editor-level formatting via EditorConfig, static analysis via ESLint, and pre-commit validation via Husky hooks.

For detailed information about specific subsystems:

- Container configuration details: see [Container Configuration](#)
- Code formatting and linting rules: see [Code Quality and Style](#)
- Git configuration for cross-platform development: see [Version Control Configuration](#)
- Pre-commit hook implementation: see [Pre-commit Quality Gates](#)

Development Environment Architecture

The nhsv-admin project uses a fully containerized development environment based on VS Code Dev Containers. This approach eliminates "works on my machine" issues by providing a standardized Docker image with all

required runtimes and tools pre-installed.

Technology Stack

Component	Version/Configuration	Purpose
Base Image	<code>mcr.microsoft.com/vscode/devcontainers/java:0-11</code>	Microsoft's official Java dev container
Java	11	Backend runtime for JHipster application
Node.js	<code>lts/*</code> (latest LTS)	Frontend build tooling and npm dependencies
Maven	Installed via SDKMAN	Java build automation and dependency management
Gradle	Not installed (<code>INSTALL_GRADLE: false</code>)	Not used in this project
Docker-in-Docker	Latest	Enables Docker commands within the container

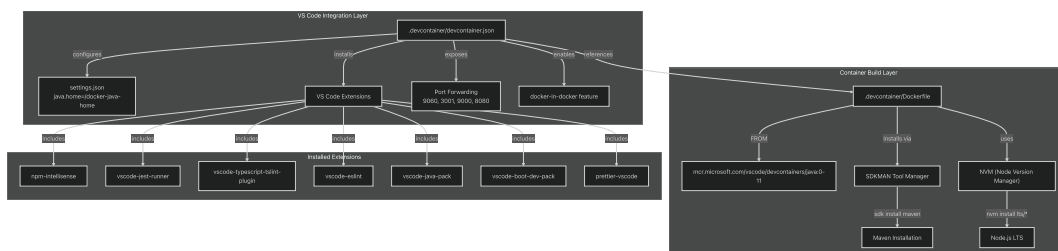
Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/Dockerfile#L1-L26>

[.devcontainer/Dockerfile1-26](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/Dockerfile#L1-L26) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/Dockerfile#L1-L26>)

[.devcontainer/devcontainer.json1-47](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L1-L47) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L1-L47>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L1-L47>)

Development Container Structure

The development environment consists of two primary configuration files that define the container build and VS Code integration:



Development Container Build and Configuration Flow

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/Dockerfile#L1-L26>

[.devcontainer/Dockerfile1-26](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/Dockerfile#L1-L26) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/Dockerfile#L1-L26>)

[.devcontainer/devcontainer.json1-47](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L1-L47) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L1-L47>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L1-L47>)

Container Build Arguments

The Dockerfile accepts several build arguments configured in `devcontainer.json`:

Argument	Value	Description
VARIANT	"11"	Java version selection
INSTALL_MAVEN	"true"	Enable Maven installation via SDKMAN
INSTALL_GRADLE	"false"	Gradle not required for this project
NODE_VERSION	"lts/*"	Install latest LTS version of Node.js

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L7-L16>.

[.devcontainer/devcontainer.json7-16](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L7-L16) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L7-L16>).

VS Code Extension Ecosystem

The development container automatically installs seven VS Code extensions that provide IDE support for the project's technology stack:

Extension ID	Purpose
christian-kohler.npm-intellisense	Auto-complete npm module imports
firsttris.vscode-jest-runner	Run Jest tests from editor
ms-vscode.vscode-typescript-tslint-plugin	TypeScript linting support
dbaeumer.vscode-eslint	ESLint integration for code quality
vscjava.vscode-java-pack	Java language support, debugging, testing
pivotal.vscode-boot-dev-pack	Spring Boot development tools
esbenp.prettier-vscode	Code formatting via Prettier

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L25-L33>.

[.devcontainer/devcontainer.json25-33](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L25-L33) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L25-L33>).

Port Forwarding Configuration

The container exposes four ports to the host machine for accessing various services:

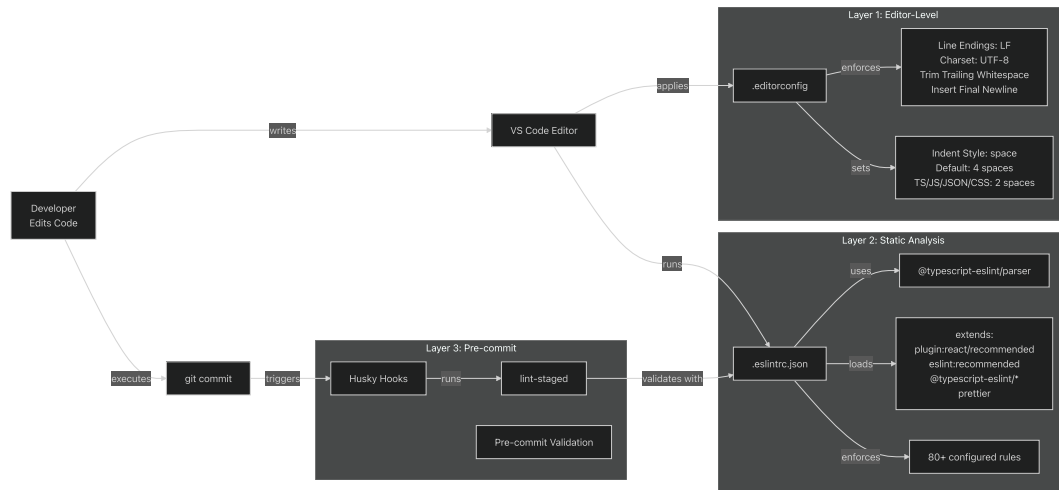
Port	Service	Purpose
9060	Backend (development)	JHipster backend in development mode
3001	Frontend (development)	React frontend development server
9000	Backend (production)	Production-mode backend
8080	Backend (standard)	Standard Spring Boot port

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L36-L36>.

[.devcontainer/devcontainer.json36](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L36-L36) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L36-L36>)

Code Quality Toolchain

The development environment enforces code quality through three integrated layers that progressively validate code from editing through commit:



Multi-Layer Code Quality Enforcement Pipeline

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig#L1-L24>

[.editorconfig1-24](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig#L1-L24) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig#L1-L24>)

[.eslintrc.json1-81](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L1-L81) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L1-L81>)

[%5B](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L1-L81)

[.devcontainer/devcontainer.json25-33](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L25-L33) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L25-L33>)

[admin/blob/2cf90647/.devcontainer/devcontainer.json#L25-L33](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L25-L33) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L25-L33>)

EditorConfig Settings

EditorConfig provides real-time formatting as developers type, ensuring consistency before code is even saved. The configuration defines universal rules and file-type-specific overrides:

Universal Rules (all files):

- `end_of_line = lf` - Unix-style line endings
- `charset = utf-8` - UTF-8 encoding
- `trim_trailing_whitespace = true` - Remove trailing spaces
- `insert_final_newline = true` - Ensure newline at EOF
- `indent_style = space` - Use spaces, not tabs
- `indent_size = 4` - Default 4-space indentation

Frontend Files (`*.{ts,tsx,js,jsx,json,css,scss,yml,html,vue}`):

- `indent_size = 2` - 2-space indentation for consistency with frontend conventions

Markdown Files (`*.md`):

- `trim_trailing_whitespace = false` - Preserve trailing spaces (used for line breaks in Markdown)

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig#L1-L24>

[.editorconfig1-24](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig#L1-L24) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig#L1-L24)

ESLint Configuration

ESLint performs static analysis on TypeScript and JavaScript code, enforcing code quality rules and catching potential bugs. The configuration uses the TypeScript parser and extends multiple rule sets:

Parser and Plugin Configuration:

- Parser: `@typescript-eslint/parser` - TypeScript-aware parsing
- Plugins: `@typescript-eslint` - TypeScript-specific linting rules
- `parserOptions.project: [ './tsconfig.json', './tsconfig.test.json' ]` - Type-aware linting

Extended Rule Sets:

```
plugin:react/recommended      → React best practices
eslint:recommended           → Core ESLint rules
plugin:@typescript-eslint/*    → TypeScript rules
prettier                     → Prettier integration
eslint-config-prettier        → Disable conflicting rules
```

Key Custom Rules:

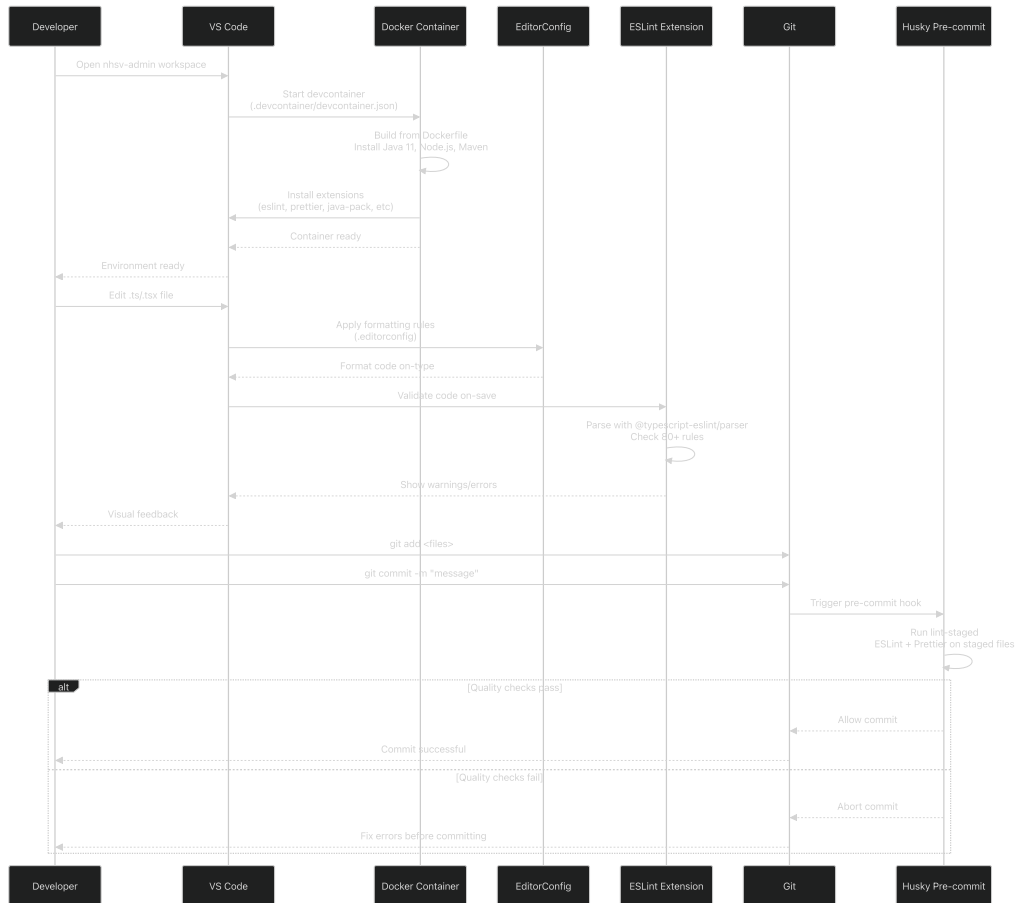
- `@typescript-eslint/member-ordering` - Enforce class member order (static fields → instance fields → constructor → methods)
- `@typescript-eslint/no-shadow` - Prevent variable shadowing
- `no-console` - Allow only `console.warn()` and `console.error()`
- `complexity: 40` - Maximum cyclomatic complexity of 40
- `eqeqeq: always` - Require strict equality (`===` and `!==`)
- `prefer-const` - Use `const` for variables that are never reassigned

Sources: (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L1-L81)

[.eslintrc.json1-81](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L1-L81) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L1-L81)

Development Workflow Integration

The following diagram shows how a developer interacts with the environment from container startup through code commit:



Developer Workflow from Container Start to Code Commit

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L1-L47>.

[.devcontainer/devcontainer.json1-47](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L1-L47) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L1-L47>)

[.editorconfig1-24](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig#L1-L24) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig#L1-L24>) ([%5B](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig#L1-L24))

[.eslintrc.json1-81](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L1-L81) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L1-L81>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L1-L81>)

Container Startup Sequence

When a developer opens the nhsv-admin workspace in VS Code:

1. VS Code reads[

[.devcontainer/devcontainer.json1-47](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L1-L47)] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L1-L47>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L1-L47>)

2. Builds Docker image from[

[.devcontainer/Dockerfile1-26](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/Dockerfile#L1-L26)] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/Dockerfile#L1-L26>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/Dockerfile#L1-L26>) with arguments from[

[devcontainer.json7-16](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/devcontainer.json#L7-L16)] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/devcontainer.json#L7-L16>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/devcontainer.json#L7-L16>)

3. Installs Maven via SDKMAN[

Dockerfile13](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/Dockerfile#L13-L13>)
(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/Dockerfile#L13-L13>))

4. Installs Node.js LTS via NVM[

Dockerfile18](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/Dockerfile#L18-L18>)
(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/Dockerfile#L18-L18>))

5. Installs 7 VS Code extensions[

devcontainer.json25-33](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/devcontainer.json#L25-L33>)
(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/devcontainer.json#L25-L33>))

6. Sets Java home to `/docker-java-home` [

devcontainer.json21](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/devcontainer.json#L21-L21>)
(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/devcontainer.json#L21-L21>))

7. Forwards ports 9060, 3001, 9000, 8080[

devcontainer.json36](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/devcontainer.json#L36-L36>)
(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/devcontainer.json#L36-L36>))

8. Switches to non-root `vscode` user[

devcontainer.json42](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/devcontainer.json#L42-L42>)
(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/devcontainer.json#L42-L42>))

Sources: (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L1-L47>).

[.devcontainer/devcontainer.json1-47](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L1-L47) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L1-L47>) [

[.devcontainer/Dockerfile1-26](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/Dockerfile#L1-L26)](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/Dockerfile#L1-L26>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/Dockerfile#L1-L26>))

Code Editing Experience

During development, code quality is enforced in real-time:

1. Editor-level formatting (EditorConfig):

- Applied automatically as developer types
- Enforces LF line endings[

.editorconfig10](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig#L10-L10>)
(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig#L10-L10>))

- Trims trailing whitespace[

.editorconfig12](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig#L12-L12>)
(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig#L12-L12>))

- Maintains consistent indentation[

.editorconfig16-20](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig#L16-L20>)
(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig#L16-L20>))

2. Static analysis (ESLint):

- Runs on file save via `vscode-eslint` extension
- Uses TypeScript-aware parser[

`.eslinttrc.json2](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslinttrc.json#L2-L2)`
`(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslinttrc.json#L2-L2)`)

- Checks against 80+ rules[

`.eslinttrc.json26-79](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslinttrc.json#L26-L79`
`(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslinttrc.json#L26-L79)`)

- Displays inline warnings and errors

3. Pre-commit validation (Husky + lint-staged):

- Intercepts `git commit` command
- Runs ESLint and Prettier on staged files only
- Blocks commit if quality checks fail
- See [Pre-commit Quality Gates](#) for detailed implementation

Sources: `(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig#L1-L24)`

`.editorconfig1-24` (`https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig#L1-L24`)

`.eslinttrc.json1-81](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslinttrc.json#L1-L81`
`(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslinttrc.json#L1-L81)`)

Container User and Permissions

The development container runs as the non-root `vscode` user for security best practices. This is configured via the `remoteUser` setting:

```
"remoteUser": "vscode"
```

All commands execute as this user, including Maven builds, npm installs, and Git operations. SDKMAN and NVM are installed in the `vscode` user's home directory to ensure proper permissions.

Sources: `(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L42-L42)`

`.devcontainer/devcontainer.json42` (`https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L42-L42`)

Docker-in-Docker Support

The container includes Docker-in-Docker capability via the `docker-in-docker` and `docker-from-docker` features. This enables:

- Running Docker commands within the dev container
- Building Docker images for deployment
- Testing Docker Compose configurations
- Running integration tests that require containers

Sources: `(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L43-L46)`

`.devcontainer/devcontainer.json43-46` (`https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L43-L46`)

Summary

The nhsv-admin development environment provides a consistent, fully-configured workspace through:

1. **Containerization:** Docker-based dev container eliminates environment inconsistencies
2. **Dual runtime support:** Java 11 for backend (Spring Boot/JHipster), Node.js LTS for frontend (React)
3. **Integrated tooling:** Maven for Java builds, npm for JavaScript dependencies
4. **Multi-layer quality enforcement:** EditorConfig → ESLint → Pre-commit hooks
5. **IDE integration:** 7 pre-installed VS Code extensions for comprehensive language support
6. **Service access:** Port forwarding for backend (8080, 9000, 9060) and frontend (3001)

This architecture ensures that all developers work in identical environments and that code quality standards are enforced before code enters the repository.

For implementation details of specific components, see the child pages:

- [Container Configuration](#) - Dockerfile and devcontainer.json details
- [Code Quality and Style](#) - EditorConfig and ESLint rule explanations
- [Version Control Configuration](#) - Git configuration for cross-platform compatibility
- [Pre-commit Quality Gates](#) - Husky and lint-staged implementation

Container Configuration

Relevant source files

- [

 .devcontainer/Dockerfile](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/Dockerfile>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/Dockerfile>))
- [

 .devcontainer/devcontainer.json](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json>))

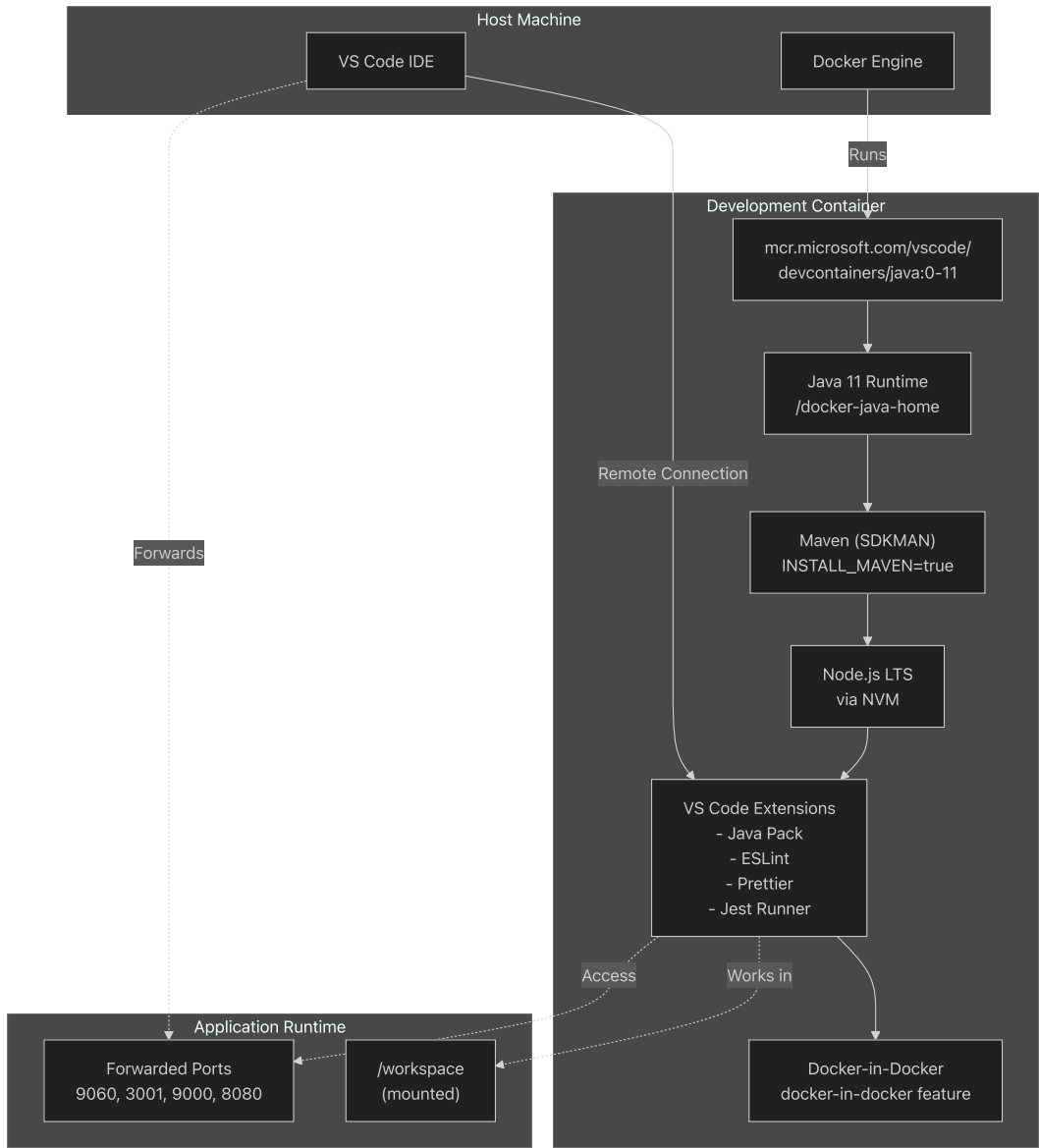
Purpose and Scope

This document details the Docker-based development container configuration that provides a standardized, reproducible environment for developing the nhsv-admin application. The configuration ensures all developers work with consistent versions of Java 11, Node.js, Maven, and development tooling regardless of their host operating system.

For information about code quality tools that run within this container, see [Code Quality and Style](#). For version control configuration, see [Version Control Configuration](#).

Container Architecture Overview

The development container is built on Microsoft's official Java development container images and extends them with Node.js and Maven support. The configuration is split between two files: `Dockerfile` defines the container image layers, and `devcontainer.json` configures VS Code integration.



Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/Dockerfile#L1-L26>)
[.devcontainer/Dockerfile1-26](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/Dockerfile#L1-L26) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/Dockerfile#L1-L26>) [
.devcontainer/devcontainer.json1-48](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L1-L48>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L1-L48>)

Base Image and Java Configuration

The container uses Microsoft's official Java development container as its foundation. The base image is specified using the `VARIANT` build argument, which defaults to Java 11.

Image Selection

Configuration Key	Value
VARIANT	"11"
.devcontainer/devcontainer.json11] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L11-L11)	
Base Image	mcr.microsoft.com/vscode/devcontainers/java:0-\${VARIANT}

.devcontainer/Dockerfile5](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/Dockerfile#L5-L5>) | | Java Home | /docker-java-home | [

.devcontainer/devcontainer.json21](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L21-L21>) |

The `VARIANT` argument supports multiple Java versions (11, 17) and OS variants (bullseye, buster). The configuration explicitly sets Java 11 as required by the JHipster application.



Sources: (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/Dockerfile#L3-L5>)

.devcontainer/Dockerfile3-5 (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/Dockerfile#L3-L5>)

.devcontainer/devcontainer.json8-22](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L8-L22>)

Development Tools Installation

The container installs Maven and Node.js as additional development tools. Both are installed conditionally based on build arguments and use specialized version managers (SDKMAN for Maven, NVM for Node.js).

Maven Installation

Maven is installed via SDKMAN when `INSTALL_MAVEN` is set to `"true"`. The installation runs under the `vscode` user to ensure proper permissions.

Build Argument	Value	Purpose
INSTALL_MAVEN	"true"	Enables Maven installation
MAVEN_VERSION	"" (empty)	Uses latest Maven version
INSTALL_GRADLE	"false"	Gradle not required

The installation command at (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/Dockerfile#L13-L14>)

.devcontainer/Dockerfile13-14 (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/Dockerfile#L13-L14>) uses SDKMAN's `sdk install maven` command, which downloads and configures Maven in the container.

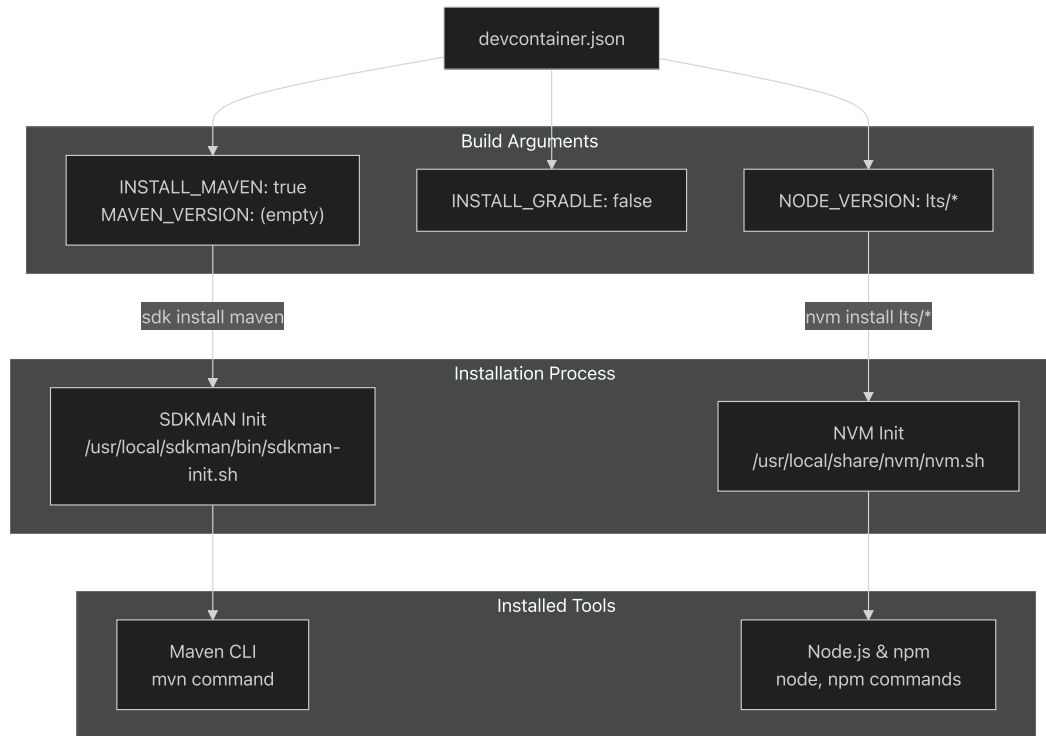
Node.js Installation

Node.js is installed via NVM (Node Version Manager) to provide frontend build capabilities. The configuration specifies the LTS (Long Term Support) version.

Build Argument	Value	Purpose
<code>NODE_VERSION</code>	<code>"lts/*"</code>	Installs latest LTS Node.js

The installation at <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/Dockerfile#L18-L18>

[.devcontainer/Dockerfile18](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/Dockerfile#L18-L18) <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/Dockerfile#L18-L18> uses NVM to download and configure Node.js, making `node` and `npm` commands available in the container.



Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/Dockerfile#L7-L18>

[.devcontainer/Dockerfile7-18](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/Dockerfile#L7-L18) <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/Dockerfile#L7-L18> [

.devcontainer/devcontainer.json13-15](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L13-L15> <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L13-L15>)

VS Code Extensions and Settings

The development container automatically installs a curated set of VS Code extensions to support Java, TypeScript, and code quality tooling. These extensions are specified in the `extensions` array.

Installed Extensions

Extension ID	Purpose
vscjava.vscode-java-pack	Java language support, debugging, testing
pivotal.vscode-boot-dev-pack	Spring Boot development tools
dbaeumer.vscode-eslint	JavaScript/TypeScript linting
esbenp.prettier-vscode	Code formatting
ms-vscode.vscode-typescript-tslint-plugin	TypeScript linting support
firsttris.vscode-jest-runner	Jest test execution
christian-kohler.npm-intellisense	NPM package IntelliSense

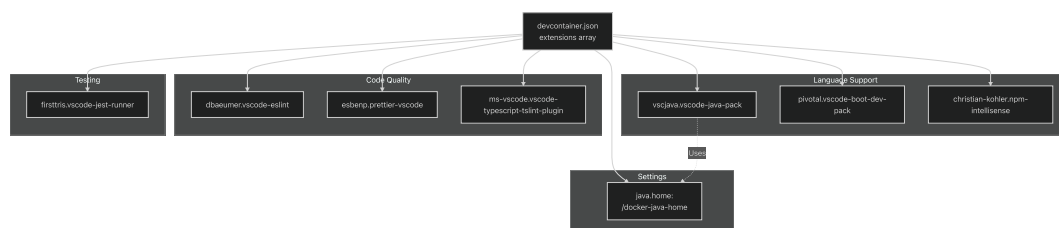
The extensions are defined at <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L25-L33> and are automatically installed when the container is first created.

Container-Specific Settings

The configuration sets the Java home path to ensure VS Code's Java extension correctly locates the JDK:

```
"settings": {
  "java.home": "/docker-java-home"
}
```

This setting at <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L20-L22> points to the Java installation within the container, enabling Java language features like IntelliSense, debugging, and refactoring.



Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L19-L33>

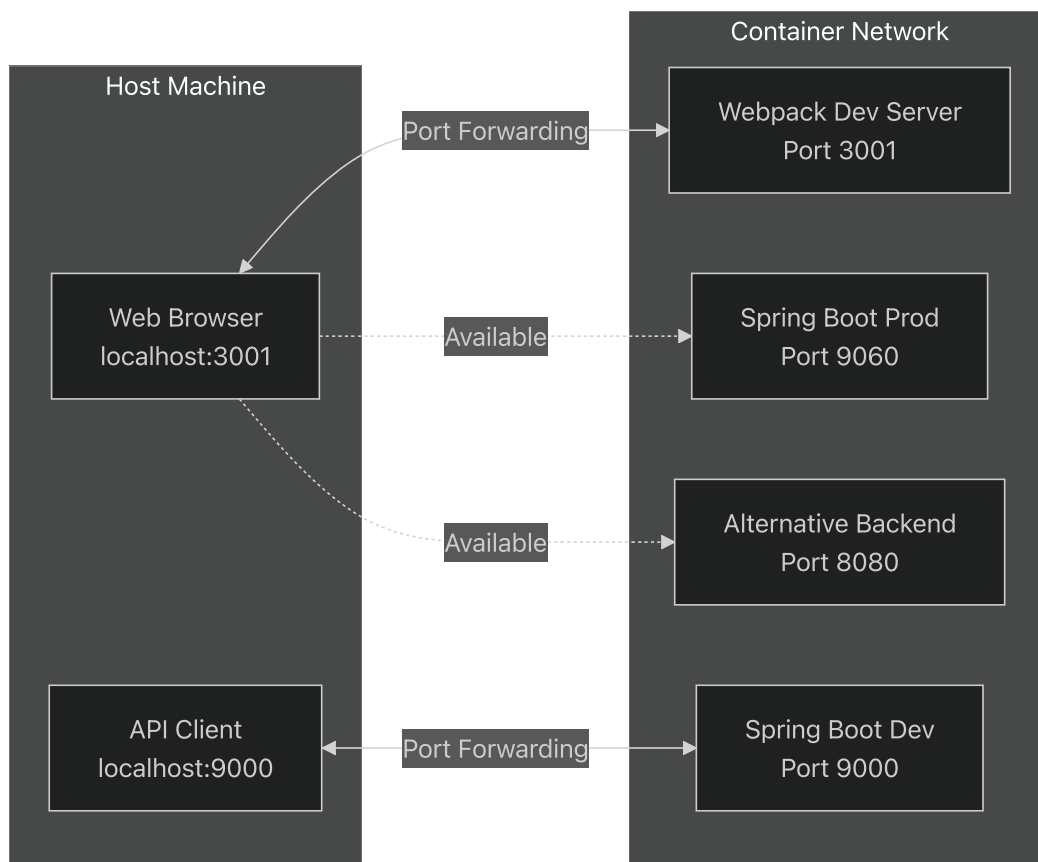
Port Forwarding Configuration

The container forwards four ports from the container to the host machine, enabling access to various application services during development.

Forwarded Ports

Port	Purpose
9060	Backend API (production mode)
3001	Frontend development server (Webpack)
9000	Backend API (development mode)
8080	Alternative backend port

These ports are configured at <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L36-L36> in the `forwardPorts` array. VS Code automatically forwards these ports when the container starts, making the services accessible at `localhost:<port>` on the host machine.



Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L36-L36> in the `forwardPorts` array. VS Code automatically forwards these ports when the container starts, making the services accessible at `localhost:<port>` on the host machine.

Docker-in-Docker Features

The development container includes Docker-in-Docker capabilities, allowing developers to build and run Docker containers from within the development container itself. This is useful for testing containerized components or running additional services.

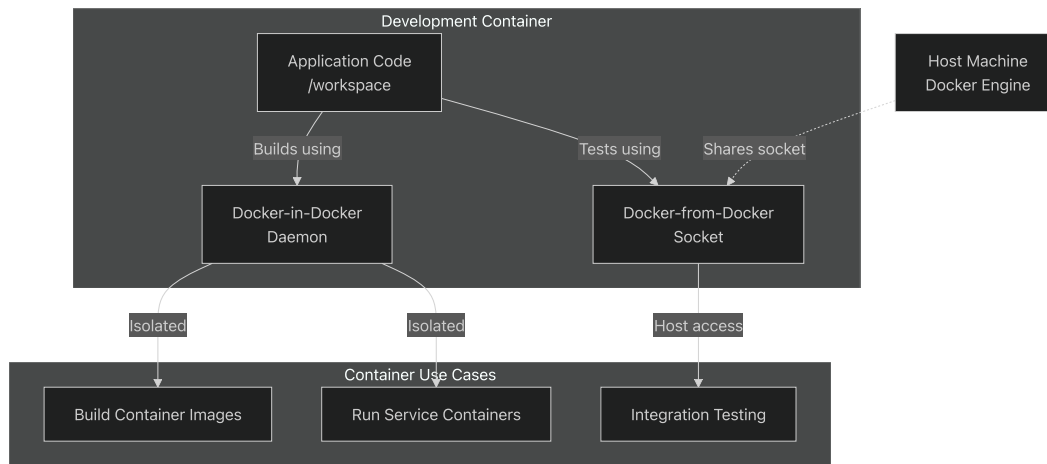
Enabled Features

Feature Name	Purpose
<code>docker-in-docker</code>	Full Docker daemon inside container
<code>docker-from-docker</code>	Access to host Docker daemon

These features are configured at <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L43-L46>.

[.devcontainer/devcontainer.json#L43-L46](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L43-L46) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L43-L46>) and provide two complementary approaches:

- **docker-in-docker:** Runs a complete Docker daemon inside the container, allowing full isolation
- **docker-from-docker:** Shares the host's Docker daemon with the container, enabling interaction with host containers



Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L43-L46>.

[.devcontainer/devcontainer.json#L43-L46](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L43-L46) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L43-L46>)

Remote User Configuration

The container runs with a non-root user named `vscode`, which is specified at <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L42-L42>

[.devcontainer/devcontainer.json#L42-L42](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L42-L42) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L42-L42>) This follows security best practices by avoiding unnecessary root privileges during development.

All tool installations (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/Dockerfile#L13-L18>)

[.devcontainer/Dockerfile#L13-L18](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/Dockerfile#L13-L18) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/Dockerfile#L13-L18>) explicitly use `su vscode -c` to ensure tools are installed under the correct user's home directory, making them accessible without permission issues.

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L42-L42>.

[.devcontainer/devcontainer.json#L42-L42](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L42-L42) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L42-L42>)

[.devcontainer/Dockerfile#L13-L18](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/Dockerfile#L13-L18) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/Dockerfile#L13-L18>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/Dockerfile#L13-L18>)

Summary

The container configuration provides a complete, standardized development environment with:

- **Java 11** runtime via Microsoft's official base image
- **Maven** build tool via SDKMAN
- **Node.js LTS** via NVM for frontend tooling
- **7 VS Code extensions** for Java, Spring Boot, TypeScript, and code quality
- **4 forwarded ports** for accessing application services
- **Docker-in-Docker** support for container operations
- **Non-root user** configuration for security

This setup ensures consistent development environments across all platforms (Windows, macOS, Linux) and eliminates "works on my machine" issues.

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/Dockerfile#L1-L26>)

[.devcontainer/Dockerfile1-26](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/Dockerfile#L1-L26) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/Dockerfile#L1-L26>)

[.devcontainer/devcontainer.json1-48](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L1-L48)) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L1-L48>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.devcontainer/devcontainer.json#L1-L48>)

Code Quality and Style

Relevant source files

- [[.editorconfig](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig)] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig>)
- [[.eslintignore](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintignore)] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintignore>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintignore>)
- [[.eslintrc.json](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json)] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json>)

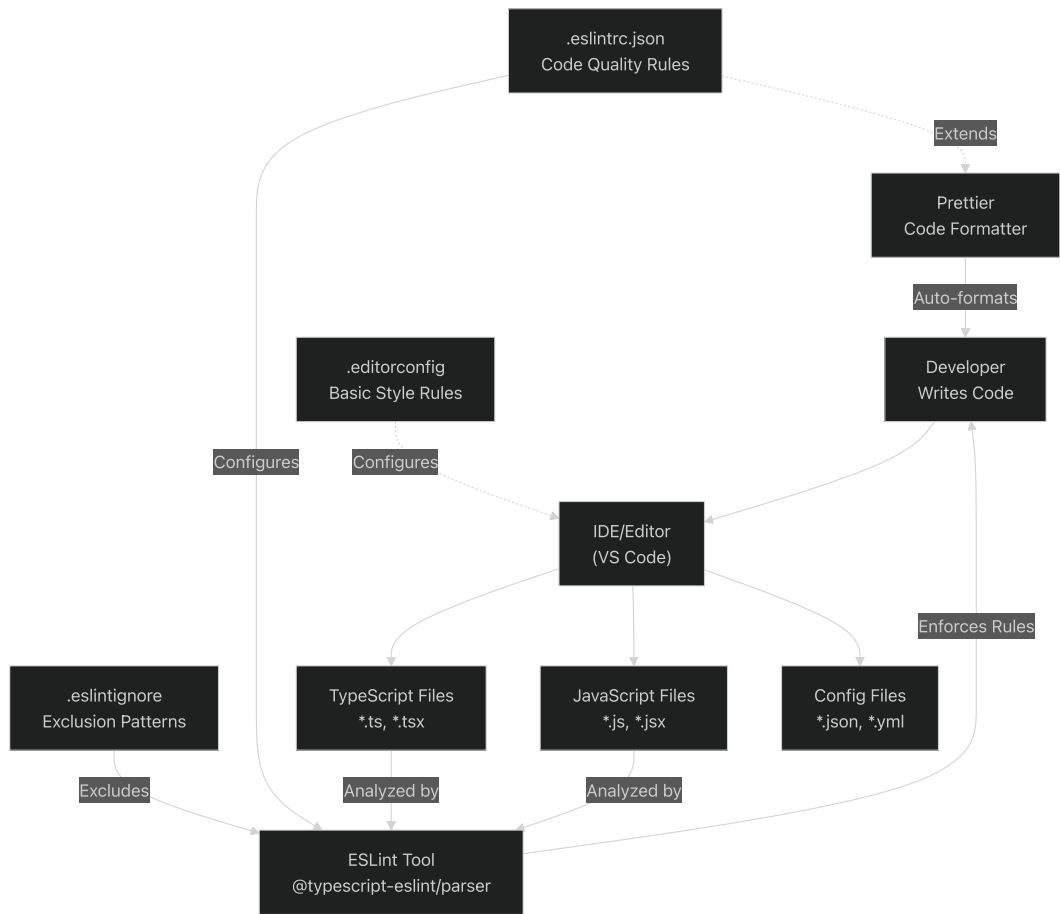
Purpose and Scope

This document details the code quality and style enforcement mechanisms used in the nhsv-admin repository. It covers EditorConfig for basic formatting rules and ESLint for TypeScript/JavaScript code quality checks. These tools establish consistent coding standards across the development team and ensure code adheres to best practices.

For information about how these tools are enforced during the commit process, see [Pre-commit Quality Gates](#). For details on the development container environment where these tools run, see [Container Configuration](#).

Overview of Quality Tools

The codebase employs a layered approach to code quality enforcement, with each tool serving a specific purpose in the quality assurance pipeline.



Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig#L1-L24>)
[.editorconfig1-24](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig#L1-L24)(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig#L1-L24>) [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L1-L81>]([%5B](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L1-L81))
[.eslintrc.json1-81](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L1-L81)](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L1-L81>) [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintignore#L1-L9>]([%5B](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintignore#L1-L9))
[.eslintignore1-9](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintignore#L1-L9)](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintignore#L1-L9>)

EditorConfig - Basic Style Rules

EditorConfig provides IDE-agnostic configuration for fundamental code formatting. The configuration applies universally across different editors and IDEs, ensuring developers see consistent formatting regardless of their tooling choices.

Global Settings

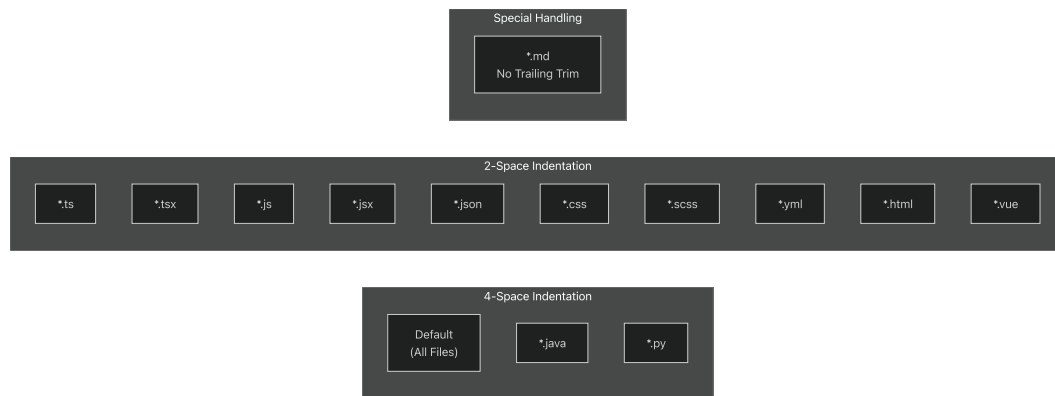
The following settings apply to all files in the repository:

Setting	Value	Purpose
<code>end_of_line</code>	<code>lf</code>	Unix-style line endings for cross-platform consistency
<code>charset</code>	<code>utf-8</code>	Universal character encoding
<code>trim_trailing_whitespace</code>	<code>true</code>	Remove trailing spaces
<code>insert_final_newline</code>	<code>true</code>	Ensure files end with a newline
<code>indent_style</code>	<code>space</code>	Use spaces instead of tabs

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig#L7-L17>

[.editorconfig7-17](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig#L7-L17) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig#L7-L17>)

File-Type Specific Indentation



The configuration defines different indentation sizes based on file type:

- **4 spaces** - Default for most files[

`.editorconfig17` (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig#L17-L17>)
(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig#L17-L17>)

- **2 spaces** - Web and configuration files including TypeScript, JavaScript, JSON, CSS, SCSS, YAML, HTML, and Vue files[

`.editorconfig19-20` (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig#L19-L20>)
(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig#L19-L20>)

- **Markdown exception** - Trailing whitespace is preserved in Markdown files as it can be semantically significant[

`.editorconfig22-23` (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig#L22-L23>)
(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig#L22-L23>)

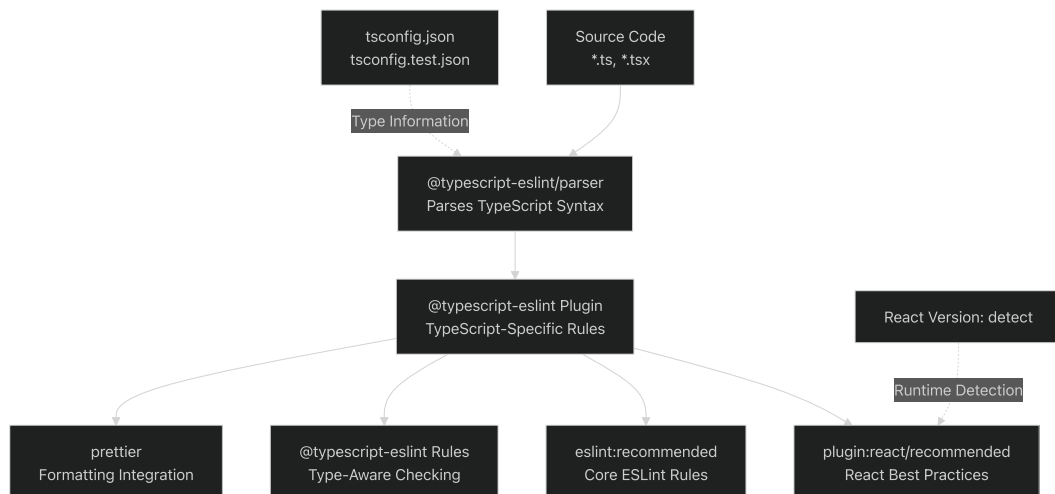
Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig#L15-L24>

[.editorconfig15-24](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig#L15-L24) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig#L15-L24>)

ESLint Configuration

ESLint provides comprehensive static analysis for TypeScript and JavaScript code. The configuration extends multiple recommended rule sets and customizes specific rules for the project's needs.

Parser and Plugin Architecture



The ESLint configuration uses `@typescript-eslint/parser` to parse TypeScript syntax (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L2-L2>)

`.eslintrc.json2` (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L2-L2>) and loads the `@typescript-eslint` plugin for TypeScript-specific linting[

`.eslintrc.json3`](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L3-L3> (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L3-L3>)) The parser is configured with ECMAScript 2018 features and JSX support[

`.eslintrc.json13-18`](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L13-L18> (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L13-L18>))

Type-aware linting is enabled by referencing TypeScript configuration files (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L19-L19>).

`.eslintrc.json19` (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L19-L19>) This allows ESLint to perform sophisticated type-checking-based analysis.

Sources: (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L2-L25>)

`.eslintrc.json2-25` (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L2-L25>).

Extended Rule Sets

The configuration extends six different rule sets in priority order:

1. **plugin:react/recommended** - React-specific best practices[

`.eslintrc.json5`](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L5-L5> (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L5-L5>))

2. **eslint:recommended** - Core ESLint recommended rules[

`.eslintrc.json6`](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L6-L6> (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L6-L6>))

3. **plugin:@typescript-eslint/eslint-recommended** - Disables ESLint rules that conflict with TypeScript[

`.eslintrc.json7`](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L7-L7> (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L7-L7>))

4. **plugin:@typescript-eslint/recommended** - TypeScript recommended rules[

`.eslintrc.json8`](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L8-L8> (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L8-L8>))

5. **plugin:@typescript-eslint/recommended-requiring-type-checking** - Type-aware TypeScript rules[

.eslintrc.json9](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L9-L9>
(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L9-L9>))

6. **prettier** and **eslint-config-prettier** - Disables formatting rules handled by Prettier[

.eslintrc.json10-11](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L10-L11>
(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L10-L11>))

Sources: (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L4-L12>)

[.eslintrc.json4-12](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L4-L12) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L4-L12>).

TypeScript Rule Configuration

The configuration heavily customizes TypeScript rules to balance code quality with development velocity:

Member Ordering Enforcement



The `@typescript-eslint/member-ordering` rule enforces a specific order for class members

(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L27-L32>).

[.eslintrc.json27-32](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L27-L32) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L27-L32>). This creates consistent class structure across the codebase.

Sources: (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L27-L32>).

[.eslintrc.json27-32](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L27-L32) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L27-L32>).

Relaxed Type Safety Rules

Several strict TypeScript rules are disabled to improve developer experience:

Rule	Setting	Rationale
<code>no-unused-vars</code>	<code>off</code>	Disabled to prevent false positives during development[
.eslintrc.json34 (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L34-L34 (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L34-L34))		
<code>explicit-member-accessibility</code>	<code>off</code>	Allows implicit public accessibility[

[.eslintrc.json35](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L35-L35)(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L35-L35>
(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L35-L35>)) | `explicit-function-return-type` |

`off` | Permits type inference for return types[

[.eslintrc.json36](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L36-L36)(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L36-L36>
(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L36-L36>)) | `no-explicit-any` | `off` | Allows any type when necessary[

[.eslintrc.json37](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L37-L37)(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L37-L37>
(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L37-L37>)) | `no-unsafe-argument` | `off` | Permits potentially unsafe arguments[

.eslintrc.json38](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L38-L38>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L38-L38>) | | no-unsafe-return | off | Allows potentially unsafe returns[

.eslintrc.json39](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L39-L39>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L39-L39>) | | no-unsafe-member-access | off | Permits access to any-typed members[

.eslintrc.json40](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L40-L40>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L40-L40>) | | no-unsafe-call | off | Allows calling any-typed functions[

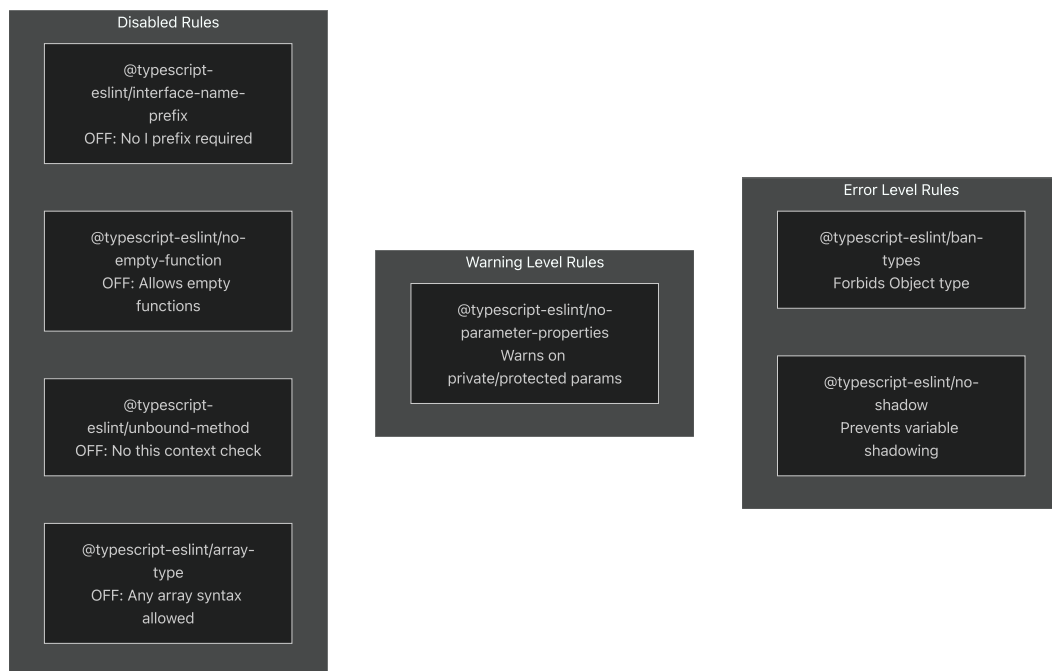
.eslintrc.json41](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L41-L41>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L41-L41>) | | no-unsafe-assignment | off | Permits assignments from any[

.eslintrc.json42](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L42-L42>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L42-L42>) |

Sources: (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L34-L46>).

[.eslintrc.json34-46](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L34-L46) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L34-L46>).

Enforced TypeScript Rules



Key enforced rules include:

- **ban-types** - Prohibits the `Object` type in favor of `{}` [

.eslintrc.json47-54](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L47-L54>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L47-L54>)

- **no-shadow** - Prevents variable shadowing (set to `error`)[

.eslintrc.json59](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L59-L59>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L59-L59>)

- **no-parameter-properties** - Warns when using parameter properties without public/private/protected modifiers[

.eslintrc.json33](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L33-L33>)
(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L33-L33>))

Sources: (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L33-L59>).

[.eslintrc.json33-59](#) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L33-L59>).

General JavaScript/TypeScript Rules

The configuration enforces several language-level best practices:

Code Quality Rules (Error Level)

Rule	Configuration	Description
<code>guard-for-in</code>	<code>error</code>	Requires <code>hasOwnProperty</code> check in for-in loops[
.eslintrc.json61](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L61-L61 (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L61-L61))		
<code>no-labels</code>	<code>error</code>	Disallows labeled statements[

[.eslintrc.json62\]\(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L62-L62>](#) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L62-L62>)) | `no-caller` | `error` | Prohibits arguments.caller/callee[

[.eslintrc.json63\]\(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L63-L63>](#) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L63-L63>)) | `no-bitwise` | `error` | Disallows bitwise operators[

[.eslintrc.json64\]\(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L64-L64>](#) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L64-L64>)) | `no-console` | `error` (allows warn/error) | Prevents console.log but allows console.warn/error[

[.eslintrc.json65\]\(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L65-L65>](#) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L65-L65>)) | `no-new-wrappers` | `error` | Disallows new String/Number/Boolean[

[.eslintrc.json66\]\(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L66-L66>](#) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L66-L66>)) | `no-eval` | `error` | Prohibits eval() usage[

[.eslintrc.json67\]\(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L67-L67>](#) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L67-L67>)) | `no-new` | `error` | Disallows new without assignment[

[.eslintrc.json68\]\(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L68-L68>](#) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L68-L68>)) | `no-var` | `error` | Requires let/const instead of var[

[.eslintrc.json69\]\(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L69-L69>](#) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L69-L69>)) | `radix` | `error` | Requires radix parameter in parseInt[

[.eslintrc.json70\]\(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L70-L70>](#) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L70-L70>)) | `eqeqeq` | `error` | Requires ===

instead of == (except for null)[

.eslintrc.json71](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L71-L71>
(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L71-L71>)) | | prefer-const | error | Requires
const for variables never reassigned[

.eslintrc.json72](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L72-L72>
(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L72-L72>)) | | object-shorthand | error | Requires
ES6 object shorthand[

.eslintrc.json73](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L73-L73>
(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L73-L73>)) | | default-case | error | Requires
default case in switch statements[

.eslintrc.json74](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L74-L74>
(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L74-L74>)) | | complexity | error (max 40) |
Enforces cyclomatic complexity limit[

.eslintrc.json75](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L75-L75>
(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L75-L75>)) |

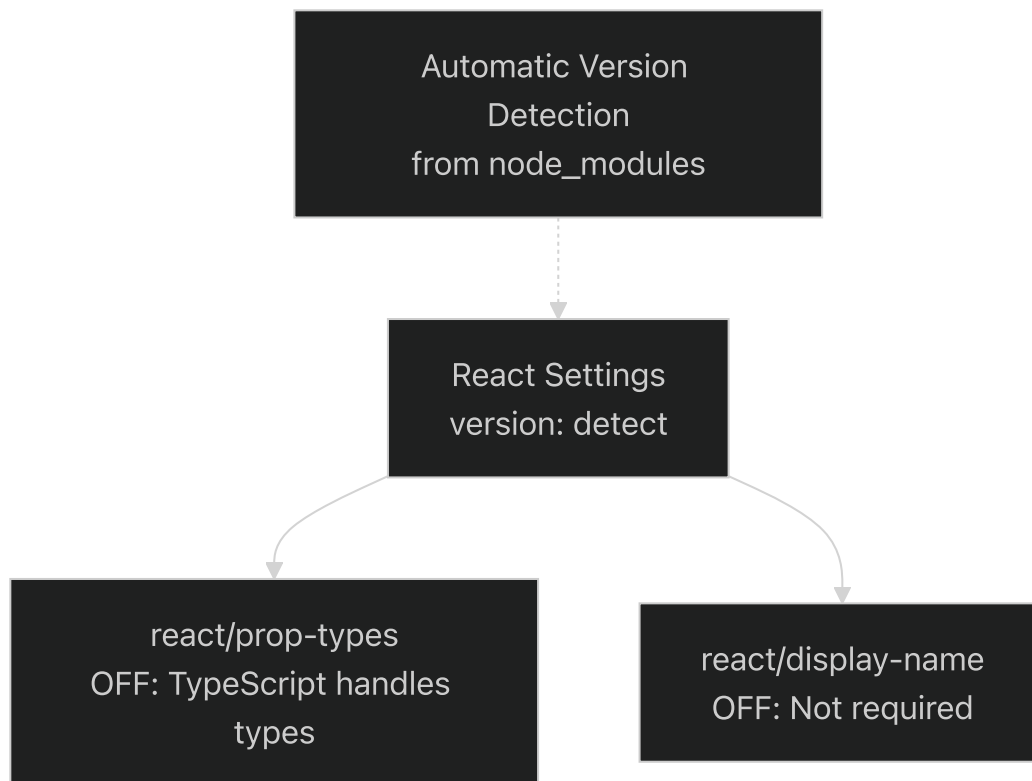
Code Style Rules (Warning Level)

Rule	Configuration	Description
spaced-comment	warn	Requires space after // or /*[
.eslintrc.json60](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L60-L60 (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L60-L60))		

Sources: (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L60-L76>)

[.eslintrc.json60-76](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L60-L76) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L60-L76>)

React-Specific Configuration



React-specific rules are configured to work harmoniously with TypeScript:

- **version: detect** - Automatically detects React version from package.json[.eslintrc.json22-24](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L22-L24> (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L22-L24>))
- **prop-types: off** - Disabled because TypeScript provides type checking[.eslintrc.json77](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L77-L77> (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L77-L77>))
- **display-name: off** - Component display names are not required[.eslintrc.json78](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L78-L78> (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L78-L78>))

Sources: (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L21-L78>).

[.eslintrc.json21-78](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L21-L78) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L21-L78>).

Linting Scope and Exclusions

ESLint is configured to skip certain directories and files that should not be analyzed:

Exclusion Pattern Mapping



The exclusions fall into three categories:

1. **Third-party code** - `node_modules/` contains external dependencies that should not be linted (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintignore#L1-L11>)

[.eslintignore1](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintignore#L1-L1) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintignore#L1-L1)

2. **Build artifacts** - `target/`, `build/`, and `node/` directories contain generated code (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintignore#L5-L7>)

[.eslintignore5-7](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintignore#L5-L7) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintignore#L5-L7)

3. **Infrastructure and configuration** - Files like `jest.config.js`, `postcss.config.js`, Docker configurations, and webpack configurations are excluded (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintignore#L2-L8>)

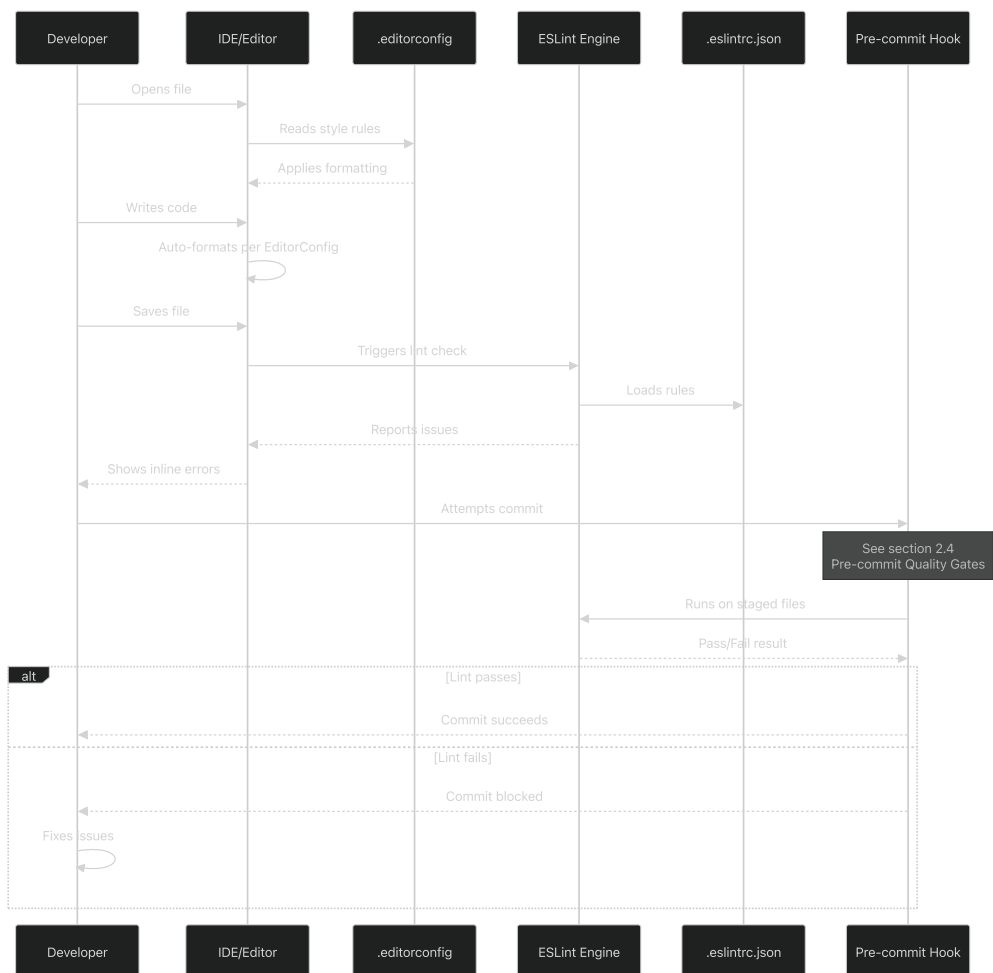
[.eslintignore2-8](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintignore#L2-L8) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintignore#L2-L8)

Sources: (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintignore#L1-L9>)

[.eslintignore1-9](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintignore#L1-L9) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintignore#L1-L9)

Integration with Development Workflow

The code quality tools integrate into the development workflow at multiple stages:



Real-time Feedback

EditorConfig provides immediate formatting feedback as code is written. Most modern editors (VS Code, IntelliJ, etc.) support EditorConfig natively or through plugins, automatically applying the configured style rules

(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig#L1-L5>)

[.editorconfig1-5](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig#L1-L5) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig#L1-L5)

On-Save Analysis

ESLint typically runs when files are saved (if the editor has ESLint integration enabled). The TypeScript parser analyzes the code structure and the configured rules evaluate the code for issues. <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L2-L3>.

[.eslintrc.json2-3](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L2-L3) <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L2-L3>.

Pre-commit Enforcement

The most critical integration point is the pre-commit hook, which prevents code that violates linting rules from being committed. This enforcement mechanism is documented in [Pre-commit Quality Gates](#).

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig#L1-L24>
[.editorconfig1-24](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig#L1-L24) <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.editorconfig#L1-L24> [
.eslintrc.json1-81] <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L1-L81>
<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L1-L81>])

Rule Severity Summary

The following table summarizes the severity levels of enforced rules:

Category	Error Rules	Warning Rules	Disabled Rules
TypeScript	<code>ban-types</code> , <code>no-shadow</code>	<code>no-parameter-properties</code>	15+ type safety rules
JavaScript	<code>no-var</code> , <code>prefer-const</code> , <code>egeqeq</code> , <code>no-eval</code> , <code>no-bitwise</code> , <code>guard-for-in</code> , <code>no-labels</code> , <code>no-caller</code> , <code>no-new-wrappers</code> , <code>no-new</code> , <code>radix</code> , <code>object-shorthand</code> , <code>default-case</code> , <code>complexity</code>	<code>spaced-comment</code>	<code>no-invalid-this</code>
React	N/A	N/A	<code>prop-types</code> , <code>display-name</code>

The configuration prioritizes pragmatic development velocity while maintaining essential code quality standards. Strict type safety rules are relaxed to reduce friction, while fundamental JavaScript best practices (avoiding `eval` , using `const` , etc.) remain strictly enforced.

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L26-L79>
[.eslintrc.json26-79](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L26-L79) <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L26-L79>.

Version Control Configuration

Relevant source files

- [
[.gitattributes](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitattributes)] <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitattributes> <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitattributes>)
- [
[.gitignore](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore)] <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore> <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore>)

Purpose and Scope

This document explains the Git configuration files that ensure consistent version control behavior across different development platforms and environments. The configuration addresses two critical concerns: line ending normalization through `.gitattributes` and file exclusion through `.gitignore`. These configurations support the containerized development environment described in [Container Configuration](#) and work in conjunction with the pre-commit quality gates described in [Pre-commit Quality Gates](#).

Overview

The nhsv-admin repository uses two primary Git configuration files to standardize version control behavior:

Configuration File	Primary Purpose	Key Benefit
<code>.gitattributes</code>	Line ending normalization and file type handling	Cross-platform consistency
<code>.gitignore</code>	Build artifact and IDE file exclusion	Clean repository state

These configurations are particularly important for teams developing across Windows, macOS, and Linux platforms, and for containerized environments where consistent file handling is critical.

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitattributes#L1-L151>)
[.gitattributes1-151](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitattributes#L1-L151) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitattributes#L1-L151>) [[.gitignore1-160](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore#L1-L160)](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore#L1-L160>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore#L1-L160>))

Line Ending Normalization

Automatic Text Detection Strategy

The `.gitattributes` file implements a comprehensive line ending normalization strategy based on the pattern `* text=auto` (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitattributes#L5-L5>). [.gitattributes5](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitattributes#L5-L5) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitattributes#L5-L5>) This directive instructs Git to automatically detect text files and normalize line endings to LF in the repository while allowing platform-appropriate line endings in working directories.



Diagram: Line Ending Normalization Flow

The normalization process ensures that regardless of the platform on which files are edited, they are stored consistently in the repository with LF line endings, except for platform-specific files that require CRLF.

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitattributes#L1-L32>

[.gitattributes1-32](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitattributes#L1-L32) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitattributes#L1-L32>)

File Type Categories

The `.gitattributes` configuration organizes files into distinct categories with specific line ending rules:

Platform-Specific Scripts

Pattern	Line Ending	Lines	Purpose
*.bat	CRLF	10	Windows batch files
*.cmd	CRLF	11	Windows command files
*.ps1	CRLF	12	PowerShell scripts
*.sh	LF	26	Unix shell scripts

These patterns ensure that executable scripts maintain platform-appropriate line endings that are critical for execution.

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitattributes#L10-L26>

[.gitattributes10-26](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitattributes#L10-L26) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitattributes#L10-L26>)

Source Code and Configuration Files

The configuration explicitly marks numerous file types as text to ensure proper line ending normalization:

Category	File Patterns	Line Range
Web Assets	*.css, *.html, *.js, *.ts, *.scss, *.less, *.sass	14, 18, 20, 22, 24, 25, 29
Backend Code	*.java, *.xml, *.properties, *.sql	19, 23, 27, 30
Data Formats	*.json, *.yaml, *.yml, *.cql	21, 31, 32, 15
Templates	*.ejs, *.mustache	17, 49
Documentation	*.md, *.adoc, *.txt	46, 47, 28

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitattributes#L13-L67>

[.gitattributes13-67](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitattributes#L13-L67) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitattributes#L13-L67>)

Binary Files

Binary files are explicitly marked to prevent any line ending manipulation:

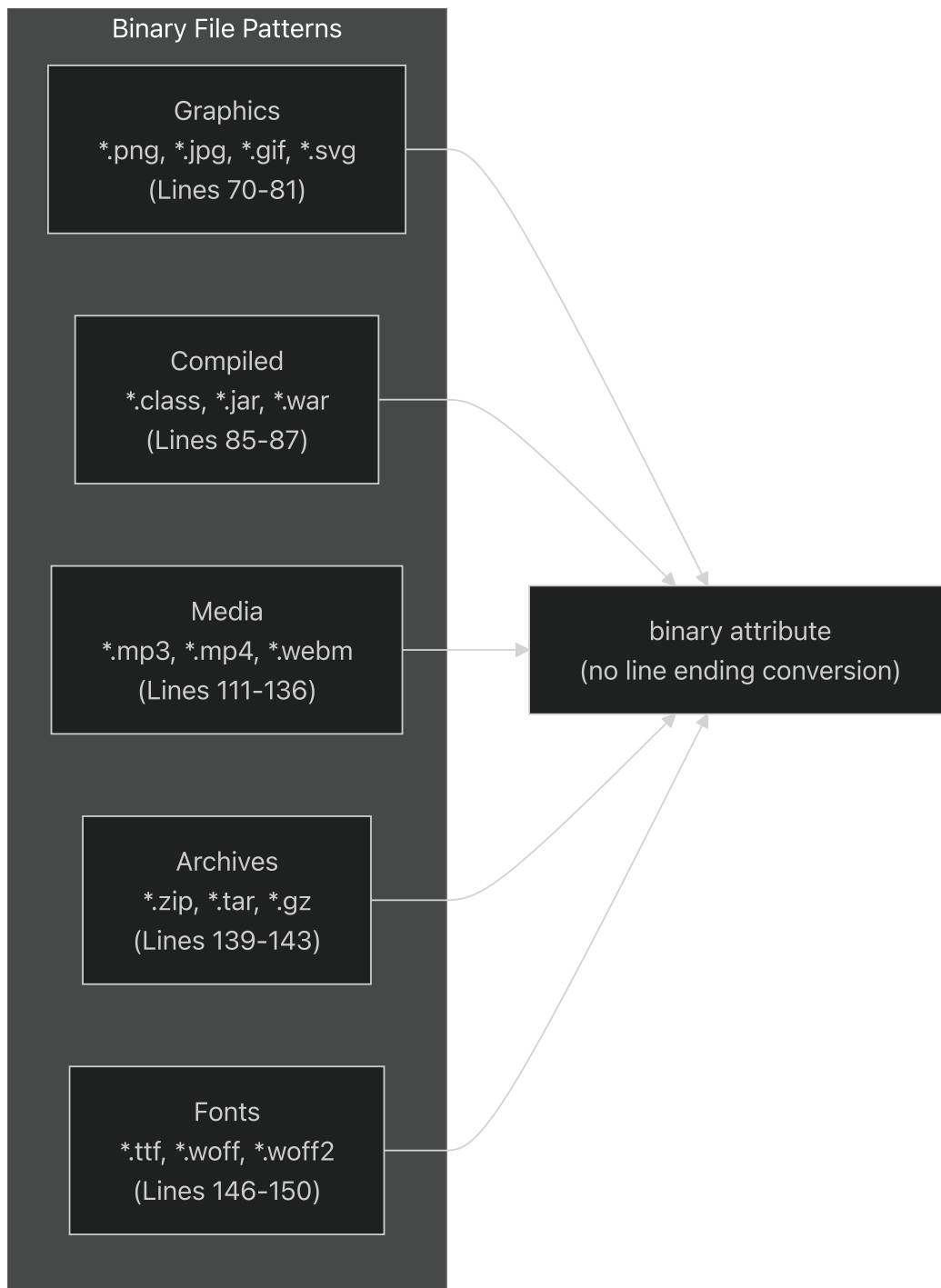


Diagram: Binary File Classification

The `binary` attribute is a macro for `-text -diff`, meaning these files are neither subject to line ending conversion nor shown in text diffs.

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitattributes#L69-L150>.

[.gitattributes69-150](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitattributes#L69-L150) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitattributes#L69-L150>).

Special Configuration Files

Linters and configuration files receive explicit text treatment to ensure consistency:

File Type	Pattern	Line
ESLint	<code>.eslintrc</code>	91
EditorConfig	<code>.editorconfig</code>	100
Git Config	<code>.gitattributes, .gitconfig, .gitignore</code>	101-103
NPM	<code>.npmignore</code>	105

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitattributes#L89-L105>.
[.gitattributes89-105](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitattributes#L89-L105) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitattributes#L89-L105>)

File Exclusion Strategy

Exclusion Categories

The `.gitignore` file organizes exclusions into logical categories to prevent unnecessary files from entering version control:

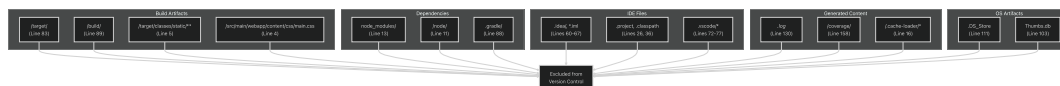


Diagram: File Exclusion Categories

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore#L1-L160>,
[.gitignore1-160](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore#L1-L160) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore#L1-L160>)

Project-Specific Exclusions

The configuration begins with project-specific exclusions that are unique to the nhsv-admin application:

Path	Reason for Exclusion	Line
/src/main/webapp/content/css/main.css	Generated from SCSS compilation	4
/target/classes/static/**	Generated static resources	5
/src/test/javascript/coverage/	Test coverage reports	6

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore#L1-L6>.
[.gitignore#L1-L6](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore#L1-L6) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore#L1-L6>)

Build System Exclusions

Both Maven and Gradle build outputs are excluded:

Build System	Excluded Paths	Line Range
Maven	<code>/target/ , /log/</code>	82-83
Gradle	<code>.gradle/ , /build/</code>	88-89
Package Files	<code>*.jar , *.war , *.ear</code>	94-96

The configuration explicitly allows wrapper files through negation patterns:

```
!gradle/wrapper/gradle-wrapper.jar      # Line 143
!.mvn/wrapper/maven-wrapper.jar        # Line 148
```

These wrapper files are committed to ensure consistent build tool versions across the team.

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore#L80-L148>

[.gitignore80-148](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore#L80-L148) <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore#L80-L148>

Node.js Ecosystem Exclusions

The Node.js dependency and build artifacts are extensively excluded:

Pattern	Purpose	Line
/node/	Node.js installation directory	11
node_modules/	NPM dependencies	13
npm-debug.log.*	NPM error logs	14
/.awcache/*	Angular webpack cache	15
/.cache-loader/*	Webpack cache-loader output	16
.sass-cache/	SASS compilation cache	21

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore#L8-L21>

[.gitignore8-21](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore#L8-L21) <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore#L8-L21>

IDE and Editor Exclusions

The configuration supports multiple development environments by excluding their metadata:

Eclipse

```
.project          # Line 27
.classpath        # Line 36
.settings/        # Line 37
.factorypath      # Line 39
*.launch          # Line 46
```

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore#L24-L56>

[.gitignore24-56](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore#L24-L56) <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore#L24-L56>

IntelliJ IDEA

```
.idea/            # Line 60
*.iml             # Line 61
*.iws             # Line 62
*.ipr             # Line 63
```

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore#L58-L67>

[.gitignore58-67](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore#L58-L67) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore#L58-L67)

Visual Studio Code

The VSCode configuration uses a whitelist approach:

```
.vscode/*                                # Line 72 (exclude all)
!.vscode/settings.json                  # Line 73 (allow settings)
!.vscode/tasks.json                     # Line 74 (allow tasks)
!.vscode/launch.json                    # Line 75 (allow launch)
!.vscode/extensions.json                 # Line 76 (allow extensions)
```

This approach allows shared team configurations while excluding personal workspace files.

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore#L69-L77>

[.gitignore69-77](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore#L69-L77) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore#L69-L77)

Operating System Artifacts

Cross-platform development requires excluding OS-specific metadata:

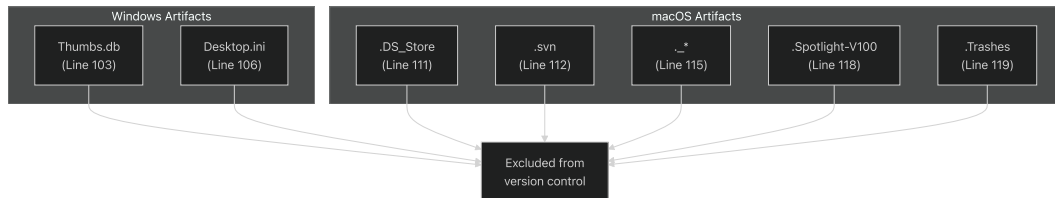


Diagram: Operating System Artifact Exclusions

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore#L99-L119>

[.gitignore99-119](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore#L99-L119) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore#L99-L119)

Development Artifact Exclusions

Additional patterns exclude common development artifacts:

Category	Pattern	Line
Logs	*.log*	130
ESLint Cache	.eslintcache	153
Code Coverage	/coverage/ , /.nyc_output/	158-159
Temporary Files	*.bak , *.swp , *~	32-33, 137
Merge Files	.merge_file*	138

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore#L128-L159>

[.gitignore128-159](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore#L128-L159) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore#L128-L159)

Integration with Development Workflow

Pre-commit Processing

The version control configuration integrates with the pre-commit hooks described in [Pre-commit Quality Gates](#). When files are staged using `git add`, the following sequence occurs:

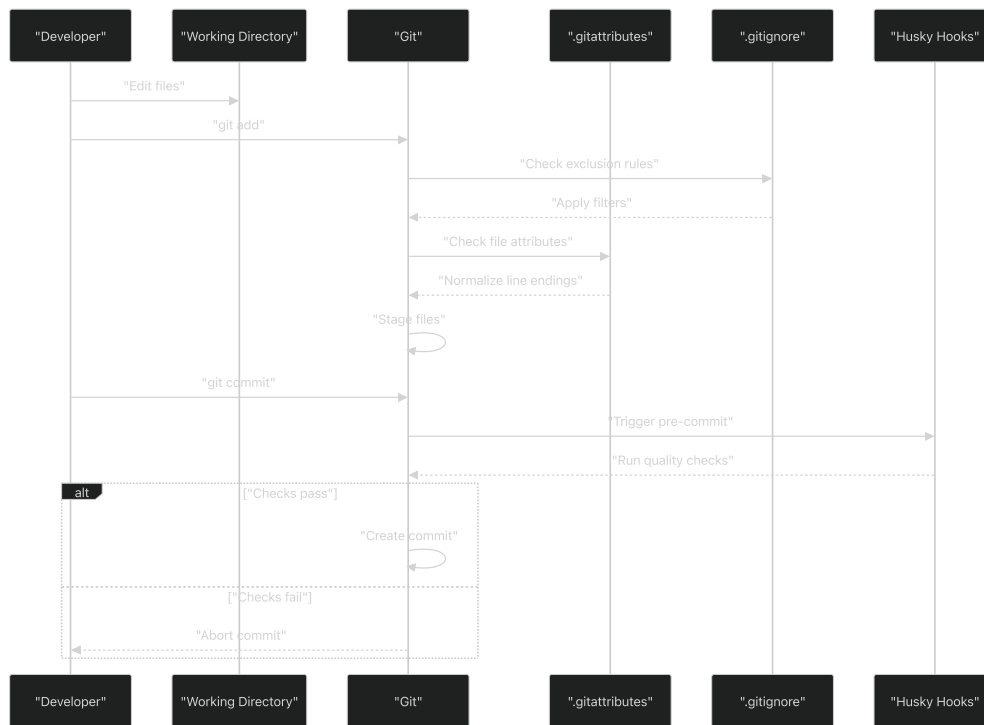


Diagram: Version Control Integration Sequence

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitattributes#L1-L151>

[.gitattributes1-151](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitattributes#L1-L151) <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitattributes#L1-L151>

[.gitignore1-160](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore#L1-L160) <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore#L1-L160>

<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore#L1-L160>

Container Development Support

The configuration particularly supports the containerized development environment defined in [Container Configuration](#):

Container Feature	Configuration Support	Benefit
Docker volumes	Excludes <code>node_modules/</code> , <code>/target/</code>	Faster volume mounting
Cross-platform	Normalizes line endings via <code>.gitattributes</code>	Consistent behavior Windows/Linux
Build isolation	Excludes all build artifacts	Clean rebuild in container
IDE agnostic	Supports Eclipse, IntelliJ, VSCode	Team flexibility

The exclusion of build directories (`/target/`, `/build/`, `node_modules/`) is particularly important for Docker volume performance, as these directories can contain thousands of files that slow down volume synchronization.

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore#L11-L89>

[.gitignore11-89](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore#L11-L89) <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore#L11-L89>

[.gitattributes5-32](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitattributes#L5-L32) <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitattributes#L5-L32>

<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitattributes#L5-L32>

Configuration Maintenance

Adding New File Types

When introducing new file types to the project, developers should update `.gitattributes` with appropriate line ending rules:

1. **Text files:** Add pattern with `text` attribute (e.g., `*.newext text`)
2. **Binary files:** Add pattern with `binary` attribute (e.g., `*.newbin binary`)
3. **Platform scripts:** Specify explicit line endings (e.g., `*.newsh text eol=lf`)

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitattributes#L1-L151>

[.gitattributes1-151](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitattributes#L1-L151) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitattributes#L1-L151>)

Adding New Exclusions

When build processes or tools generate new artifacts:

1. Identify the output directory or file pattern
2. Add to appropriate category in `.gitignore`
3. Test with `git status` to verify exclusion
4. Use `git check-ignore -v <file>` to debug exclusion rules

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore#L1-L160>

[.gitignore1-160](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore#L1-L160) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore#L1-L160>)

Verification

Developers can verify the configuration using Git commands:

```
git check-ignore -v path/to/file

git check-attr -a path/to/file

git ls-files
```

The `git check-attr` command is particularly useful for verifying that text files receive proper line ending normalization based on `.gitattributes` rules.

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitattributes#L1-L151>

[.gitattributes1-151](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitattributes#L1-L151) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitattributes#L1-L151>)

[.gitignore1-160](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore#L1-L160) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.gitignore#L1-L160>)

Pre-commit Quality Gates

Relevant source files

- [[.eslintrc.json](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json>)]
- [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json>]

`.husky/pre-commit`](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.husky/pre-commit>)
(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.husky/pre-commit>)

Purpose and Scope

This document describes the automated code quality enforcement mechanism that intercepts Git commits and validates code before it enters the repository. The pre-commit quality gate system uses Husky to trigger `lint-staged`, which executes ESLint and Prettier checks on staged files. Failed checks abort the commit, ensuring that only code meeting quality standards is committed.

For information about the code style rules enforced by these tools, see [Code Quality and Style](#). For the version control configuration that works alongside these gates, see [Version Control Configuration](#).

Hook Installation and Execution

Husky Pre-commit Hook

The pre-commit hook is installed by Husky and located at <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.husky/pre-commit#L1-L4>.

[.husky/pre-commit1-4](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.husky/pre-commit#L1-L4) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.husky/pre-commit#L1-L4>) This shell script intercepts every `git commit` command and delegates to `lint-staged` for validation.



Diagram: Husky Hook Trigger Chain

The hook script sources the Husky shell helper (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.husky/pre-commit#L2-L2>).

[.husky/pre-commit2](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.husky/pre-commit#L2-L2) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.husky/pre-commit#L2-L2>) to set up the execution environment, then invokes `lint-staged` through the `npmw` wrapper ([

`.husky/pre-commit3`](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.husky/pre-commit#L3-L3>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.husky/pre-commit#L3-L3>)). The `--no-install` flag ensures that dependencies are already present and prevents installation during the commit process.

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.husky/pre-commit#L1-L4>

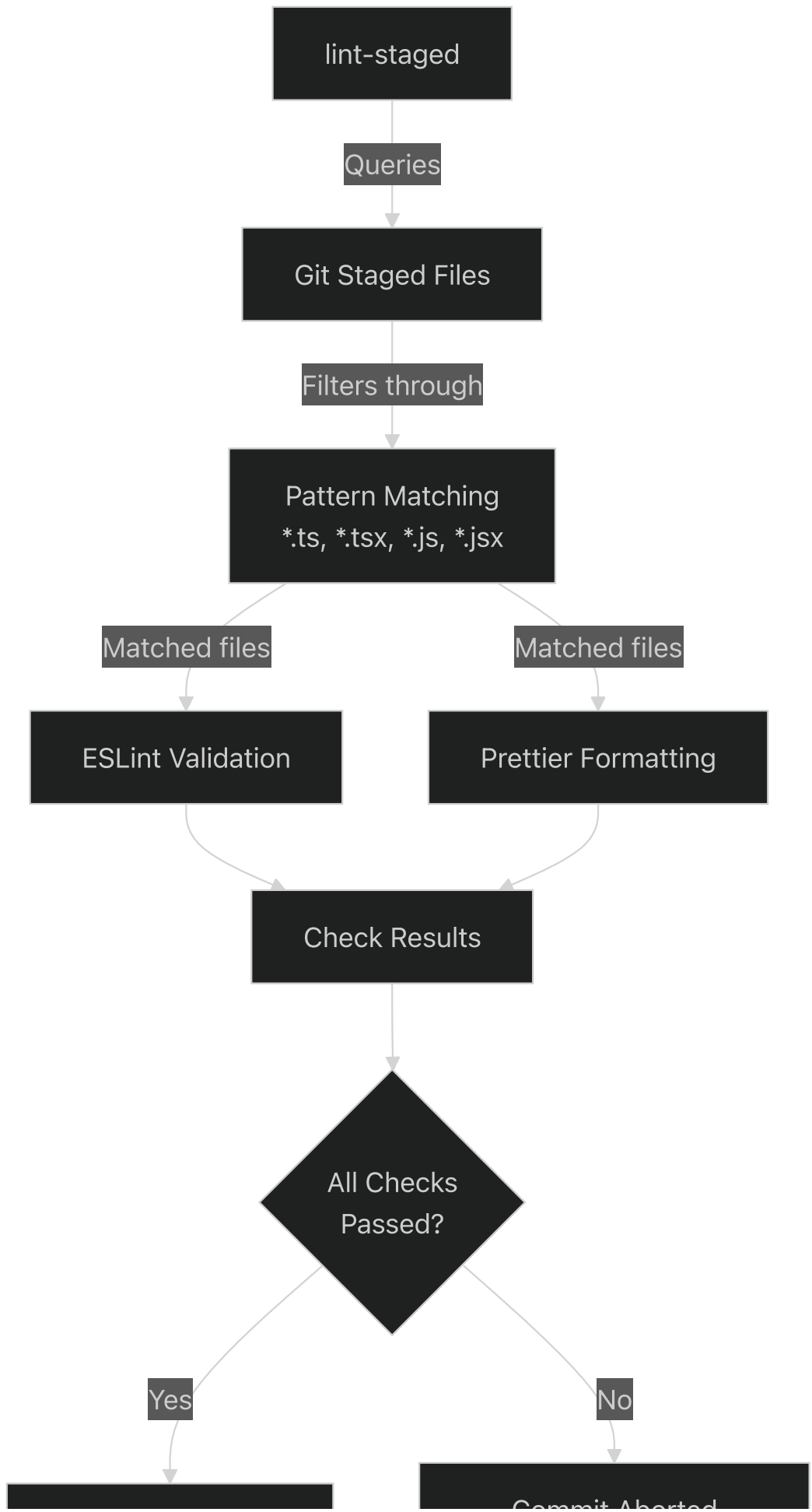
[.husky/pre-commit1-4](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.husky/pre-commit#L1-L4) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.husky/pre-commit#L1-L4>)

Lint-staged Integration

The `lint-staged` tool restricts quality checks to only files that are staged for commit, improving performance and focusing validation on changed code. When invoked by the pre-commit hook, `lint-staged`:

1. Identifies all staged files matching configured patterns
2. Executes configured linters and formatters on matched files
3. Reports results and exits with appropriate status codes
4. Allows commit to proceed only if all checks pass

The typical configuration targets TypeScript and JavaScript files, running both ESLint for code quality and Prettier for formatting consistency.



Commit Proceeds

Commit Aborted
Error Messages Shown**Diagram: Lint-staged Execution Flow****Sources:** <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.husky/pre-commit#L3-L3>[.husky/pre-commit3](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.husky/pre-commit#L3-L3) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.husky/pre-commit#L3-L3>)**ESLint Configuration**The ESLint configuration at <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L1-L81>[.eslintrc.json1-81](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L1-L81) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L1-L81>) enforces code quality standards for TypeScript and React code. The configuration uses the TypeScript ESLint parser and extends multiple rule sets to provide comprehensive code analysis.**Parser and Plugin Configuration**

Configuration Key	Value	Purpose
<code>parser</code>	<code>@typescript-eslint/parser</code>	Parses TypeScript syntax for analysis
<code>plugins</code>	<code>["@typescript-eslint"]</code>	Enables TypeScript-specific linting rules
<code>parserOptions.ecmaVersion</code>	<code>2018</code>	Supports ES2018 syntax features
<code>parserOptions.sourceType</code>	<code>module</code>	Treats files as ES modules
<code>parserOptions.project</code>	<code>["./tsconfig.json", "./tsconfig.test.json"]</code>	Type-aware linting using TypeScript compiler

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L2-L20>[.eslintrc.json2-20](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L2-L20) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L2-L20>)**Extended Rule Sets**The configuration extends multiple base rule sets (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L4-L12>).[.eslintrc.json4-12](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L4-L12) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L4-L12>):

1. `plugin:react/recommended` - React-specific best practices
2. `eslint:recommended` - Core ESLint recommended rules
3. `plugin:@typescript-eslint/eslint-recommended` - Disables ESLint rules conflicting with TypeScript
4. `plugin:@typescript-eslint/recommended` - TypeScript best practices
5. `plugin:@typescript-eslint/recommended-requiring-type-checking` - Type-aware analysis rules
6. `prettier` and `eslint-config-prettier` - Disables formatting rules that conflict with Prettier

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L4-L12>[.eslintrc.json4-12](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L4-L12) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L4-L12>)

Key Enforced Rules

The configuration defines custom rules that override or supplement the extended rule sets:

Code Organization Rules

Rule	Severity	Configuration	Purpose
<code>@typescript-eslint/member-ordering</code>	error	Static fields → Instance fields → Constructor → Methods	Enforces consistent class member ordering
<code>@typescript-eslint/no-shadow</code>	error	N/A	Prevents variable shadowing
<code>complexity</code>	error	Max complexity: 40	Limits cyclomatic complexity

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L27-L32>).

[.eslintrc.json#27-32](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L27-L32) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L27-L32>)

[.eslintrc.json#59](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L59-L59) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L59-L59>)

([%5B](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L59-L59)).

[.eslintrc.json#75](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L75-L75) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L75-L75>)

(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L75-L75>)).

Code Quality Rules

Rule	Severity	Purpose
<code>guard-for-in</code>	error	Requires filtering in for-in loops
<code>no-eval</code>	error	Prohibits eval() usage
<code>no-var</code>	error	Enforces let/const over var
<code>prefer-const</code>	error	Requires const for non-reassigned variables
<code>eql</code>	error	Requires === and !== (except for null)
<code>default-case</code>	error	Requires default case in switch statements

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L61-L74>).

[.eslintrc.json#61-74](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L61-L74) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L61-L74>).

Prohibited Constructs

The following dangerous or legacy constructs are explicitly forbidden (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L62-L68>).

[.eslintrc.json#62-68](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L62-L68) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L62-L68>)).

- `no-labels` - Labels for break/continue statements
- `no-caller` - `arguments.caller` and `arguments.callee`
- `no-bitwise` - Bitwise operators (potential typos for logical operators)
- `no-console` - Console statements except `warn` and `error`

- `no-new-wrappers` - Primitive wrapper constructors (e.g., `new String()`)
- `no-new` - Constructor calls without assignment

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L62-L68>

[.eslintrc.json#62-68](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L62-L68) <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L62-L68>

TypeScript Relaxations

Many strict TypeScript rules are disabled to balance type safety with development velocity (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L34-L58>).

[.eslintrc.json#34-58](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L34-L58) <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L34-L58>):

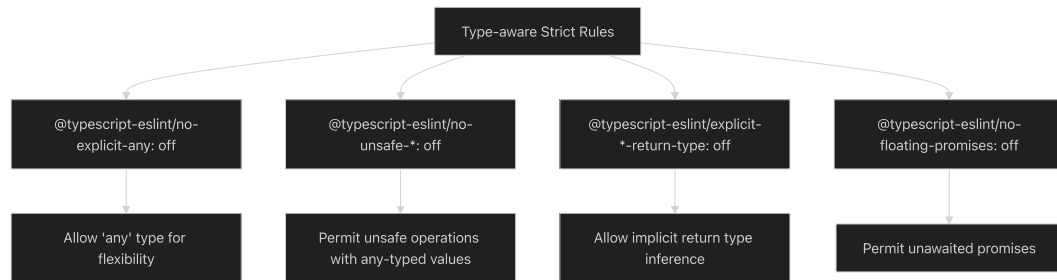


Diagram: TypeScript Rule Relaxations

These relaxations indicate a pragmatic approach prioritizing development speed over maximum type safety, particularly for operations involving dynamically-typed data or external APIs.

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L34-L58>

[.eslintrc.json#34-58](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L34-L58) <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L34-L58>

Quality Gate Workflow

The complete quality gate workflow integrates multiple components from the development environment:

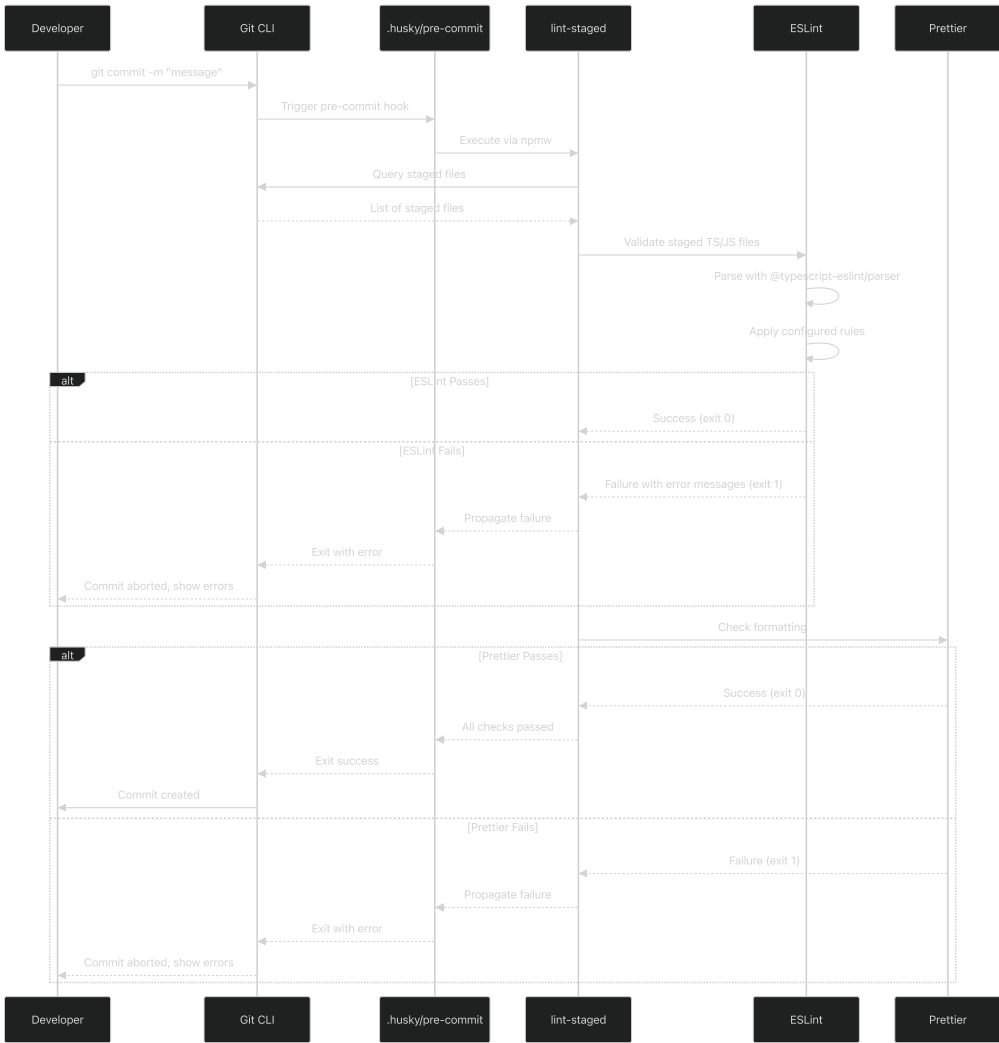


Diagram: Quality Gate Sequence Flow

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.husky/pre-commit#L1-L4>)
[.husky/pre-commit1-4](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.husky/pre-commit1-4) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.husky/pre-commit#L1-L4>) [
.eslintrc.json1-81](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L1-L81>
(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L1-L81>))

Failure Handling

When quality checks fail:

- ESLint Errors:** Specific rule violations are reported with file locations, line numbers, and rule identifiers. Developers must fix code issues before retrying the commit.
- Prettier Errors:** Formatting inconsistencies are reported. Developers can typically auto-fix these by running `npm run prettier:format` or using editor integrations.
- Exit Codes:** Any non-zero exit code from lint-staged causes the pre-commit hook to fail, which in turn causes Git to abort the commit operation.
- Staged Changes Preserved:** Failed commits do not modify staged files, allowing developers to fix issues and retry the commit without re-staging.

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.husky/pre-commit#L1-L4>)
[.husky/pre-commit1-4](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.husky/pre-commit1-4) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.husky/pre-commit#L1-L4>)

Integration with Development Environment

The pre-commit quality gates integrate with other development environment components:

Component	Relationship	Benefit
EditorConfig (2.2)	Provides first-line formatting in editor	Reduces Prettier failures by catching issues during editing
Dev Container (2.1)	Ensures consistent Node.js and npm versions	Guarantees identical linter behavior across developer machines
Git Configuration (2.3)	Handles line ending normalization	Prevents false Prettier failures from CRLF/LF mismatches

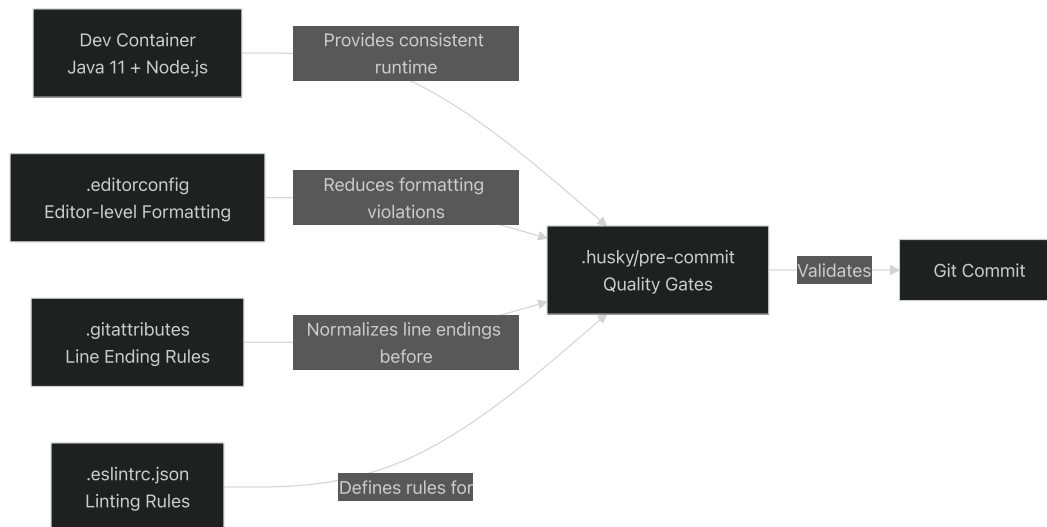


Diagram: Quality Gate Integration Points

This layered approach creates multiple quality checkpoints:

- **Editor level:** EditorConfig and IDE integrations catch issues during typing
- **Pre-commit level:** Husky + lint-staged enforce standards before repository entry
- **Runtime level:** Consistent container environment ensures deterministic behavior

Sources: (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.husky/pre-commit#L1-L4>)

[.husky/pre-commit1-4](#) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.husky/pre-commit#L1-L4>)

[.eslintrc.json1-81](#) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L1-L81>)
(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L1-L81>)

Configuration Files Summary

File	Purpose	Key Settings
<code>.husky/pre-commit</code>	Hook script executed before commits	Runs <code>lint-staged</code> via <code>npmw</code> wrapper
<code>.eslintrc.json</code>	TypeScript/React linting rules	Parser: <code>@typescript-eslint/parser</code>
Extends: React, TypeScript, Prettier		
Complexity limit: 40		

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.husky/pre-commit#L1-L4>)

[.husky/pre-commit-4](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.husky/pre-commit#L1-L4) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.husky/pre-commit#L1-L4>)

`.eslintrc.json`1-81](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L1-L81>
(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.eslintrc.json#L1-L81>))

Domain Model Overview

Overview

Relevant source files

- [`.jhipster/Broker.json`](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json>
(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json>))
- [`.jhipster/ChatRoom.json`](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json>))
- [`.jhipster/CopyPortfolio.json`](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json>))
- [`.jhipster/CopySubscriber.json`](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json>))
- [`.jhipster/CopyTradingOrder.json`](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json>))
- [`.jhipster/InviteUser.json`](<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/InviteUser.json>
(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/InviteUser.json>))

Purpose and Scope

This document provides a high-level overview of the domain model architecture in the nhsv-admin system. It explains how JHipster entity definitions drive the generation of the entire backend stack, describes the four main domain subsystems, and illustrates the relationships between entities.

For detailed information about specific entities within each subsystem, see:

- User and Access Management: [User and Access Management](#)
- Chat Room System: [Chat Room System](#)
- Copy Trading Platform: [Copy Trading Platform](#)
- Supporting Entities: [Supporting Entities](#)

JHipster Entity-Driven Architecture

The nhsv-admin system follows JHipster's entity-driven architecture pattern, where all domain entities are defined declaratively in JSON files located in the `.jhipster/` directory. Each entity definition specifies:

- **Fields:** Data types, validation rules, default values
- **Relationships:** One-to-many, many-to-one, many-to-many associations
- **Generation Options:** DTOs, pagination, filtering, service layer type
- **Database Configuration:** Table names, incremental changelogs

From these JSON definitions, JHipster generates a complete backend stack including:

- JPA entity classes with `@Entity` annotations
- Spring Data JPA repositories
- Service layer implementations (`ServiceImpl` or `ServiceClass`)
- REST controllers with `@RestController`
- DTO classes with MapStruct mappings (when `dto` is configured)
- Liquibase database migration changelogs

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json#L1-L70>

[.jhipster/Broker.json#L1-L70](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json#L1-L70) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json#L1-L70>)

[.jhipster/ChatRoom.json#L1-L86](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L1-L86) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L1-L86>)

[.jhipster/CopyPortfolio.json#L1-L35](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L1-L35) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L1-L35>)

Domain Subsystems Overview

The system is organized into four primary domain subsystems, each addressing a distinct business capability:

Subsystem	Primary Entities	Purpose
User and Access Management	Broker, InviteUser	Manages user authentication, broker roles, and invitation workflows
Chat Room System	ChatRoom, SocialLink, RecentViewChatRoom, CreatedChatRoom	Handles chat room creation, approval, and social media integration
Copy Trading Platform	CopyMarketLeaderDetails, CopyPortfolio, CopySubscriber, CopyTradingOrder	Core business domain for copying market leader portfolios to subscriber accounts
Supporting Entities	Holiday, MarketHistoryJobResult	Provides ancillary services like holiday calendars and market data processing

The Copy Trading Platform represents the core business domain, implementing the complex workflows that enable subscribers to automatically replicate trades from market leaders.

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json#L1-L70>

[.jhipster/Broker.json#L1-L70](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json#L1-L70) <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json#L1-L70>

[.jhipster/ChatRoom.json#L1-L86](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L1-L86) <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L1-L86> <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L1-L86>

[.jhipster/CopyPortfolio.json#L1-L35](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L1-L35) <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L1-L35> <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L1-L35>

[.jhipster/CopySubscriber.json#L1-L72](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json#L1-L72) <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json#L1-L72> <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json#L1-L72>

[.jhipster/CopyTradingOrder.json#L1-L92](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json#L1-L92) <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json#L1-L92> <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json#L1-L92>

Entity Definition Structure

All entity definitions follow a consistent structure within `.jhipster/*.json` files:

Entity Definition Structure

```

├─ name: Entity class name
├─ changelogDate: Liquibase migration timestamp
├─ entityTableName: Database table name (optional, defaults to entity name)
├─ dto: "mapstruct" | "no" (DTO generation strategy)
├─ pagination: "pagination" | "no" (pagination support)
├─ service: "serviceImpl" | "serviceClass" (service layer type)
├─ jpaMetamodelFiltering: true | false (JPA Criteria API filtering)
├─ incrementalChangelog: true | false (incremental Liquibase migrations)
├─ fields: Array of field definitions
|   └─ fieldName: Java property name
|   └─ fieldType: Java type or enum name
|   └─ fieldValues: Enum values (for enum types)
|   └─ fieldValidateRules: ["required", "unique", "maxlength", etc.]
|   └─ defaultValue: Default value (optional)
├─ relationships: Array of relationship definitions
|   └─ relationshipName: Java property name
|   └─ relationshipType: "one-to-many"
|       "many-to-one" | "many-to-many" |
| --- | --- |
|   └─ otherEntityField: Foreign key field (optional) |
|   `` ` |

```

Example: Broker Entity Definition

The `Broker` entity demonstrates key patterns:

- ****Unique Constraints****: `id` field has `required` and `unique` validation
- ****Email Uniqueness****: `email` field marked with `unique` validation
- ****Status Tracking****: Boolean `status` field for active/inactive state
- ****Audit Fields****: `createdAt`, `updatedAt`, `deactivatedAt` timestamps
- ****No DTOs****: `dto: "no"` indicates direct entity exposure via REST
- ****Service Class****: `service: "serviceClass"` generates a simple service wrapper

****Sources****: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.js>)

[.jhipster/Broker.jsonl-70] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.jsonl-70>)

Example: CopyPortfolio Entity Definition

The `CopyPortfolio` entity shows more advanced patterns:

- ****Custom Table Name****: `entityTableName: "t\_copy\_portfolio"` overrides default naming
- ****DTO Mapping****: `dto: "mapstruct"` generates separate DTO classes with MapStruct conversion
- ****Relationships****: Defines `one-to-many` to `copyPortfolioDetails` and `many-to-one` to `copyPortfolio`
- ****Service Implementation****: `service: "serviceImpl"` generates full service implementation
- ****Pagination****: `pagination: "pagination"` enables paginated REST endpoints

****Sources****: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.js>)

[.jhipster/CopyPortfolio.jsonl-35] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.jsonl-35>)

* * *

Entity Relationship Model

High-Level Entity Relationships

![Chart 1] (<nhsv-admin-images/nhsv-admin-section6-1.svg>)

****Diagram 1: Core Entity Relationships and Key Attributes****

This diagram illustrates the primary entity relationships and their key fields. The `User`

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.js>)
 [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json4-60>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/InviteUser.json5-49>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json3-69>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json7-32>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json5-69>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json6-83>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json6-83>)

* * *

Persistence Layer Mapping

JHipster entities are mapped to PostgreSQL database tables following these conventions:

Database Table Naming Patterns

! [Chart 2] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Chart2.svg>)

****Diagram 2: Entity-to-Database Mapping Pipeline****

This diagram shows how entity definitions flow through the JHipster generation pipeline to

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.js>)
 [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json64>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/InviteUser.json5-49>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json73>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json6>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json4>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json4>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json4>)

* * *

Entity Configuration Patterns

Configuration Matrix

Entity	DTO Strategy	Pagination	Service Type	JPA Filtering	Incremental Changes
`Broker`	`no`	`no`	`serviceClass`	✓	✓
`InviteUser`	`no`	`pagination`	`serviceClass`	✓	✓
`ChatRoom`	`no`	`pagination`	`serviceClass`	✓	✓
`CopyPortfolio`	`mapstruct`	`pagination`	`serviceImpl`	✓	✓
`CopySubscriber`	`mapstruct`	`pagination`	`serviceImpl`	✓	✓
`CopyTradingOrder`	`mapstruct`	`pagination`	`serviceImpl`	✓	✓

****Pattern Analysis**:**

- **DTO Strategy**:** Copy trading entities (`CopyPortfolio`, `CopySubscriber`, `CopyTradingOrder`) use `mapstruct` for DTOs.
- **Service Layer**:** Copy trading entities use `serviceImpl` for full business logic implementation.

```
3.  **Pagination**: All entities except `Broker` support paginated REST endpoints, reflect:

4.  **Universal Features**: All entities enable JPA Metamodel filtering (for dynamic query

**Sources**: [] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.js
[.jhipster/Broker.json63-68] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhip
.jhipster/InviteUser.json51-57] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.j
.jhipster/ChatRoom.json71-84] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhi
.jhipster/CopyPortfolio.json3-34] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.
.jhipster/CopySubscriber.json2-71] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647,
.jhipster/CopyTradingOrder.json2-91] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf906

* * *

#### Field Validation Patterns

Entities use declarative validation rules in their field definitions:

##### Common Validation Rules

| Validation Rule | Purpose | Example Usage |
| --- | --- | --- |
| `required` | Field cannot be null | `Broker.id`, `CopyPortfolio.createdAt` |
| `unique` | Field value must be unique across all records | `Broker.id`, `Broker.email`
| `maxlength` | String field maximum length constraint | `CopySubscriber.accountNumber` (2
| `min` / `max` | Numeric range constraints | (not shown in provided entities) |

##### Enum Field Types

Several entities use enum types for constrained value sets:

**`ChatRoom` Status Workflow**:
```

StatusEnum: PENDING → APPROVED | REJECTED → CREATED → UPDATED ActionEnum: CREATE

UPDATE	DELETE
--------	--------

(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L30-L31>).
[.jhipster/ChatRoom.json30-31](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L30-L31) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L30-L31>) [
.jhipster/ChatRoom.json66-68]([https://github.com/nhsv-tradex/nhsv-](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L66-L68)
[admin/blob/2cf90647/.jhipster/ChatRoom.json#L66-L68](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L66-L68) ([https://github.com/nhsv-tradex/nhsv-](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L66-L68)
[admin/blob/2cf90647/.jhipster/ChatRoom.json#L66-L68](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L66-L68)))

InviteUser Status Lifecycle:

```
InviteStatusEnum: PENDING → EXPIRED | ACCOUNT_CREATED → ACCOUNT_DEACTIVATED
```

(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/InviteUser.json#L15-L16>).
[.jhipster/InviteUser.json15-16](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/InviteUser.json#L15-L16) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/InviteUser.json#L15-L16>).

CopySubscriber Order Set Types:

```
OrderSetTypeEnum: QUICK_MATCH | SAFE_MATCH
```

(<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json#L30-L32>).

[.jhipster/CopySubscriber.json30-32](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json#L30-L32) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json#L30-L32>).

CopyTradingOrder Order Types:

```
SellBuyTypeEnum: BUY | SELL
ExchangeTypeEnum: HOSE
| HNX | UPCOM |
| --- | --- |
| ' ' ' |
```

[] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json#L30-L32>).

[.jhipster/CopyTradingOrder.json31-44] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json#L31-L33>).

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L29-L31>).

[.jhipster/ChatRoom.json29-69] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L29-L31>).

[.jhipster/InviteUser.json14-16] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/InviteUser.json#L14-L16>).

[.jhipster/CopySubscriber.json30-33] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json#L30-L32>).

[.jhipster/CopyTradingOrder.json31-44] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json#L31-L33>).

* * *

Audit and Temporal Fields

Most entities include temporal tracking fields for audit purposes:

Standard Audit Pattern

createdAt: ZonedDateTime // Entity creation timestamp updatedAt: ZonedDateTime // Last modification timestamp createdBy: String // User who created the entity

Extended Audit Fields

`Broker` Entity:

- `deactivatedAt`: When broker was deactivated
- `deactivatedBy`: Who deactivated the broker
- `invitedBy`: Who invited this broker

[] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json#L42-L59>)

[.jhipster/Broker.json42-59] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json#L42-L59>)

`ChatRoom` Entity:

- `approvedAt` / `approvedBy`: Approval workflow tracking
- `rejectedAt` / `rejectedBy`: Rejection workflow tracking
- `rejectReason`: Explanation for rejection

[] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L46-L63>)

[.jhipster/ChatRoom.json46-63] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L46-L63>)

`InviteUser` Entity:

- `activationKey`: Unique key for account activation
- `activationDate`: When invitation was activated

[] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/InviteUser.json#L35-L42>)

[.jhipster/InviteUser.json35-40] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/InviteUser.json#L35-L42>)

These temporal fields enable comprehensive audit trails and support workflow state transitions.

Sources:[] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json#L42-L59>)

[.jhipster/Broker.json42-59] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json#L42-L59>)

[.jhipster/ChatRoom.json39-63] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L39-L63>)

[.jhipster/InviteUser.json19-40] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/InviteUser.json#L19-L40>)

* * *

Relationship Definitions

Entity relationships are defined in the `relationships` array of each entity definition:

Relationship Type Patterns

One-to-Many Relationship (`ChatRoom` → `SocialLink`):

```
{
  "relationshipName": "socialLink",
  "relationshipType": "one-to-many",
  "otherEntityName": "socialLink",
  "otherEntityRelationshipName": "chatRoom"
}
```

```
[.] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L77-L82)
[.jhipster/ChatRoom.json77-82] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhi:

This generates a `List<SocialLink> socialLink` field in the `ChatRoom` entity, and a `ChatR

**Many-to-One Relationship** (`CopyPortfolio` → `User`):
```

```
{
  "relationshipName": "mlUserId",
  "relationshipType": "many-to-one",
  "otherEntityName": "user",
  "otherEntityField": "login"
}
```



```
[ ] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L28)
[.jhipster/CopyPortfolio.json28-32] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L32)
```

This creates a foreign key relationship where multiple portfolios can belong to one user (r

```
**Sources**:[ ] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L76)
[.jhipster/ChatRoom.json76-83] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L83)
.jhipster/CopyPortfolio.json20-33] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L33)
.jhipster/CopySubscriber.json61-70] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json#L70)
```

* * *

Summary

The nhsv-admin domain model is built on JHipster's entity-driven architecture, where JSON c

1. ****User and Access Management**** - Authentication, broker roles, and invitations
2. ****Chat Room System**** - Social features with approval workflows
3. ****Copy Trading Platform**** - Core business domain for portfolio copying
4. ****Supporting Entities**** - Holiday calendars and job tracking

All entities follow consistent patterns for:

- Field validation and constraints
- Audit and temporal tracking
- DTO vs. direct entity exposure
- Service layer implementations
- Pagination and filtering support

The Copy Trading Platform entities (`CopyPortfolio`, `CopySubscriber`, `CopyTradingOrder`)

For detailed information about specific entities and their business logic, refer to the sub

```
**Sources**:[ ] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json#L1)
[.jhipster/Broker.json1-70] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json#L70)
.jhipster/ChatRoom.json1-86] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L86)
.jhipster/CopyPortfolio.json1-35] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json#L35)
.jhipster/CopySubscriber.json1-72] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json#L72)
.jhipster/CopyTradingOrder.json1-92] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json#L92)
.jhipster/InviteUser.json1-58] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/InviteUser.json#L58)
```

User and Access Management

Relevant source files

- [
 - .jhipster/Broker.json] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json#L1>)
- [
 - .jhipster/InviteUser.json] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/InviteUser.json#L58>)

Purpose and Scope

This document describes the User and Access Management subsystem within the nhsv-admin application.

For information about the copy trading system where brokers may serve as market leaders or followers.

This page provides an architectural overview of the subsystem. For detailed documentation, see the following links:

- [Broker Entity] (#broker-entity) - Comprehensive details on broker account structure and management.
- [InviteUser Entity] (#inviteuser-entity) - Detailed invitation workflow and status management.

Sources: [1] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json>)

[2] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/InviteUser.json>)

[3] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/InviteUser.json>)

* * *

Subsystem Architecture

The User and Access Management subsystem follows JHipster's entity-driven architecture pattern.

![[Chart 1] (nhsv-admin-images/nhsv-admin-section7-1.svg)]

Diagram: User and Access Management Subsystem Architecture

The subsystem implements a two-stage user lifecycle:

1. **Invitation Stage:** New users are registered as `InviteUser` entities with status `PENDING`.
2. **Active Stage:** Upon activation, users transition to `Broker` entities with full platform access.

Sources: [1] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json>)

[2] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/InviteUser.json>)

[3] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/InviteUser.json>)

* * *

Entity Overview

The following table summarizes the entities in this subsystem and their primary responsibilities.

Entity	JHipster Definition	Primary Purpose	Pagination	Filtering
Broker	[1] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json)	Manage broker accounts and trading activities.	Yes	Yes
InviteUser	[2] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/InviteUser.json)	Manage user invitations and activation process.	Yes	Yes

[3] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/InviteUser.json>)

Both entities are generated with `serviceClass` implementation, meaning they include dedicated service classes for business logic.

Sources: [1] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json>)

[2] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/InviteUser.json>)

[3] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/InviteUser.json>)

* * *

Entity Relationship Mapping

![Chart 2] (nhsv-admin-images/nhsv-admin-section7-2.svg)

****Diagram: Entity Relationship Diagram - Code Entity Mapping****

This diagram shows the logical relationship between `InviteUser` and `Broker`, though it should be noted that the diagram is not explicitly shown in the provided image.

****Key Field Mappings:****

- `InviteUser.email` → `Broker.email` (upon activation)
- `InviteUser.login` → `Broker.username` (upon activation)
- `InviteUser.createdBy` → `Broker.invitedBy` (tracks invitation chain)

****Sources:**** [(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json4-60)] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json4-60)

[(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/InviteUser.json4-49)] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/InviteUser.json4-49)

[(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/InviteUser.json4-49)] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/InviteUser.json4-49)

* * *

Broker Entity Structure

The `Broker` entity represents active broker accounts within the system. It includes comprehensive fields for identity, status, and lifecycle tracking.

Core Identity Fields

Field	Type	Validation	Purpose
id	Long	Required, Unique	Primary key for broker identification
username	String	Required	Login username
fullname	String	Required	Display name
email	String	Unique	Contact email and unique identifier

Status and Activity Tracking

Field	Type	Default	Purpose
status	Boolean	false	Active/inactive flag
totalChatRoom	Long	0	Count of chat rooms created by this broker
currentRank	Integer	1	Broker ranking metric
isDynamic	Boolean	false	Dynamic status indicator

Lifecycle Timestamps

Field	Type	Purpose
createdAt	ZonedDateTime	Initial account creation timestamp
updatedAt	ZonedDateTime	Last modification timestamp
deactivatedAt	ZonedDateTime	Account deactivation timestamp (null if active)

Audit Fields

Field	Type	Purpose
deactivatedBy	String	Username of administrator who deactivated the account
invitedBy	String	Username of broker who sent the invitation

****Sources:**** [(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json4-60)] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json4-60)

[(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json4-60)] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json4-60)

* * *

InviteUser Entity Structure

The `InviteUser` entity manages the invitation workflow from initial invitation through activation.

Identity and Contact

Field	Type	Purpose
login	String	Proposed username for the invited user
email	String	Email address where invitation is sent

Status Lifecycle

The `status` field uses the `InviteStatusEnum` enumeration with four possible states:

Status Value	Description	Terminal State
PENDING	Invitation sent, awaiting activation	No
EXPIRED	Activation period elapsed without action	Yes
ACCOUNT_CREATED	User successfully activated and broker created	Yes
ACCOUNT_DEACTIVATED	Associated broker account was subsequently deactivated	Yes

Activation Mechanism

Field	Type	Purpose
activationKey	String	Unique key sent in invitation email for verification
activationDate	ZonedDateTime	Timestamp when user completed activation

Audit and Configuration

Field	Type	Purpose
createdAt	ZonedDateTime	Invitation creation timestamp
updatedAt	ZonedDateTime	Last status update timestamp
createdId	Long	ID of the broker who created the invitation
createdBy	String	Username of the broker who created the invitation
authorities	String	Role permissions to be assigned upon activation
langKey	String	Language preference for the invited user

Sources: [1] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/InviteUser.json>) [2] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/InviteUser.json#L49>) [3] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/InviteUser.json#L49>)

* * *

User Invitation and Activation Flow

![Chart 3] (nhsv-admin-images/nhsv-admin-section7-3.svg)

Diagram: InviteUser Status State Machine

Invitation Creation Process

- Existing broker creates invitation via admin interface
- System generates `InviteUser` record with `status=PENDING`
- Unique `activationKey` is generated and stored
- `createdBy` and `createdId` fields capture the inviting broker
- Email containing activation link is sent to `email` address

Activation Process


```

This generates metamodel classes that enable type-safe, dynamic filtering on REST API endpoints.

**Sources:**[] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json52) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json63) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/InviteUser.json52) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/InviteUser.json65)

##### Pagination Support

| Entity | Pagination Enabled | Configuration |
|---|---|---|
| `Broker` | No | `"pagination": "no"` |
| `InviteUser` | Yes | `"pagination": "pagination"` |

The `InviteUser` entity supports pagination due to the expectation that invitation lists may be large.

**Sources:**[] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json52) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json65) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/InviteUser.json54) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/InviteUser.json65)

##### Incremental Changelog

Both entities use incremental Liquibase changelogs:

```

"incrementalChangelog": true

This means schema modifications to these entities generate separate changelog files rather

Sources: (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.js) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json62) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/InviteUser.json51) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/InviteUser.json62)

* * *

Database Schema Mapping

The JHipster entity definitions map to PostgreSQL tables with the following naming convention:

Entity	Table Name	Primary Key	Unique Constraints
Broker	t_broker	id	email
InviteUser	t_invite_user	id	None explicitly defined

The `Broker.email` field has a unique constraint, ensuring no duplicate email addresses across the system.

Sources: (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.js) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json5-39) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/InviteUser.json) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/InviteUser.json62)

* * *

Integration Points

Authentication System Integration

The `Broker` entity integrates with JHipster's built-in authentication system through the following fields:

- Login credentials
- Email address
- Authority assignments (from `InviteUser.authorities`)
- Language preference (from `InviteUser.langKey`)

Chat Room System Integration

Brokers can create and manage chat rooms. The `Broker.totalChatRoom` field maintains a count of chat rooms created by the broker.

Sources: (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.js) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.json24-27) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/InviteUser.json) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/InviteUser.json62)

Copy Trading Platform Integration

Brokers may participate in the copy trading platform as market leaders or subscribers. The `Broker.copyTrading` field indicates their role.

* * *

Validation Rules

Broker Validation

- `id`: Required, Unique
- `email`: Unique (prevents duplicate accounts)

All other fields are optional at the database level, though application logic may enforce additional constraints.

```

**Sources:** [(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.js
[.jhipster/Broker.json5-39] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipst

##### InviteUser Validation

No explicit validation rules are defined in the JHipster entity definition. Validation is 1

- Email format validation
- Activation key uniqueness
- Status transition rules
- Expiration period enforcement

**Sources:** [(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/InviteUser
[.jhipster/InviteUser.json] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipst

* * *

#### Service Layer Architecture

Both entities specify `"service": "serviceClass"`, generating dedicated service implementat

![[Chart 5] (nhsv-admin-images/nhsv-admin-section7-5.svg)

**Diagram: Service Layer Architecture - Code Entity Structure**

This three-tier architecture provides:

1. **REST Layer**: Handles HTTP requests, validation, and response formatting
2. **Service Layer**: Implements business logic, transaction management, and status trans
3. **Repository Layer**: Provides data access through Spring Data JPA

The service layer is where invitation activation logic, deactivation workflows, and status

**Sources:** [(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.js
[.jhipster/Broker.json68] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipst
.jhipster/InviteUser.json57] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipst

* * *

#### Summary

The User and Access Management subsystem provides a robust foundation for broker account 1:

- **Auditable invitation workflow** with status tracking and expiration handling
- **Flexible account lifecycle** including creation, activation, and deactivation
- **Integration foundation** for chat room management and copy trading participation
- **JHipster-generated full-stack** with consistent service layer, repository access, and

For implementation details on individual entities, refer to:

- [Broker Entity] (#broker-entity) for broker-specific functionality
- [InviteUser Entity] (#inviteuser-entity) for invitation workflow details

**Sources:** [(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Broker.js
[.jhipster/Broker.json] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/I
.jhipster/InviteUser.json] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipst

```



```
---
```

Broker Entity

Relevant source files

```
- [
    .jhipster/ChatRoom.json] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json)
- [
    .jhipster/CreatedChatRoom.json] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CreatedChatRoom.json)
- [
    .jhipster/RecentViewChatRoom.json] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/RecentViewChatRoom.json)
- [
    .jhipster/SocialLink.json] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/SocialLink.json)
```

Purpose and Scope

The Chat Room System manages the lifecycle of chat rooms within the NHSV Admin platform, including creation, approval, and user interaction.

For information about user authentication and broker management, see [User and Access Management].

* * *

Entity Architecture Overview

The Chat Room System consists of four primary entities that work together to provide chat room functionality:

****Entity Relationship Diagram****

! [Chart 1] (nhsv-admin-images/nhsv-admin-section8-1.svg)

****Sources:**** [(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L1-86) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CreatedChatRoom.json) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CreatedChatRoom.json#L1-72) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/RecentViewChatRoom.json) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/RecentViewChatRoom.json#L1-34) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/SocialLink.json) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/SocialLink.json#L1-29)

* * *

Entity Roles and Responsibilities

Entity	Primary Purpose	Key Features
`ChatRoom`	Main entity for managing chat room approval workflow	Status tracking, approval process
`CreatedChatRoom`	Stores approved chat rooms for end-user consumption	Simplified schema, ready for use
`SocialLink`	Manages social media links attached to chat rooms	Type-link pairs, many-to-one relationship
`RecentViewChatRoom`	Tracks individual user views of chat rooms	User-chatRoom view relationship

****Sources:**** [(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json#L1-86) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CreatedChatRoom.json) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CreatedChatRoom.json#L1-72) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/RecentViewChatRoom.json) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/RecentViewChatRoom.json#L1-34) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/SocialLink.json) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/SocialLink.json#L1-29)

```
.jhipster/SocialLink.json1-29] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/SocialLink.json1-29)

.jhipster/RecentViewChatRoom.json1-34] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/RecentViewChatRoom.json1-34)

* * *

#### ChatRoom Entity Structure

The `ChatRoom` entity serves as the primary entity for chat room lifecycle management, containing the following fields:

#### Core Fields

| Field | Type | Purpose |
|---|---|---|
| `groupName` | String | Name of the chat room group |
| `groupOwner` | String | Identifier of the group owner |
| `introduction` | String | Description or introduction text for the chat room |
| `photo` | String | Photo URL or identifier for the chat room |
| `brokerName` | String | Associated broker name |
| `brokerContact` | String | Contact information for the broker |

**Sources:** [1] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json1-29) [2] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json3-27) [3] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json3-28)

#### Audit and Workflow Fields

The entity includes extensive audit trail fields to track the complete lifecycle of a chat room, including creation, approval, rejection, and updates.

| Field | Type | Purpose |
|---|---|---|
| `createdBy` | String | User who created the chat room |
| `createdAt` | ZonedDateTime | Creation timestamp |
| `updatedAt` | ZonedDateTime | Last update timestamp |
| `approvedBy` | String | Administrator who approved the request |
| `approvedAt` | ZonedDateTime | Approval timestamp |
| `rejectedBy` | String | Administrator who rejected the request |
| `rejectedAt` | ZonedDateTime | Rejection timestamp |
| `rejectReason` | String | Reason for rejection |

**Sources:** [4] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json3-29) [5] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json33-64) [6] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json33-65)

#### Status and Action Enumerations

The `ChatRoom` entity uses two enumerations to manage state and action tracking:

**StatusEnum Values:**

- `PENDING` - Initial state when chat room is created
- `APPROVED` - Chat room has been approved by administrator
- `REJECTED` - Chat room has been rejected by administrator
- `CREATED` - Chat room has been successfully created in the system
- `UPDATED` - Chat room has been modified

**ActionEnum Values:**

- `CREATE` - Indicates a creation action
- `UPDATE` - Indicates an update action
- `DELETE` - Indicates a deletion action

**Sources:** [7] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json33-66) [8] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json33-67) [9] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json33-68)
```

```
[.jhipster/ChatRoom.json29-32] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhi
.jhipster/ChatRoom.json66-69] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhi

* * *

#### Status Workflow and State Transitions

The chat room approval process follows a defined state machine pattern:

**Chat Room Status State Machine**

! [Chart 2] (nhsv-admin-images/nhsv-admin-section8-2.svg)

**Sources:** [] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.
[.jhipster/ChatRoom.json29-32] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhi
.jhipster/ChatRoom.json38-64] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhi

* * *

#### CreatedChatRoom Entity

The `CreatedChatRoom` entity represents the final, approved state of a chat room and is opt

#### Differences from ChatRoom

While `CreatedChatRoom` shares many fields with `ChatRoom`, it has several key differences:

**Simplified Workflow Fields:**

- No `rejectedAt` or `rejectedBy` fields (only approved rooms are copied)
- No `action` field (workflow complete)
- Includes `totalView` field for tracking aggregate view count

**Purpose:**

- Serves as the source of truth for active, approved chat rooms
- Optimized for frontend queries and public API access
- Separates operational workflow data from production data

**Sources:** [] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CreatedCha
[.jhipster/CreatedChatRoom.json1-72] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf906

#### View Count Tracking

The `totalView` field tracks the aggregate number of views for a chat room:
```

Field: totalView Type: Long Default Value: 0

This field is incremented when users view the chat room, providing analytics on chat room g

****Sources:**** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CreatedChatRoom.json59-62>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CreatedChatRoom.json59-62>)

* * *

SocialLink Management

The `SocialLink` entity provides a flexible mechanism for attaching social media links to c

Entity Structure

****SocialLink Field Mapping****

![[Chart 3] (nhsv-admin-images/nhsv-admin-section8-3.svg)]

****Sources:**** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/SocialLink.json1-29>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/SocialLink.json1-29>)

Relationship Configuration

The `SocialLink` entity maintains a many-to-one relationship with `ChatRoom`:

Relationship Type: many-to-one Foreign Entity: chatRoom Foreign Entity Field: id Relationship Name: chatRoom

```

This allows multiple social media links to be associated with a single chat room, supporting
**Sources:**[] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/SocialLink
[jhipster/SocialLink.json19-26] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.j

##### Example Social Link Types

The `type` field is a free-form string that typically contains values such as:

- `TELEGRAM`
- `DISCORD`
- `FACEBOOK`
- `TWITTER`
- `INSTAGRAM`
- `YOUTUBE`

**Sources:**[] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/SocialLink
[jhipster/SocialLink.json5-8] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhi

* * *

#### View Tracking System

The system implements two levels of view tracking: individual user view history through `RecentViewChatRoom`
##### RecentViewChatRoom Entity

**Purpose:** Tracks individual user viewing history for chat rooms, enabling features like

**Field Structure:**

| Field | Type | Purpose |
|---|---|---|
| `userId` | Long | Identifier of the user who viewed the chat room |
| `chatRoomId` | Long | Identifier of the viewed chat room |
| `createdAt` | ZonedDateTime | Timestamp of first view |
| `updatedAt` | ZonedDateTime | Timestamp of most recent view |
| `deletedAt` | ZonedDateTime | Soft deletion timestamp (enables view history cleanup) |

**Sources:**[] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/RecentView
[jhipster/RecentViewChatRoom.json4-25] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhi

##### View Tracking Data Flow

! [Chart 4] (nhsv-admin-images/nhsv-admin-section8-4.svg)

**Sources:**[] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/RecentView
[jhipster/RecentViewChatRoom.json1-34] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhi
[jhipster/CreatedChatRoom.json59-62] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhi

* * *

#### JHipster Configuration

All chat room system entities are configured with JHipster-specific settings for code generation.

##### ChatRoom Configuration

```

Service Type: serviceClass Pagination: pagination JPA Metamodel Filtering: enabled Incremental Changelog: enabled DTO: Not specified (uses default entity exposure)

```

**Sources:**[] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json)
[.jhipster/ChatRoom.json71-84] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json)
##### CreatedChatRoom Configuration

```

Service Type: serviceClass Pagination: pagination JPA Metamodel Filtering: enabled Incremental Changelog: enabled DTO: no (explicitly disabled)

```

**Sources:**[] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CreatedCha
[jhipster/CreatedChatRoom.json64-71] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf906
##### SocialLink Configuration

```

Service Type: serviceClass Pagination: no (small dataset expected) JPA Metamodel Filtering: disabled
Incremental Changelog: enabled DTO: no

```

**Sources:**[] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/SocialLink
[.jhipster/SocialLink.json14-27] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.
##### RecentViewChatRoom Configuration

```

Service Type: serviceClass Pagination: no (typically queried per user) JPA Metamodel Filtering: disabled
Incremental Changelog: enabled DTO: no

```

**Sources:**[] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/RecentViewChatRoom.json26-32) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/RecentViewChatRoom.json26-32)

* * *

#### Database Schema Summary

The chat room system generates four database tables with the following naming convention (

| Entity | Table Name | Primary Key | Relationships |
|---|---|---|---|
| `ChatRoom` | `t_chat_room` | `id` | One-to-many with `t_social_link` |
| `CreatedChatRoom` | `t_created_chat_room` | `id` | None (denormalized for performance)
| `SocialLink` | `t_social_link` | `id` | Many-to-one with `t_chat_room` |
| `RecentViewChatRoom` | `t_recent_view_chat_room` | `id` | None (junction table pattern)

**Changelog Dates:**

- `ChatRoom`: 20230621020056
- `SocialLink`: 20230621013037
- `RecentViewChatRoom`: 20230704023729
- `CreatedChatRoom`: 20230704092431

**Sources:**[] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json2) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/ChatRoom.json2) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/SocialLink.json2) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/RecentViewChatRoom.json2) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CreatedChatRoom.json2) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CreatedChatRoom.json2)

---

### InviteUser Entity

Relevant source files

- [
  .jhipster/CopyMarketLeaderDetails.json] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyMarketLeaderDetails.json)
- [
  .jhipster/CopyMarketLeaderProfitLoss.json] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyMarketLeaderProfitLoss.json)
- [
  .jhipster/CopyPortfolio.json] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json)
- [
  .jhipster/CopySubscriber.json] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json)
- [
  .jhipster/CopyTradingOrder.json] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json)
- [
  .jhipster/CopyTradingRegister.json] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingRegister.json)

#### Purpose and Scope

The Copy Trading Platform is the core business domain of the nhsv-admin system. It enables

```

This document provides a comprehensive overview of the platform's architecture, including t

- Market Leader Management (#3.3.1)
- Portfolio Management (#3.3.2)
- Subscriber Management (#3.3.3)
- Trading Operations (#3.3.4)

System Architecture

The Copy Trading Platform consists of six JHipster entities that form three logical subsystems.

![[Chart 1](nhsv-admin-images/nhsv-admin-section9-1.svg)]

Sources: (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyMarketLeaderDetails.json) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyMarketLeaderProfitLoss.json) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingRegister.json)

Entity Overview

The following table summarizes the six core entities that comprise the Copy Trading Platform:

Entity	Table Name	Primary Purpose	Key Relationships
CopyMarketLeaderDetails	t_copy_market_leader_details	Stores flexible key-value pairs for market leader details.	Linked to CopyMarketLeaderProfitLoss.
CopyMarketLeaderProfitLoss	t_copy_market_leader_profit_loss	Tracks aggregate profit/loss for market leaders.	Linked to CopyMarketLeaderDetails.
CopyPortfolio	t_copy_portfolio	Represents a market leader's current portfolio snapshot.	Linked to CopyMarketLeaderDetails.
CopySubscriber	t_copy_subscriber	Registers a subscriber's copy trading subscription.	Linked to CopyTradingOrder.
CopyTradingOrder	t_copy_trading_order	Stores individual trade orders generated by subscribers.	Linked to CopyTradingRegister.
CopyTradingRegister	t_copy_trading_register	Tracks registration status for copy trading.	Linked to CopySubscriber.

Sources: (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyMarketLeaderDetails.json) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyMarketLeaderProfitLoss.json) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingRegister.json)

Market Leader Components

CopyMarketLeaderDetails Entity

The `CopyMarketLeaderDetails` entity provides a flexible key-value storage mechanism for market leader information.

Key Fields:

[.hipster/CopyMarketLeaderDetails.json3-43] (<https://github.com/nhsv-tradex/nhsv-admin/blob/master/src/main/resources/static/3rdparty/hipster/CopyMarketLeaderDetails.json3-43>)

The `CopyMarketLeaderProfitLoss` entity tracks time-series performance metrics for market

```
- `reportDate`: Timestamp for the performance snapshot (required) [
    .jhipster/CopyMarketLeaderProfitLoss.json9-11] (https://github.com/nhsv-tradex/nhsv-admin)
- `netAssetsValue`: Total net asset value at report time (required) [
    .jhipster/CopyMarketLeaderProfitLoss.json14-16] (https://github.com/nhsv-tradex/nhsv-admin)
- `profitLossRatio`: Calculated profit/loss percentage (required) [
    .jhipster/CopyMarketLeaderProfitLoss.json19-21] (https://github.com/nhsv-tradex/nhsv-admin)
- `type`: Classification of the P&L record (required) [
    .jhipster/CopyMarketLeaderProfitLoss.json24-26] (https://github.com/nhsv-tradex/nhsv-admin)
```

```

**Sources:**[] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyMarket
[jhipster/CopyMarketLeaderDetails.json] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyMarketLeaderDetails.json)
[jhipster/CopyMarketLeaderProfitLoss.json] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyMarketLeaderProfitLoss.json)

```

```
##### CopyPortfolio Entity
```

```
**Key Characteristics:**
```

- [illegible]

.jhipster/CopyPortfolio.json21-25] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json21-25>)

The entity supports pagination and JPA Metamodel filtering for querying[] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json16-18>)

[.jhipster/CopyPortfolio.json16-18] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json16-18>)

! [Chart 2] (nhsv-admin-images/nhsv-admin-section9-2.svg)

****Sources:**** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json>)

[.jhipster/CopyPortfolio.json] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json>)

Subscriber Management

CopySubscriber Entity

The `CopySubscriber` entity registers a user's subscription to copy a market leader's portfolio.

****Key Fields:****

- `accountNumber`: Subscriber's trading account identifier (max 255 characters, required) [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json7-10>]
- `subNumber`: Sub-account identifier (max 255 characters, required) [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json13-16>]
- `userName`: Subscriber username (max 255 characters, required) [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json19-22>]
- `allocatedRatio`: Percentage of portfolio to allocate (required) [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json25-27>]
- `orderSetType`: Execution strategy enum (`QUICK\_MATCH` or `SAFE\_MATCH`) [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json30-32>]
- `deviceUniqueId`: Device identifier for tracking (max 255 characters, required) [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json44-47>]
- `customerName`: Subscriber's full name (max 500 characters, required) [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json50-53>]

The `orderSetType` field uses the `OrderSetTypeEnum` enumeration with two strategies:

Order Set Type	Description
QUICK_MATCH	Faster execution with potentially less precise matching
SAFE_MATCH	Conservative execution prioritizing price matching accuracy

****Relationship:**** The entity has a `many-to-one` relationship with `User` via `mlUserId`, [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json62-69>]

****Sources:**** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json>)

[.jhipster/CopySubscriber.json] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json>)

Trading Execution

CopyTradingOrder Entity

The `CopyTradingOrder` entity stores individual trade orders generated when a subscriber copies a market leader's portfolio.

```

**Order Identification:**

- `jobId`: Batch job identifier for the order generation run (max 255 characters, required) [
  .jhipster/CopyTradingOrder.json7-10] (https://github.com/nhsv-tradex/nhsv-admin/blob/2c
- `symbol`: Stock ticker symbol (max 255 characters, required) [
  .jhipster/CopyTradingOrder.json13-16] (https://github.com/nhsv-tradex/nhsv-admin/blob/2c
- `orderNumber`: Broker-assigned order number after submission [
  .jhipster/CopyTradingOrder.json27-28] (https://github.com/nhsv-tradex/nhsv-admin/blob/2c

**Order Parameters:**

- `sellBuyType`: Enum specifying `BUY` or `SELL` [
  .jhipster/CopyTradingOrder.json31-33] (https://github.com/nhsv-tradex/nhsv-admin/blob/2c
- `exchangeType`: Market exchange enum (`HOSE`, `HNX`, `UPCOM`) [
  .jhipster/CopyTradingOrder.json36-38] (https://github.com/nhsv-tradex/nhsv-admin/blob/2c
- `orderType`: Order execution type enum (`ATO`, `ATC`, `LO`, `MP`, `MAK`, `MOK`, `MTL`,
  .jhipster/CopyTradingOrder.json41-43] (https://github.com/nhsv-tradex/nhsv-admin/blob/2c
- `orderQuantity`: Number of shares to trade [
  .jhipster/CopyTradingOrder.json46-47] (https://github.com/nhsv-tradex/nhsv-admin/blob/2c
- `orderPrice`: Price per share [
  .jhipster/CopyTradingOrder.json50-51] (https://github.com/nhsv-tradex/nhsv-admin/blob/2c

**Cost Fields:**

- `fee`: Transaction fee [
  .jhipster/CopyTradingOrder.json19-20] (https://github.com/nhsv-tradex/nhsv-admin/blob/2c
- `tax`: Transaction tax [
  .jhipster/CopyTradingOrder.json23-24] (https://github.com/nhsv-tradex/nhsv-admin/blob/2c

**API Integration:**

- `apiParam`: Parameters sent to trading API [
  .jhipster/CopyTradingOrder.json54-55] (https://github.com/nhsv-tradex/nhsv-admin/blob/2c
- `apiStatusCode`: Response status code from trading API [
  .jhipster/CopyTradingOrder.json58-59] (https://github.com/nhsv-tradex/nhsv-admin/blob/2c
- `apiErrorMessage`: Error message if API call fails [
  .jhipster/CopyTradingOrder.json62-63] (https://github.com/nhsv-tradex/nhsv-admin/blob/2c

**Foreign Keys:**

- `copySubscriberId`: Links to the subscriber executing the order (required) [
  .jhipster/CopyTradingOrder.json75-77] (https://github.com/nhsv-tradex/nhsv-admin/blob/2c
- `copyPortfolioId`: Links to the source portfolio being copied (required) [
  .jhipster/CopyTradingOrder.json80-82] (https://github.com/nhsv-tradex/nhsv-admin/blob/2c

#### Order Type Enumerations

The entity uses three enumerations for categorizing orders:

```

Enum Field	Possible Values	Purpose
`sellBuyType`	`BUY`, `SELL`	Transaction direction
`exchangeType`	`HOSE`, `HNX`, `UPCOM`	Vietnamese stock exchanges
`orderType`	`ATO`, `ATC`, `LO`, `MP`, `MAK`, `MOK`, `MTL`, `PLO`	Order execution strategy

CopyTradingRegister Entity

The `CopyTradingRegister` entity tracks the registration lifecycle for copy trading subscribers.

Key Fields:

- `accountNumber`: Registered account identifier [

[.jhipster/CopyTradingRegister.json#L7-9](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingRegister.json#L7-9) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingRegister.json#L7-9)
- `subAccount`: Sub-account identifier [

[.jhipster/CopyTradingRegister.json#L12-14](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingRegister.json#L12-14) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingRegister.json#L12-14)
- `customerName`: Registrant name [

[.jhipster/CopyTradingRegister.json#L17-19](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingRegister.json#L17-19) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingRegister.json#L17-19)
- `status`: Boolean flag indicating registration approval status [

[.jhipster/CopyTradingRegister.json#L22-23](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingRegister.json#L22-23) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingRegister.json#L22-23)
- `createAt`: Registration timestamp [

[.jhipster/CopyTradingRegister.json#L25-27](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingRegister.json#L25-27) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingRegister.json#L25-27)
- `updatedAt`: Last modification timestamp [

[.jhipster/CopyTradingRegister.json#L29-31](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingRegister.json#L29-31) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingRegister.json#L29-31)

This entity has no foreign key relationships, suggesting it operates independently for registration management.

[.jhipster/CopyTradingRegister.json#L39] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingRegister.json#L39)

Sources: [

<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingRegister.json>

[.jhipster/CopyTradingOrder.json] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json)

[.jhipster/CopyTradingRegister.json] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingRegister.json)

Copy Trading Workflow

The following diagram illustrates the complete data flow from market leader portfolio creation to order execution.

![[Chart 3] (nhsv-admin-images/nhsv-admin-section9-3.svg)]

Workflow Description:

1. **Portfolio Creation:** A market leader's portfolio is captured in `CopyPortfolio` with details like assets and ratios.
2. **Registration:** Potential subscribers register through `CopyTradingRegister`, where they provide account and personal information.
3. **Subscription Activation:** Approved registrations create `CopySubscriber` records linked to the market leader's portfolio.
4. **Order Generation:** The system generates `CopyTradingOrder` records by:
 - Referencing the `copyPortfolioId` to determine what assets to trade
 - Applying the subscriber's `allocatedRatio` to calculate proportional quantities
 - Using the `orderSetType` to determine execution strategy
 - Creating orders with appropriate `sellBuyType`, `exchangeType`, and `orderType` values
5. **Execution:** Orders are submitted to external trading APIs with parameters stored in the `CopyTradingOrder` entity.

6. ****Cost Tracking****: After execution, `fee` and `tax` fields are populated to track transac

****Sources**** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingRegister.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyMarketLeaderDetails.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyMarketLeaderProfitLoss.json>)

Entity Configuration Summary

All six entities in the Copy Trading Platform share common JHipster configuration patterns:

Configuration Aspect	Standard Pattern
DTO Mapping	`mapstruct` for type-safe object mapping
Pagination	`pagination` enabled for large result sets
Service Layer	`serviceImpl` with dedicated service implementations
Incremental Changelog	`true` for Liquibase schema evolution
Change Tracking	`createdAt` (required) and `updatedAt` (optional) timestamps

The entities use JPA Metamodel Filtering selectively:

- ****Enabled****: `CopyMarketLeaderDetails`, `CopyPortfolio`, `CopySubscriber`, `CopyTradingOrder`, `CopyTradingRegister`
- ****Disabled****: `CopyMarketLeaderProfitLoss`

This filtering capability allows dynamic query construction for complex search operations t

****Sources**** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyMarketLeaderDetails.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyMarketLeaderProfitLoss.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingRegister.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyMarketLeaderDetails.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyMarketLeaderProfitLoss.json>)

Database Schema Conventions

The Copy Trading Platform follows consistent naming conventions for database tables and re

****Table Naming****

- All entities use the prefix `t\_copy\_` for table names
- Examples: `t\_copy\_portfolio`, `t\_copy\_subscriber`, `t\_copy\_trading\_order`

****Relationship Naming****

- Market leader relationships consistently use `mlUserId` as the foreign key name
- This references the `User` entity's `login` field as the display attribute
- Example: [

.jhipster/CopySubscriber.json62-69] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json>)

****Foreign Key Storage:****

- Direct foreign key fields (like `copySubscriberId` and `copyPortfolioId` in `CopyTradingOrder`)
- This approach provides flexibility in querying without enforcing strict referential integrity

****Sources:**** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyMarketLeaderDetails.json4>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyPortfolio.json6>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopySubscriber.json4>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json4-82>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json4-82>)

Chat Room System

Relevant source files

- [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)
- [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

Purpose and Scope

This document describes supporting entities that provide ancillary functionality to the core copy trading system.

The two supporting entities covered in this document are:

- ****Holiday****: Manages market holidays and non-trading days
- ****MarketHistoryJobResult****: Tracks execution results for market data processing jobs

For information about the core copy trading entities, see [Copy Trading Platform] (#copy-trading-platform).

* * *

Overview

Supporting entities serve operational and administrative needs that span multiple business units.

1. ****Temporal Business Logic****: The `Holiday` entity enables the system to account for market holidays and non-trading days.
2. ****Job Observability****: The `MarketHistoryJobResult` entity provides audit trails and execution status for market data processing jobs.

These entities are defined using JHipster's entity configuration system and follow the same naming conventions as the core entities.

****Sources****: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>)

* * *

Entity Architecture Diagram

```

![[Chart 1]](nhsv-admin-images/nhsv-admin-section10-1.svg)

**Diagram: Supporting Entities and Their System Integration**

This diagram shows how supporting entities integrate with core business entities. The `Holiday` entity is a supporting entity that integrates with the `MarketHistoryJobResult` entity.

**Sources**:[ ](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45)

* * *

#### Holiday Entity

#### Purpose

The `Holiday` entity maintains a calendar of market holidays and non-trading days. This includes:

- Determining valid trading windows
- Scheduling automated copy trading operations
- Calculating business day intervals for performance metrics
- Displaying market availability to users

#### Entity Definition

The `Holiday` entity is defined in `.jhipster/Holiday.json` and generates a JPA entity mapping.

**Sources**:[ ](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json6) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json7-35)

#### Field Structure



| Field Name     | Type            | Constraints             | Description                        |
|----------------|-----------------|-------------------------|------------------------------------|
| `year`         | `Integer`       | Required                | Calendar year for the holiday      |
| `eventHoliday` | `String`        | Required, Max 255 chars | Name or description of the holiday |
| `startDate`    | `ZonedDateTime` | None                    | Holiday period start timestamp     |
| `endDate`      | `ZonedDateTime` | None                    | Holiday period end timestamp       |
| `createdAt`    | `ZonedDateTime` | Required                | Record creation timestamp          |
| `updatedAt`    | `ZonedDateTime` | None                    | Last modification timestamp        |



**Sources**:[ ](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json36-40) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json41-44)

#### Entity Structure Diagram

![[Chart 2]](nhsv-admin-images/nhsv-admin-section10-2.svg)

**Diagram: Holiday Entity Class Structure**

This diagram shows the generated backend components for the `Holiday` entity. The entity is defined in the `Holiday` class, and the JPA entity mapping is defined in the `Holiday` entity.

**Sources**:[ ](https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json45) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json46-49) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json50-53)

#### Configuration Details

```

```

**JHipster Settings**:

| Setting | Value | Meaning |
|---|---|---|
| `dto` | `mapstruct` | Uses MapStruct for entity-DTO mapping |
| `service` | `serviceImpl` | Generates service interface and implementation |
| `pagination` | `pagination` | Enables paginated REST endpoints |
| `jpaMetamodelFiltering` | `false` | Disables JPA Metamodel filtering |
| `incrementalChangelog` | `true` | Uses incremental Liquibase changelogs |

**Sources**:[] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js
[.jhipster/Holiday.json4-44] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhip

##### Time Range Representation

The `Holiday` entity supports flexible holiday duration modeling through `startDate` and `e

- **Single-day holidays**: `startDate` set to beginning of day, `endDate` set to end of d
- **Multi-day closures**: `startDate` and `endDate` define a continuous non-trading perio
- **Null dates**: Fields are optional, allowing for year-only holiday records

The use of `ZonedDateTime` ensures proper timezone handling across different market regions

**Sources**:[] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js
[.jhipster/Holiday.json20-25] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhip

* * *

#### MarketHistoryJobResult Entity

##### Purpose

The `MarketHistoryJobResult` entity tracks the execution results of background jobs that p

- Audit trail for data processing operations
- Error tracking and debugging information
- Job performance monitoring (start/end times)
- User accountability for job execution

##### Entity Definition

The `MarketHistoryJobResult` entity is defined in `.jhipster/MarketHistoryJobResult.json` a

**Sources**:[] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHist
[.jhipster/MarketHistoryJobResult.json1-46] (https://github.com/nhsv-tradex/nhsv-admin/blob,

##### Field Structure

| Field Name | Type | Constraints | Description |
|---|---|---|---|
| `isSuccess` | `Boolean` | None | Whether the job completed successfully |
| `timeStart` | `ZonedDateTime` | None | Job execution start timestamp |
| `timeEnd` | `ZonedDateTime` | None | Job execution end timestamp |
| `error` | `String` | None | Error message if job failed |
| `eventId` | `String` | None | Identifier for the specific job event |
| `symbols` | `String` | None | Market symbols processed by the job |

**Sources**:[] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHist

```


[.jhipster/MarketHistoryJobResult.json3-29] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>)

Relationship Structure

The entity has a **many-to-one** relationship with the ``User`` entity, tracking which user initiated the job.

Relationship Property	Value
Relationship Type	<code>`many-to-one`</code>
Related Entity	<code>`user`</code>
Owner Side	<code>`true`</code>
Display Field	<code>`login`</code>

This relationship enables:

- Tracking job execution by user
- Filtering jobs by user login
- Auditing who triggered specific data processing operations

Sources: [.jhipster/MarketHistoryJobResult.json3-29] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>)

Entity Relationship Diagram

! [Chart 3] (nhsv-admin-images/nhsv-admin-section10-3.svg)

Diagram: MarketHistoryJobResult Entity Relationships

This ER diagram shows the relationship between ``User`` and ``MarketHistoryJobResult``. Each job is associated with exactly one user.

Sources: [.jhipster/MarketHistoryJobResult.json3-29] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>)

Configuration Details

JHipster Settings:

Setting	Value	Meaning
<code>`dto`</code>	Not specified	Uses entity directly (no DTO generation)
<code>`service`</code>	<code>`no`</code>	No service layer generation; direct repository access
<code>`pagination`</code>	<code>`no`</code>	No pagination on REST endpoints
<code>`incrementalChangelog`</code>	<code>`true`</code>	Uses incremental Liquibase changelogs

The absence of DTO and service layer generation suggests this entity is primarily used for data processing or reporting.

Sources: [.jhipster/MarketHistoryJobResult.json31-45] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>)

Job Execution Tracking

! [Chart 4] (nhsv-admin-images/nhsv-admin-section10-4.svg)

Diagram: Market History Job Execution Flow

This sequence diagram illustrates how the ``MarketHistoryJobResult`` entity tracks job lifecycle events.

Sources: [.jhipster/MarketHistoryJobResult.json3-29] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>)

Symbols Field Usage

The `symbols` field stores market symbols processed during the job, likely as a comma-separated list.

- Understanding job scope (which symbols were included)
- Filtering failed jobs by specific symbols
- Debugging symbol-specific processing issues

****Example storage formats**:**

- Comma-separated: `"AAPL,GOOGL,MSFT"`
- JSON array: `["AAPL","GOOGL","MSFT"]`

The actual format depends on the implementation of the job processing logic.

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>27-28] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>27-28)

* * *

System Integration Pattern

! [Chart 5] (nhsv-admin-images/nhsv-admin-section10-5.svg)

****Diagram: Supporting Entities in System Context****

This diagram shows how supporting entities integrate with external dependencies and business logic.

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json>1-45] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json>1-45) [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>1-46] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>1-46)

* * *

Comparison Table

Aspect	Holiday	MarketHistoryJobResult
Primary Purpose	Define non-trading days	Track background job execution
Data Source	Manual entry or calendar API	Generated by job scheduler
Update Frequency	Annually (typically)	Per job execution
Core Fields	`year`, `eventHoliday`, date range	`isSuccess`, `error`, time range
Relationships	None	Many-to-one with User
DTO Generation	Yes (MapStruct)	No
Service Layer	Yes (serviceImpl)	No
Pagination	Yes	No
Primary Consumers	Trading schedulers, availability checks	System administrators, monitoring tools

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json>4-44] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json>4-44) [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>31-45] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>31-45)

* * *

Database Schema

Holiday Table Structure

```
-- t_holiday table
CREATE TABLE t_holiday (
  id BIGSERIAL PRIMARY KEY,
  year INTEGER NOT NULL,
  event_holiday VARCHAR(255) NOT NULL,
  start_date TIMESTAMP WITH TIME ZONE,
  end_date TIMESTAMP WITH TIME ZONE,
  created_at TIMESTAMP WITH TIME ZONE NOT NULL,
  updated_at TIMESTAMP WITH TIME ZONE
);
```

MarketHistoryJobResult Table Structure

```
-- market_history_job_result table
CREATE TABLE market_history_job_result (
  id BIGSERIAL PRIMARY KEY,
  is_success BOOLEAN,
  time_start TIMESTAMP WITH TIME ZONE,
  time_end TIMESTAMP WITH TIME ZONE,
  error TEXT,
  event_id VARCHAR(255),
  symbols TEXT,
  user_id BIGINT,
  FOREIGN KEY (user_id) REFERENCES jhi_user(id)
);
```

* * *

Supporting entities provide essential infrastructure services that enable core business operations.

- Both entities leverage JHipster's code generation to provide consistent REST APIs, database

— — —

Relevant source files

- ### #### Purpose and Scope

This document describes supporting entities that provide ancillary functionality to the co

The two supporting entities covered in this document are:

- For information about the core copy trading entities, see [Copy Trading Platform](#copy-trading).

* * *

Supporting entities serve operational and administrative needs that span multiple business

- Page 92 of 200

These entities are defined using JHipster's entity configuration system and follow the same

Sources: [1] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
 [2] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-46>)

* * *

Entity Architecture Diagram

! [Chart 1] (nhsv-admin-images/nhsv-admin-section11-1.svg)

Diagram: Supporting Entities and Their System Integration

This diagram shows how supporting entities integrate with core business entities. The `Holiday` entity is a supporting entity that integrates with the `MarketHistoryJobResult` entity.

Sources: [1] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
 [2] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-46>)

* * *

Holiday Entity

Purpose

The `Holiday` entity maintains a calendar of market holidays and non-trading days. This includes:

- Determining valid trading windows
- Scheduling automated copy trading operations
- Calculating business day intervals for performance metrics
- Displaying market availability to users

Entity Definition

The `Holiday` entity is defined in `.jhipster/Holiday.json` and generates a JPA entity mapping.

Sources: [1] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
 [2] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json6>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-46>)

Field Structure

Field Name	Type	Constraints	Description
`year`	`Integer`	Required	Calendar year for the holiday
`eventHoliday`	`String`	Required, Max 255 chars	Name or description of the holiday
`startDate`	`ZonedDateTime`	None	Holiday period start timestamp
`endDate`	`ZonedDateTime`	None	Holiday period end timestamp
`createdAt`	`ZonedDateTime`	Required	Record creation timestamp
`updatedAt`	`ZonedDateTime`	None	Last modification timestamp

Sources: [1] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
 [2] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json7-35>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-46>)

Entity Structure Diagram

! [Chart 2] (nhsv-admin-images/nhsv-admin-section11-2.svg)

Diagram: Holiday Entity Class Structure

This diagram shows the generated backend components for the `Holiday` entity. The entity fo

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json4] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json4>)

[.jhipster/Holiday.json41-44] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json41-44>)

Configuration Details

****JHipster Settings**:**

Setting	Value	Meaning
dto	mapstruct	Uses MapStruct for entity-DTO mapping
service	serviceImpl	Generates service interface and implementation
pagination	pagination	Enables paginated REST endpoints
jpaMetamodelFiltering	false	Disables JPA Metamodel filtering
incrementalChangelog	true	Uses incremental Liquibase changelogs

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json4-44] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json4-44>)

Time Range Representation

The `Holiday` entity supports flexible holiday duration modeling through `startDate` and `e

- ****Single-day holidays**:** `startDate` set to beginning of day, `endDate` set to end of day
- ****Multi-day closures**:** `startDate` and `endDate` define a continuous non-trading period
- ****Null dates**:** Fields are optional, allowing for year-only holiday records

The use of `ZonedDateTime` ensures proper timezone handling across different market regions

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json20-25] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json20-25>)

* * *

MarketHistoryJobResult Entity

Purpose

The `MarketHistoryJobResult` entity tracks the execution results of background jobs that p

- Audit trail for data processing operations
- Error tracking and debugging information
- Job performance monitoring (start/end times)
- User accountability for job execution

Entity Definition

The `MarketHistoryJobResult` entity is defined in `.jhipster/MarketHistoryJobResult.json` &

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

[.jhipster/MarketHistoryJobResult.json1-46] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>)

Field Structure

Field Name	Type	Constraints	Description
`isSuccess`	`Boolean`	None	Whether the job completed successfully
`timeStart`	`ZonedDateTime`	None	Job execution start timestamp
`timeEnd`	`ZonedDateTime`	None	Job execution end timestamp
`error`	`String`	None	Error message if job failed
`eventId`	`String`	None	Identifier for the specific job event
`symbols`	`String`	None	Market symbols processed by the job

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Relationship Structure

The entity has a **many-to-one** relationship with the ``User`` entity, tracking which user triggered the job.

Relationship Property	Value
Relationship Type	<code>`many-to-one`</code>
Related Entity	<code>`user`</code>
Owner Side	<code>`true`</code>
Display Field	<code>`login`</code>

This relationship enables:

- Tracking job execution by user
- Filtering jobs by user login
- Auditing who triggered specific data processing operations

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Entity Relationship Diagram

! [Chart 3] (nhsv-admin-images/nhsv-admin-section11-3.svg)

****Diagram: MarketHistoryJobResult Entity Relationships****

This ER diagram shows the relationship between ``User`` and ``MarketHistoryJobResult``. Each job is associated with exactly one user.

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Configuration Details

****JHipster Settings**:**

Setting	Value	Meaning
<code>`dto`</code>	Not specified	Uses entity directly (no DTO generation)
<code>`service`</code>	<code>`no`</code>	No service layer generation; direct repository access
<code>`pagination`</code>	<code>`no`</code>	No pagination on REST endpoints
<code>`incrementalChangelog`</code>	<code>`true`</code>	Uses incremental Liquibase changelogs

The absence of DTO and service layer generation suggests this entity is primarily used for data processing or reporting.

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>]

Job Execution Tracking

![Chart 4] (nhsv-admin-images/nhsv-admin-section11-4.svg)

****Diagram: Market History Job Execution Flow****

This sequence diagram illustrates how the `MarketHistoryJobResult` entity tracks job lifecycle.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>)

Symbols Field Usage

The `symbols` field stores market symbols processed during the job, likely as a comma-separated list.

- Understanding job scope (which symbols were included)
- Filtering failed jobs by specific symbols
- Debugging symbol-specific processing issues

****Example storage formats**:**

- Comma-separated: `"AAPL,GOOGL,MSFT"`
- JSON array: `["AAPL","GOOGL","MSFT"]`

The actual format depends on the implementation of the job processing logic.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json27-28>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json27-28>)

* * *

System Integration Pattern

![Chart 5] (nhsv-admin-images/nhsv-admin-section11-5.svg)

****Diagram: Supporting Entities in System Context****

This diagram shows how supporting entities integrate with external dependencies and business logic.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>)

* * *

Comparison Table

Aspect	Holiday	MarketHistoryJobResult
Primary Purpose	Define non-trading days	Track background job execution
Data Source	Manual entry or calendar API	Generated by job scheduler
Update Frequency	Annually (typically)	Per job execution
Core Fields	`year`, `eventHoliday`, date range	`isSuccess`, `error`, time range
Relationships	None	Many-to-one with User
DTO Generation	Yes (MapStruct)	No
Service Layer	Yes (serviceImpl)	No
Pagination	Yes	No
Primary Consumers	Trading schedulers, availability checks	System administrators,


```

**Sources**:[] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js
[.jhipster/Holiday.json4-44] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhip
.jhipster/MarketHistoryJobResult.json31-45] (https://github.com/nhsv-tradex/nhsv-admin/blob,

* * *

#### Database Schema

##### Holiday Table Structure

```

-- t_holiday table

```

CREATE TABLE t_holiday (
  id BIGSERIAL PRIMARY KEY,
  year INTEGER NOT NULL,
  event_holiday VARCHAR(255) NOT NULL,
  start_date TIMESTAMP WITH TIME ZONE,
  end_date TIMESTAMP WITH TIME ZONE,
  created_at TIMESTAMP WITH TIME ZONE NOT NULL,
  updated_at TIMESTAMP WITH TIME ZONE
);

```

```

##### MarketHistoryJobResult Table Structure

```

-- market_history_job_result table

```

CREATE TABLE market_history_job_result (
  id BIGSERIAL PRIMARY KEY,
  is_success BOOLEAN,
  time_start TIMESTAMP WITH TIME ZONE,
  time_end TIMESTAMP WITH TIME ZONE,
  error TEXT,
  event_id VARCHAR(255),
  symbols TEXT,
  user_id BIGINT,
  FOREIGN KEY (user_id) REFERENCES jhi_user(id)
);

```

These schemas are managed by Liquibase changelogs generated from the JHipster entity definition.

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

* * *

Summary

Supporting entities provide essential infrastructure services that enable core business operations.

- **Holiday**: Enables time-aware business logic by defining non-trading periods, critical for accurate market data processing.
- **MarketHistoryJobResult**: Provides observability into asynchronous data processing, ensuring transparency and reliability of market data.

Both entities leverage JHipster's code generation to provide consistent REST APIs, database schemas, and frontend components.

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

SocialLink and View Tracking

Relevant source files

- [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)
- [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

Purpose and Scope

This document describes supporting entities that provide ancillary functionality to the core trading platform.

The two supporting entities covered in this document are:

- **Holiday**: Manages market holidays and non-trading days
- **MarketHistoryJobResult**: Tracks execution results for market data processing jobs

For information about the core copy trading entities, see [Copy Trading Platform] (#copy-trading-platform).

* * *

Overview

Supporting entities serve operational and administrative needs that span multiple business units.

1. **Temporal Business Logic**: The `Holiday` entity enables the system to account for market holidays and non-trading days.
2. **Job Observability**: The `MarketHistoryJobResult` entity provides audit trails and execution details for market data processing jobs.

These entities are defined using JHipster's entity configuration system and follow the same

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
[.jhipster/Holiday.json1-45] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

* * *

Entity Architecture Diagram

! [Chart 1] (nhsv-admin-images/nhsv-admin-section12-1.svg)

Diagram: Supporting Entities and Their System Integration

This diagram shows how supporting entities integrate with core business entities. The `Holiday` entity is a core business entity, while `MarketHistoryJobResult` is a supporting entity.

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
[.jhipster/Holiday.json1-45] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

* * *

Holiday Entity

Purpose

The `Holiday` entity maintains a calendar of market holidays and non-trading days. This includes:

- Determining valid trading windows
- Scheduling automated copy trading operations
- Calculating business day intervals for performance metrics
- Displaying market availability to users

Entity Definition

The `Holiday` entity is defined in `.jhipster/Holiday.json` and generates a JPA entity mapping.

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
[.jhipster/Holiday.json6] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

Field Structure

Field Name	Type	Constraints	Description
`year`	`Integer`	Required	Calendar year for the holiday
`eventHoliday`	`String`	Required, Max 255 chars	Name or description of the holiday
`startDate`	`ZonedDateTime`	None	Holiday period start timestamp
`endDate`	`ZonedDateTime`	None	Holiday period end timestamp
`createdAt`	`ZonedDateTime`	Required	Record creation timestamp
`updatedAt`	`ZonedDateTime`	None	Last modification timestamp

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
[.jhipster/Holiday.json7-35] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

Entity Structure Diagram

! [Chart 2] (nhsv-admin-images/nhsv-admin-section12-2.svg)

Diagram: Holiday Entity Class Structure

This diagram shows the generated backend components for the `Holiday` entity. The entity fo

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json4] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json4>)

[.jhipster/Holiday.json41-44] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json41-44>)

Configuration Details

JHipster Settings:

Setting	Value	Meaning
dto	mapstruct	Uses MapStruct for entity-DTO mapping
service	serviceImpl	Generates service interface and implementation
pagination	pagination	Enables paginated REST endpoints
jpaMetamodelFiltering	false	Disables JPA Metamodel filtering
incrementalChangelog	true	Uses incremental Liquibase changelogs

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json4-44] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json4-44>)

Time Range Representation

The `Holiday` entity supports flexible holiday duration modeling through `startDate` and `e

- ****Single-day holidays**:** `startDate` set to beginning of day, `endDate` set to end of day
- ****Multi-day closures**:** `startDate` and `endDate` define a continuous non-trading period
- ****Null dates**:** Fields are optional, allowing for year-only holiday records

The use of `ZonedDateTime` ensures proper timezone handling across different market regions

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json20-25] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json20-25>)

* * *

MarketHistoryJobResult Entity

Purpose

The `MarketHistoryJobResult` entity tracks the execution results of background jobs that p

- Audit trail for data processing operations
- Error tracking and debugging information
- Job performance monitoring (start/end times)
- User accountability for job execution

Entity Definition

The `MarketHistoryJobResult` entity is defined in `.jhipster/MarketHistoryJobResult.json` &

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

[.jhipster/MarketHistoryJobResult.json1-46] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>)

Field Structure

Field Name	Type	Constraints	Description
`isSuccess`	`Boolean`	None	Whether the job completed successfully
`timeStart`	`ZonedDateTime`	None	Job execution start timestamp
`timeEnd`	`ZonedDateTime`	None	Job execution end timestamp
`error`	`String`	None	Error message if job failed
`eventId`	`String`	None	Identifier for the specific job event
`symbols`	`String`	None	Market symbols processed by the job

****Sources**:** [(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29)]

Relationship Structure

The entity has a **many-to-one** relationship with the ``User`` entity, tracking which user triggered the job.

Relationship Property	Value
Relationship Type	<code>`many-to-one`</code>
Related Entity	<code>`user`</code>
Owner Side	<code>`true`</code>
Display Field	<code>`login`</code>

This relationship enables:

- Tracking job execution by user
- Filtering jobs by user login
- Auditing who triggered specific data processing operations

****Sources**:** [(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43)]

Entity Relationship Diagram

! [Chart 3] (nhsv-admin-images/nhsv-admin-section12-3.svg)

****Diagram: MarketHistoryJobResult Entity Relationships****

This ER diagram shows the relationship between ``User`` and ``MarketHistoryJobResult``. Each job is associated with exactly one user.

****Sources**:** [(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43)]

Configuration Details

****JHipster Settings**:**

Setting	Value	Meaning
<code>`dto`</code>	Not specified	Uses entity directly (no DTO generation)
<code>`service`</code>	<code>`no`</code>	No service layer generation; direct repository access
<code>`pagination`</code>	<code>`no`</code>	No pagination on REST endpoints
<code>`incrementalChangelog`</code>	<code>`true`</code>	Uses incremental Liquibase changelogs

The absence of DTO and service layer generation suggests this entity is primarily used for data processing or reporting.

****Sources**:** [(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45)]

Job Execution Tracking

![Chart 4] (nhsv-admin-images/nhsv-admin-section12-4.svg)

****Diagram: Market History Job Execution Flow****

This sequence diagram illustrates how the `MarketHistoryJobResult` entity tracks job lifecycle.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>)

Symbols Field Usage

The `symbols` field stores market symbols processed during the job, likely as a comma-separated list.

- Understanding job scope (which symbols were included)
- Filtering failed jobs by specific symbols
- Debugging symbol-specific processing issues

****Example storage formats**:**

- Comma-separated: `"AAPL,GOOGL,MSFT"`
- JSON array: `["AAPL","GOOGL","MSFT"]`

The actual format depends on the implementation of the job processing logic.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json27-28>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json27-28>)

* * *

System Integration Pattern

![Chart 5] (nhsv-admin-images/nhsv-admin-section12-5.svg)

****Diagram: Supporting Entities in System Context****

This diagram shows how supporting entities integrate with external dependencies and business logic.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>)

* * *

Comparison Table

Aspect	Holiday	MarketHistoryJobResult
Primary Purpose	Define non-trading days	Track background job execution
Data Source	Manual entry or calendar API	Generated by job scheduler
Update Frequency	Annually (typically)	Per job execution
Core Fields	`year`, `eventHoliday`, date range	`isSuccess`, `error`, time range
Relationships	None	Many-to-one with User
DTO Generation	Yes (MapStruct)	No
Service Layer	Yes (serviceImpl)	No
Pagination	Yes	No
Primary Consumers	Trading schedulers, availability checks	System administrators, monitoring tools

```

**Sources**:[] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js
[.jhipster/Holiday.json4-44] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhip
.jhipster/MarketHistoryJobResult.json31-45] (https://github.com/nhsv-tradex/nhsv-admin/blob,

* * *

#### Database Schema

##### Holiday Table Structure

```

-- t_holiday table

```

CREATE TABLE t_holiday (
  id BIGSERIAL PRIMARY KEY,
  year INTEGER NOT NULL,
  event_holiday VARCHAR(255) NOT NULL,
  start_date TIMESTAMP WITH TIME ZONE,
  end_date TIMESTAMP WITH TIME ZONE,
  created_at TIMESTAMP WITH TIME ZONE NOT NULL,
  updated_at TIMESTAMP WITH TIME ZONE
);

```

```

##### MarketHistoryJobResult Table Structure

```

-- market_history_job_result table

```

CREATE TABLE market_history_job_result (
  id BIGSERIAL PRIMARY KEY,
  is_success BOOLEAN,
  time_start TIMESTAMP WITH TIME ZONE,
  time_end TIMESTAMP WITH TIME ZONE,
  error TEXT,
  event_id VARCHAR(255),
  symbols TEXT,
  user_id BIGINT,
  FOREIGN KEY (user_id) REFERENCES jhi_user(id)
);

```

```
These schemas are managed by Liquibase changelogs generated from the JHipster entity definition.

**Sources**: [] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json#L7-35) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json#L3-43) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json#L43-44)

* * *

#### Summary

Supporting entities provide essential infrastructure services that enable core business operations.

- **Holiday**: Enables time-aware business logic by defining non-trading periods, critical for market data processing.
- **MarketHistoryJobResult**: Provides observability into asynchronous data processing, ensuring audit trails and execution results.

Both entities leverage JHipster's code generation to provide consistent REST APIs, database schemas, and frontend components.

**Sources**: [] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json#L1-45) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json#L1-46) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json#L46-47)

---

### Copy Trading Platform

Relevant source files

- [CopyTradingPlatformCopyTradeEntity.java] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingPlatformCopyTradeEntity.java) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingPlatformCopyTradeEntity.java#L1-18)
- [CopyTradingPlatformCopyTradeRepository.java] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingPlatformCopyTradeRepository.java) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingPlatformCopyTradeRepository.java#L1-10)
- [CopyTradingPlatformCopyTradeService.java] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingPlatformCopyTradeService.java) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingPlatformCopyTradeService.java#L1-18)
- [CopyTradingPlatformCopyTradeController.java] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingPlatformCopyTradeController.java) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingPlatformCopyTradeController.java#L1-18)
- [CopyTradingPlatformCopyTradeResource.java] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingPlatformCopyTradeResource.java) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingPlatformCopyTradeResource.java#L1-18)
- [CopyTradingPlatformCopyTradeTest.java] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingPlatformCopyTradeTest.java) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingPlatformCopyTradeTest.java#L1-18)
- [CopyTradingPlatformCopyTradeIntegrationTest.java] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingPlatformCopyTradeIntegrationTest.java) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingPlatformCopyTradeIntegrationTest.java#L1-18)
- [CopyTradingPlatformCopyTradeMock.java] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingPlatformCopyTradeMock.java) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingPlatformCopyTradeMock.java#L1-18)
- [CopyTradingPlatformCopyTradeFactory.java] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingPlatformCopyTradeFactory.java) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingPlatformCopyTradeFactory.java#L1-18)
- [CopyTradingPlatformCopyTradeExceptionHandler.java] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingPlatformCopyTradeExceptionHandler.java) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingPlatformCopyTradeExceptionHandler.java#L1-18)
- [CopyTradingPlatformCopyTradeInterceptor.java] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingPlatformCopyTradeInterceptor.java) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingPlatformCopyTradeInterceptor.java#L1-18)
- [CopyTradingPlatformCopyTradeFilter.java] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingPlatformCopyTradeFilter.java) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingPlatformCopyTradeFilter.java#L1-18)
- [CopyTradingPlatformCopyTradeValidator.java] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingPlatformCopyTradeValidator.java) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingPlatformCopyTradeValidator.java#L1-18)
- [CopyTradingPlatformCopyTradeConverter.java] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingPlatformCopyTradeConverter.java) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingPlatformCopyTradeConverter.java#L1-18)
- [CopyTradingPlatformCopyTradeSerializer.java] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingPlatformCopyTradeSerializer.java) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingPlatformCopyTradeSerializer.java#L1-18)
- [CopyTradingPlatformCopyTradeDeserializer.java] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingPlatformCopyTradeDeserializer.java) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingPlatformCopyTradeDeserializer.java#L1-18)
- [CopyTradingPlatformCopyTradeHandler.java] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingPlatformCopyTradeHandler.java) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingPlatformCopyTradeHandler.java#L1-18)
- [CopyTradingPlatformCopyTradeListener.java] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingPlatformCopyTradeListener.java) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingPlatformCopyTradeListener.java#L1-18)
- [CopyTradingPlatformCopyTradeObserver.java] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingPlatformCopyTradeObserver.java</
```


These entities are defined using JHipster's entity configuration system and follow the same

Sources: [1] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
 [2] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>)
 [3] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>)
 [4] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>)
 [5] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-46>)

* * *

Entity Architecture Diagram

! [Chart 1] (nhsv-admin-images/nhsv-admin-section13-1.svg)

Diagram: Supporting Entities and Their System Integration

This diagram shows how supporting entities integrate with core business entities. The `Holiday` entity is a core business entity that integrates with the `MarketHistoryJobResult` entity.

Sources: [1] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
 [2] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>)
 [3] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>)
 [4] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-46>)

* * *

Holiday Entity

Purpose

The `Holiday` entity maintains a calendar of market holidays and non-trading days. This includes:

- Determining valid trading windows
- Scheduling automated copy trading operations
- Calculating business day intervals for performance metrics
- Displaying market availability to users

Entity Definition

The `Holiday` entity is defined in `.jhipster/Holiday.json` and generates a JPA entity mapping.

Sources: [1] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
 [2] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json6>)
 [3] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>)
 [4] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-46>)

Field Structure

Field Name	Type	Constraints	Description
`year`	`Integer`	Required	Calendar year for the holiday
`eventHoliday`	`String`	Required, Max 255 chars	Name or description of the holiday
`startDate`	`ZonedDateTime`	None	Holiday period start timestamp
`endDate`	`ZonedDateTime`	None	Holiday period end timestamp
`createdAt`	`ZonedDateTime`	Required	Record creation timestamp
`updatedAt`	`ZonedDateTime`	None	Last modification timestamp

Sources: [1] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
 [2] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json7-35>)
 [3] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>)
 [4] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-46>)

Entity Structure Diagram

! [Chart 2] (nhsv-admin-images/nhsv-admin-section13-2.svg)

Diagram: Holiday Entity Class Structure

This diagram shows the generated backend components for the `Holiday` entity. The entity fo

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json4] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json4>)

[.jhipster/Holiday.json41-44] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json41-44>)

Configuration Details

JHipster Settings:

Setting	Value	Meaning
dto	mapstruct	Uses MapStruct for entity-DTO mapping
service	serviceImpl	Generates service interface and implementation
pagination	pagination	Enables paginated REST endpoints
jpaMetamodelFiltering	false	Disables JPA Metamodel filtering
incrementalChangelog	true	Uses incremental Liquibase changelogs

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json4-44] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json4-44>)

Time Range Representation

The `Holiday` entity supports flexible holiday duration modeling through `startDate` and `e

- ****Single-day holidays**:** `startDate` set to beginning of day, `endDate` set to end of day
- ****Multi-day closures**:** `startDate` and `endDate` define a continuous non-trading period
- ****Null dates**:** Fields are optional, allowing for year-only holiday records

The use of `ZonedDateTime` ensures proper timezone handling across different market regions

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json20-25] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json20-25>)

* * *

MarketHistoryJobResult Entity

Purpose

The `MarketHistoryJobResult` entity tracks the execution results of background jobs that p

- Audit trail for data processing operations
- Error tracking and debugging information
- Job performance monitoring (start/end times)
- User accountability for job execution

Entity Definition

The `MarketHistoryJobResult` entity is defined in `.jhipster/MarketHistoryJobResult.json` &

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

[.jhipster/MarketHistoryJobResult.json1-46] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>)

Field Structure

Field Name	Type	Constraints	Description
`isSuccess`	`Boolean`	None	Whether the job completed successfully
`timeStart`	`ZonedDateTime`	None	Job execution start timestamp
`timeEnd`	`ZonedDateTime`	None	Job execution end timestamp
`error`	`String`	None	Error message if job failed
`eventId`	`String`	None	Identifier for the specific job event
`symbols`	`String`	None	Market symbols processed by the job

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Relationship Structure

The entity has a **many-to-one** relationship with the ``User`` entity, tracking which user triggered the job.

Relationship Property	Value
Relationship Type	<code>`many-to-one`</code>
Related Entity	<code>`user`</code>
Owner Side	<code>`true`</code>
Display Field	<code>`login`</code>

This relationship enables:

- Tracking job execution by user
- Filtering jobs by user login
- Auditing who triggered specific data processing operations

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Entity Relationship Diagram

! [Chart 3] (nhsv-admin-images/nhsv-admin-section13-3.svg)

****Diagram: MarketHistoryJobResult Entity Relationships****

This ER diagram shows the relationship between ``User`` and ``MarketHistoryJobResult``. Each job is associated with exactly one user.

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Configuration Details

****JHipster Settings**:**

Setting	Value	Meaning
<code>`dto`</code>	Not specified	Uses entity directly (no DTO generation)
<code>`service`</code>	<code>`no`</code>	No service layer generation; direct repository access
<code>`pagination`</code>	<code>`no`</code>	No pagination on REST endpoints
<code>`incrementalChangelog`</code>	<code>`true`</code>	Uses incremental Liquibase changelogs

The absence of DTO and service layer generation suggests this entity is primarily used for data processing or reporting.

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>]

Job Execution Tracking

![Chart 4] (nhsv-admin-images/nhsv-admin-section13-4.svg)

****Diagram: Market History Job Execution Flow****

This sequence diagram illustrates how the `MarketHistoryJobResult` entity tracks job lifecycle.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>)

Symbols Field Usage

The `symbols` field stores market symbols processed during the job, likely as a comma-separated list.

- Understanding job scope (which symbols were included)
- Filtering failed jobs by specific symbols
- Debugging symbol-specific processing issues

****Example storage formats**:**

- Comma-separated: `"AAPL,GOOGL,MSFT"`
- JSON array: `["AAPL","GOOGL","MSFT"]`

The actual format depends on the implementation of the job processing logic.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json27-28>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json27-28>)

* * *

System Integration Pattern

![Chart 5] (nhsv-admin-images/nhsv-admin-section13-5.svg)

****Diagram: Supporting Entities in System Context****

This diagram shows how supporting entities integrate with external dependencies and business logic.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>)

* * *

Comparison Table

Aspect	Holiday	MarketHistoryJobResult
Primary Purpose	Define non-trading days	Track background job execution
Data Source	Manual entry or calendar API	Generated by job scheduler
Update Frequency	Annually (typically)	Per job execution
Core Fields	`year`, `eventHoliday`, date range	`isSuccess`, `error`, time range
Relationships	None	Many-to-one with User
DTO Generation	Yes (MapStruct)	No
Service Layer	Yes (serviceImpl)	No
Pagination	Yes	No
Primary Consumers	Trading schedulers, availability checks	System administrators, monitoring tools

```

**Sources**:[] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js
[.jhipster/Holiday.json4-44] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhip
.jhipster/MarketHistoryJobResult.json31-45] (https://github.com/nhsv-tradex/nhsv-admin/blob,

* * *

#### Database Schema

##### Holiday Table Structure

```

-- t_holiday table

```

CREATE TABLE t_holiday (
  id BIGSERIAL PRIMARY KEY,
  year INTEGER NOT NULL,
  event_holiday VARCHAR(255) NOT NULL,
  start_date TIMESTAMP WITH TIME ZONE,
  end_date TIMESTAMP WITH TIME ZONE,
  created_at TIMESTAMP WITH TIME ZONE NOT NULL,
  updated_at TIMESTAMP WITH TIME ZONE
);

```

```

##### MarketHistoryJobResult Table Structure

```

-- market_history_job_result table

```

CREATE TABLE market_history_job_result (
  id BIGSERIAL PRIMARY KEY,
  is_success BOOLEAN,
  time_start TIMESTAMP WITH TIME ZONE,
  time_end TIMESTAMP WITH TIME ZONE,
  error TEXT,
  event_id VARCHAR(255),
  symbols TEXT,
  user_id BIGINT,
  FOREIGN KEY (user_id) REFERENCES jhi_user(id)
);

```

```

These schemas are managed by Liquibase changelogs generated from the JHipster entity definition.

**Sources**: [] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json)
[.jhipster/Holiday.json7-35] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-43] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json4-43] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json5-43]

* * *

#### Summary

Supporting entities provide essential infrastructure services that enable core business operations.

- **Holiday**: Enables time-aware business logic by defining non-trading periods, critical for accurate market data processing.
- **MarketHistoryJobResult**: Provides observability into asynchronous data processing, ensuring transparency and auditability.

Both entities leverage JHipster's code generation to provide consistent REST APIs, database schemas, and frontend components.

**Sources**: [] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json)
[.jhipster/Holiday.json1-45] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json2-46] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-46]

---

### Market Leader Management

Relevant source files

- [https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json)
- [https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json)

#### Purpose and Scope

This document describes supporting entities that provide ancillary functionality to the core trading platform.

The two supporting entities covered in this document are:

- **Holiday**: Manages market holidays and non-trading days
- **MarketHistoryJobResult**: Tracks execution results for market data processing jobs

For information about the core copy trading entities, see [Copy Trading Platform] (#copy-trading-platform)

* * *

#### Overview

Supporting entities serve operational and administrative needs that span multiple business units.

1. **Temporal Business Logic**: The `Holiday` entity enables the system to account for market closures and non-trading periods.
2. **Job Observability**: The `MarketHistoryJobResult` entity provides audit trails and performance metrics for asynchronous data processing jobs.

```

These entities are defined using JHipster's entity configuration system and follow the same

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
[.jhipster/Holiday.json1-45] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

* * *

Entity Architecture Diagram

! [Chart 1] (nhsv-admin-images/nhsv-admin-section14-1.svg)

Diagram: Supporting Entities and Their System Integration

This diagram shows how supporting entities integrate with core business entities. The `Holiday` entity is a core business entity, while `MarketHistoryJobResult` is a supporting entity.

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
[.jhipster/Holiday.json1-45] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

* * *

Holiday Entity

Purpose

The `Holiday` entity maintains a calendar of market holidays and non-trading days. This includes:

- Determining valid trading windows
- Scheduling automated copy trading operations
- Calculating business day intervals for performance metrics
- Displaying market availability to users

Entity Definition

The `Holiday` entity is defined in `.jhipster/Holiday.json` and generates a JPA entity mapping.

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
[.jhipster/Holiday.json6] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

Field Structure

Field Name	Type	Constraints	Description
`year`	`Integer`	Required	Calendar year for the holiday
`eventHoliday`	`String`	Required, Max 255 chars	Name or description of the holiday
`startDate`	`ZonedDateTime`	None	Holiday period start timestamp
`endDate`	`ZonedDateTime`	None	Holiday period end timestamp
`createdAt`	`ZonedDateTime`	Required	Record creation timestamp
`updatedAt`	`ZonedDateTime`	None	Last modification timestamp

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
[.jhipster/Holiday.json7-35] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

Entity Structure Diagram

! [Chart 2] (nhsv-admin-images/nhsv-admin-section14-2.svg)

Diagram: Holiday Entity Class Structure

This diagram shows the generated backend components for the `Holiday` entity. The entity fo

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json4] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json4>)

[.jhipster/Holiday.json41-44] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json41-44>)

Configuration Details

JHipster Settings:

Setting	Value	Meaning
dto	mapstruct	Uses MapStruct for entity-DTO mapping
service	serviceImpl	Generates service interface and implementation
pagination	pagination	Enables paginated REST endpoints
jpaMetamodelFiltering	false	Disables JPA Metamodel filtering
incrementalChangelog	true	Uses incremental Liquibase changelogs

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json4-44] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json4-44>)

Time Range Representation

The `Holiday` entity supports flexible holiday duration modeling through `startDate` and `e

- ****Single-day holidays**:** `startDate` set to beginning of day, `endDate` set to end of day
- ****Multi-day closures**:** `startDate` and `endDate` define a continuous non-trading period
- ****Null dates**:** Fields are optional, allowing for year-only holiday records

The use of `ZonedDateTime` ensures proper timezone handling across different market regions

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json20-25] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json20-25>)

* * *

MarketHistoryJobResult Entity

Purpose

The `MarketHistoryJobResult` entity tracks the execution results of background jobs that p

- Audit trail for data processing operations
- Error tracking and debugging information
- Job performance monitoring (start/end times)
- User accountability for job execution

Entity Definition

The `MarketHistoryJobResult` entity is defined in `.jhipster/MarketHistoryJobResult.json` &

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

[.jhipster/MarketHistoryJobResult.json1-46] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>)

Field Structure

Field Name	Type	Constraints	Description
`isSuccess`	`Boolean`	None	Whether the job completed successfully
`timeStart`	`ZonedDateTime`	None	Job execution start timestamp
`timeEnd`	`ZonedDateTime`	None	Job execution end timestamp
`error`	`String`	None	Error message if job failed
`eventId`	`String`	None	Identifier for the specific job event
`symbols`	`String`	None	Market symbols processed by the job

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>]

Relationship Structure

The entity has a ****many-to-one**** relationship with the ``User`` entity, tracking which user triggered the job.

Relationship Property	Value
Relationship Type	<code>`many-to-one`</code>
Related Entity	<code>`user`</code>
Owner Side	<code>`true`</code>
Display Field	<code>`login`</code>

This relationship enables:

- Tracking job execution by user
- Filtering jobs by user login
- Auditing who triggered specific data processing operations

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>]

Entity Relationship Diagram

! [Chart 3] (nhsv-admin-images/nhsv-admin-section14-3.svg)

****Diagram: MarketHistoryJobResult Entity Relationships****

This ER diagram shows the relationship between ``User`` and ``MarketHistoryJobResult``. Each job is associated with exactly one user.

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>]

Configuration Details

****JHipster Settings**:**

Setting	Value	Meaning
<code>`dto`</code>	Not specified	Uses entity directly (no DTO generation)
<code>`service`</code>	<code>`no`</code>	No service layer generation; direct repository access
<code>`pagination`</code>	<code>`no`</code>	No pagination on REST endpoints
<code>`incrementalChangelog`</code>	<code>`true`</code>	Uses incremental Liquibase changelogs

The absence of DTO and service layer generation suggests this entity is primarily used for data processing or reporting.

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Job Execution Tracking

![Chart 4] (nhsv-admin-images/nhsv-admin-section14-4.svg)

****Diagram: Market History Job Execution Flow****

This sequence diagram illustrates how the `MarketHistoryJobResult` entity tracks job lifecycle.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>)

Symbols Field Usage

The `symbols` field stores market symbols processed during the job, likely as a comma-separated list.

- Understanding job scope (which symbols were included)
- Filtering failed jobs by specific symbols
- Debugging symbol-specific processing issues

****Example storage formats**:**

- Comma-separated: `"AAPL,GOOGL,MSFT"`
- JSON array: `["AAPL","GOOGL","MSFT"]`

The actual format depends on the implementation of the job processing logic.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json27-28>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json27-28>)

* * *

System Integration Pattern

![Chart 5] (nhsv-admin-images/nhsv-admin-section14-5.svg)

****Diagram: Supporting Entities in System Context****

This diagram shows how supporting entities integrate with external dependencies and business logic.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>)

* * *

Comparison Table

Aspect	Holiday	MarketHistoryJobResult
Primary Purpose	Define non-trading days	Track background job execution
Data Source	Manual entry or calendar API	Generated by job scheduler
Update Frequency	Annually (typically)	Per job execution
Core Fields	`year`, `eventHoliday`, date range	`isSuccess`, `error`, time range
Relationships	None	Many-to-one with User
DTO Generation	Yes (MapStruct)	No
Service Layer	Yes (serviceImpl)	No
Pagination	Yes	No
Primary Consumers	Trading schedulers, availability checks	System administrators, monitoring tools

```

**Sources**:[] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js
[.jhipster/Holiday.json4-44] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhip
.jhipster/MarketHistoryJobResult.json31-45] (https://github.com/nhsv-tradex/nhsv-admin/blob,

* * *

#### Database Schema

##### Holiday Table Structure

```

-- t_holiday table

```

CREATE TABLE t_holiday (
  id BIGSERIAL PRIMARY KEY,
  year INTEGER NOT NULL,
  event_holiday VARCHAR(255) NOT NULL,
  start_date TIMESTAMP WITH TIME ZONE,
  end_date TIMESTAMP WITH TIME ZONE,
  created_at TIMESTAMP WITH TIME ZONE NOT NULL,
  updated_at TIMESTAMP WITH TIME ZONE
);

```

```

##### MarketHistoryJobResult Table Structure

```

-- market_history_job_result table

```

CREATE TABLE market_history_job_result (
  id BIGSERIAL PRIMARY KEY,
  is_success BOOLEAN,
  time_start TIMESTAMP WITH TIME ZONE,
  time_end TIMESTAMP WITH TIME ZONE,
  error TEXT,
  event_id VARCHAR(255),
  symbols TEXT,
  user_id BIGINT,
  FOREIGN KEY (user_id) REFERENCES jhi_user(id)
);

```

These schemas are managed by Liquibase changelogs generated from the JHipster entity definition.

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

* * *

Summary

Supporting entities provide essential infrastructure services that enable core business operations.

- **Holiday**: Enables time-aware business logic by defining non-trading periods, critical for accurate market data processing.
- **MarketHistoryJobResult**: Provides observability into asynchronous data processing, ensuring transparency and reliability of market data.

Both entities leverage JHipster's code generation to provide consistent REST APIs, database schemas, and frontend components.

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

Market Leader Profile

Relevant source files

- [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

Purpose and Scope

This document describes supporting entities that provide ancillary functionality to the core trading platform.

The two supporting entities covered in this document are:

- **Holiday**: Manages market holidays and non-trading days
- **MarketHistoryJobResult**: Tracks execution results for market data processing jobs

For information about the core copy trading entities, see [Copy Trading Platform] (#copy-trading-platform).

* * *

Overview

Supporting entities serve operational and administrative needs that span multiple business units.

1. **Temporal Business Logic**: The `Holiday` entity enables the system to account for market holidays and non-trading days.
2. **Job Observability**: The `MarketHistoryJobResult` entity provides audit trails and execution details for market data processing jobs.

These entities are defined using JHipster's entity configuration system and follow the same

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
[.jhipster/Holiday.json1-45] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

* * *

Entity Architecture Diagram

! [Chart 1] (nhsv-admin-images/nhsv-admin-section15-1.svg)

Diagram: Supporting Entities and Their System Integration

This diagram shows how supporting entities integrate with core business entities. The `Holiday` entity is a core business entity that integrates with the `MarketHistoryJobResult` entity.

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
[.jhipster/Holiday.json1-45] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

* * *

Holiday Entity

Purpose

The `Holiday` entity maintains a calendar of market holidays and non-trading days. This includes:

- Determining valid trading windows
- Scheduling automated copy trading operations
- Calculating business day intervals for performance metrics
- Displaying market availability to users

Entity Definition

The `Holiday` entity is defined in `.jhipster/Holiday.json` and generates a JPA entity mapping.

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
[.jhipster/Holiday.json6] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

Field Structure

Field Name	Type	Constraints	Description
`year`	`Integer`	Required	Calendar year for the holiday
`eventHoliday`	`String`	Required, Max 255 chars	Name or description of the holiday
`startDate`	`ZonedDateTime`	None	Holiday period start timestamp
`endDate`	`ZonedDateTime`	None	Holiday period end timestamp
`createdAt`	`ZonedDateTime`	Required	Record creation timestamp
`updatedAt`	`ZonedDateTime`	None	Last modification timestamp

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
[.jhipster/Holiday.json7-35] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

Entity Structure Diagram

! [Chart 2] (nhsv-admin-images/nhsv-admin-section15-2.svg)

Diagram: Holiday Entity Class Structure

This diagram shows the generated backend components for the `Holiday` entity. The entity fo

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json4] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json4>)

[.jhipster/Holiday.json41-44] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json41-44>)

Configuration Details

JHipster Settings:

Setting	Value	Meaning
dto	mapstruct	Uses MapStruct for entity-DTO mapping
service	serviceImpl	Generates service interface and implementation
pagination	pagination	Enables paginated REST endpoints
jpaMetamodelFiltering	false	Disables JPA Metamodel filtering
incrementalChangelog	true	Uses incremental Liquibase changelogs

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json4-44] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json4-44>)

Time Range Representation

The `Holiday` entity supports flexible holiday duration modeling through `startDate` and `e

- ****Single-day holidays**:** `startDate` set to beginning of day, `endDate` set to end of day
- ****Multi-day closures**:** `startDate` and `endDate` define a continuous non-trading period
- ****Null dates**:** Fields are optional, allowing for year-only holiday records

The use of `ZonedDateTime` ensures proper timezone handling across different market regions

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json20-25] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json20-25>)

* * *

MarketHistoryJobResult Entity

Purpose

The `MarketHistoryJobResult` entity tracks the execution results of background jobs that p

- Audit trail for data processing operations
- Error tracking and debugging information
- Job performance monitoring (start/end times)
- User accountability for job execution

Entity Definition

The `MarketHistoryJobResult` entity is defined in `.jhipster/MarketHistoryJobResult.json` &

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

[.jhipster/MarketHistoryJobResult.json1-46] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>)

Field Structure

Field Name	Type	Constraints	Description
`isSuccess`	`Boolean`	None	Whether the job completed successfully
`timeStart`	`ZonedDateTime`	None	Job execution start timestamp
`timeEnd`	`ZonedDateTime`	None	Job execution end timestamp
`error`	`String`	None	Error message if job failed
`eventId`	`String`	None	Identifier for the specific job event
`symbols`	`String`	None	Market symbols processed by the job

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Relationship Structure

The entity has a **many-to-one** relationship with the ``User`` entity, tracking which user triggered the job.

Relationship Property	Value
Relationship Type	<code>`many-to-one`</code>
Related Entity	<code>`user`</code>
Owner Side	<code>`true`</code>
Display Field	<code>`login`</code>

This relationship enables:

- Tracking job execution by user
- Filtering jobs by user login
- Auditing who triggered specific data processing operations

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Entity Relationship Diagram

! [Chart 3] (nhsv-admin-images/nhsv-admin-section15-3.svg)

****Diagram: MarketHistoryJobResult Entity Relationships****

This ER diagram shows the relationship between ``User`` and ``MarketHistoryJobResult``. Each job is associated with exactly one user.

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Configuration Details

****JHipster Settings**:**

Setting	Value	Meaning
<code>`dto`</code>	Not specified	Uses entity directly (no DTO generation)
<code>`service`</code>	<code>`no`</code>	No service layer generation; direct repository access
<code>`pagination`</code>	<code>`no`</code>	No pagination on REST endpoints
<code>`incrementalChangelog`</code>	<code>`true`</code>	Uses incremental Liquibase changelogs

The absence of DTO and service layer generation suggests this entity is primarily used for data processing or reporting.

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>]

Job Execution Tracking

![Chart 4] (nhsv-admin-images/nhsv-admin-section15-4.svg)

****Diagram: Market History Job Execution Flow****

This sequence diagram illustrates how the `MarketHistoryJobResult` entity tracks job lifecycle.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>)

Symbols Field Usage

The `symbols` field stores market symbols processed during the job, likely as a comma-separated list.

- Understanding job scope (which symbols were included)
- Filtering failed jobs by specific symbols
- Debugging symbol-specific processing issues

****Example storage formats**:**

- Comma-separated: `"AAPL,GOOGL,MSFT"`
- JSON array: `["AAPL","GOOGL","MSFT"]`

The actual format depends on the implementation of the job processing logic.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json27-28>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json27-28>)

* * *

System Integration Pattern

![Chart 5] (nhsv-admin-images/nhsv-admin-section15-5.svg)

****Diagram: Supporting Entities in System Context****

This diagram shows how supporting entities integrate with external dependencies and business logic.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>)

[.jhipster/MarketHistoryJobResult.json1-46] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>)

* * *

Comparison Table

Aspect	Holiday	MarketHistoryJobResult
Primary Purpose	Define non-trading days	Track background job execution
Data Source	Manual entry or calendar API	Generated by job scheduler
Update Frequency	Annually (typically)	Per job execution
Core Fields	`year`, `eventHoliday`, date range	`isSuccess`, `error`, time range
Relationships	None	Many-to-one with User
DTO Generation	Yes (MapStruct)	No
Service Layer	Yes (serviceImpl)	No
Pagination	Yes	No
Primary Consumers	Trading schedulers, availability checks	System administrators, monitoring tools


```

**Sources**:[] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js
[.jhipster/Holiday.json4-44] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhip
.jhipster/MarketHistoryJobResult.json31-45] (https://github.com/nhsv-tradex/nhsv-admin/blob,

* * *

#### Database Schema

##### Holiday Table Structure

```

-- t_holiday table

```

CREATE TABLE t_holiday (
  id BIGSERIAL PRIMARY KEY,
  year INTEGER NOT NULL,
  event_holiday VARCHAR(255) NOT NULL,
  start_date TIMESTAMP WITH TIME ZONE,
  end_date TIMESTAMP WITH TIME ZONE,
  created_at TIMESTAMP WITH TIME ZONE NOT NULL,
  updated_at TIMESTAMP WITH TIME ZONE
);

```

```

##### MarketHistoryJobResult Table Structure

```

-- market_history_job_result table

```

CREATE TABLE market_history_job_result (
  id BIGSERIAL PRIMARY KEY,
  is_success BOOLEAN,
  time_start TIMESTAMP WITH TIME ZONE,
  time_end TIMESTAMP WITH TIME ZONE,
  error TEXT,
  event_id VARCHAR(255),
  symbols TEXT,
  user_id BIGINT,
  FOREIGN KEY (user_id) REFERENCES jhi_user(id)
);

```

These schemas are managed by Liquibase changelogs generated from the JHipster entity definition.

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

* * *

Summary

Supporting entities provide essential infrastructure services that enable core business operations.

- **Holiday**: Enables time-aware business logic by defining non-trading periods, critical for accurate market data processing.
- **MarketHistoryJobResult**: Provides observability into asynchronous data processing, ensuring transparency and reliability of market data.

Both entities leverage JHipster's code generation to provide consistent REST APIs, database schemas, and frontend components.

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

Performance Tracking

Relevant source files

- [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)
- [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

Purpose and Scope

This document describes supporting entities that provide ancillary functionality to the core trading platform.

The two supporting entities covered in this document are:

- **Holiday**: Manages market holidays and non-trading days
- **MarketHistoryJobResult**: Tracks execution results for market data processing jobs

For information about the core copy trading entities, see [Copy Trading Platform] (#copy-trading-platform).

* * *

Overview

Supporting entities serve operational and administrative needs that span multiple business units.

1. **Temporal Business Logic**: The `Holiday` entity enables the system to account for market holidays and non-trading days.
2. **Job Observability**: The `MarketHistoryJobResult` entity provides audit trails and execution details for market data processing jobs.

These entities are defined using JHipster's entity configuration system and follow the same

Sources: [1] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
 [2] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>)
 [3] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>)
 [4] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>)
 [5] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-46>)

* * *

Entity Architecture Diagram

! [Chart 1] (nhsv-admin-images/nhsv-admin-section16-1.svg)

Diagram: Supporting Entities and Their System Integration

This diagram shows how supporting entities integrate with core business entities. The `Holiday` entity is a supporting entity that integrates with the `MarketHistoryJobResult` entity.

Sources: [1] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
 [2] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>)
 [3] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>)
 [4] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-46>)

* * *

Holiday Entity

Purpose

The `Holiday` entity maintains a calendar of market holidays and non-trading days. This includes:

- Determining valid trading windows
- Scheduling automated copy trading operations
- Calculating business day intervals for performance metrics
- Displaying market availability to users

Entity Definition

The `Holiday` entity is defined in `.jhipster/Holiday.json` and generates a JPA entity mapping.

Sources: [1] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
 [2] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json6>)
 [3] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>)
 [4] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-46>)

Field Structure

Field Name	Type	Constraints	Description
`year`	`Integer`	Required	Calendar year for the holiday
`eventHoliday`	`String`	Required, Max 255 chars	Name or description of the holiday
`startDate`	`ZonedDateTime`	None	Holiday period start timestamp
`endDate`	`ZonedDateTime`	None	Holiday period end timestamp
`createdAt`	`ZonedDateTime`	Required	Record creation timestamp
`updatedAt`	`ZonedDateTime`	None	Last modification timestamp

Sources: [1] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
 [2] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json7-35>)
 [3] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>)
 [4] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-46>)

Entity Structure Diagram

! [Chart 2] (nhsv-admin-images/nhsv-admin-section16-2.svg)

Diagram: Holiday Entity Class Structure

This diagram shows the generated backend components for the `Holiday` entity. The entity fo

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json4] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json4>)

[.jhipster/Holiday.json41-44] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json41-44>)

Configuration Details

JHipster Settings:

Setting	Value	Meaning
dto	mapstruct	Uses MapStruct for entity-DTO mapping
service	serviceImpl	Generates service interface and implementation
pagination	pagination	Enables paginated REST endpoints
jpaMetamodelFiltering	false	Disables JPA Metamodel filtering
incrementalChangelog	true	Uses incremental Liquibase changelogs

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json4-44] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json4-44>)

Time Range Representation

The `Holiday` entity supports flexible holiday duration modeling through `startDate` and `e

- ****Single-day holidays**:** `startDate` set to beginning of day, `endDate` set to end of day
- ****Multi-day closures**:** `startDate` and `endDate` define a continuous non-trading period
- ****Null dates**:** Fields are optional, allowing for year-only holiday records

The use of `ZonedDateTime` ensures proper timezone handling across different market regions

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json20-25] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json20-25>)

* * *

MarketHistoryJobResult Entity

Purpose

The `MarketHistoryJobResult` entity tracks the execution results of background jobs that p

- Audit trail for data processing operations
- Error tracking and debugging information
- Job performance monitoring (start/end times)
- User accountability for job execution

Entity Definition

The `MarketHistoryJobResult` entity is defined in `.jhipster/MarketHistoryJobResult.json` &

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

[.jhipster/MarketHistoryJobResult.json1-46] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>)

Field Structure

Field Name	Type	Constraints	Description
`isSuccess`	`Boolean`	None	Whether the job completed successfully
`timeStart`	`ZonedDateTime`	None	Job execution start timestamp
`timeEnd`	`ZonedDateTime`	None	Job execution end timestamp
`error`	`String`	None	Error message if job failed
`eventId`	`String`	None	Identifier for the specific job event
`symbols`	`String`	None	Market symbols processed by the job

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Relationship Structure

The entity has a **many-to-one** relationship with the ``User`` entity, tracking which user triggered the job.

Relationship Property	Value
Relationship Type	<code>`many-to-one`</code>
Related Entity	<code>`user`</code>
Owner Side	<code>`true`</code>
Display Field	<code>`login`</code>

This relationship enables:

- Tracking job execution by user
- Filtering jobs by user login
- Auditing who triggered specific data processing operations

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Entity Relationship Diagram

! [Chart 3] (nhsv-admin-images/nhsv-admin-section16-3.svg)

****Diagram: MarketHistoryJobResult Entity Relationships****

This ER diagram shows the relationship between ``User`` and ``MarketHistoryJobResult``. Each job is associated with exactly one user.

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Configuration Details

****JHipster Settings**:**

Setting	Value	Meaning
<code>`dto`</code>	Not specified	Uses entity directly (no DTO generation)
<code>`service`</code>	<code>`no`</code>	No service layer generation; direct repository access
<code>`pagination`</code>	<code>`no`</code>	No pagination on REST endpoints
<code>`incrementalChangelog`</code>	<code>`true`</code>	Uses incremental Liquibase changelogs

The absence of DTO and service layer generation suggests this entity is primarily used for data processing or reporting.

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>]

Job Execution Tracking

![Chart 4] (nhsv-admin-images/nhsv-admin-section16-4.svg)

****Diagram: Market History Job Execution Flow****

This sequence diagram illustrates how the `MarketHistoryJobResult` entity tracks job lifecycle.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>)

Symbols Field Usage

The `symbols` field stores market symbols processed during the job, likely as a comma-separated list.

- Understanding job scope (which symbols were included)
- Filtering failed jobs by specific symbols
- Debugging symbol-specific processing issues

****Example storage formats**:**

- Comma-separated: `"AAPL,GOOGL,MSFT"`
- JSON array: `["AAPL","GOOGL","MSFT"]`

The actual format depends on the implementation of the job processing logic.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json27-28>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json27-28>)

* * *

System Integration Pattern

![Chart 5] (nhsv-admin-images/nhsv-admin-section16-5.svg)

****Diagram: Supporting Entities in System Context****

This diagram shows how supporting entities integrate with external dependencies and business logic.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>)

[.jhipster/MarketHistoryJobResult.json1-46] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>)

* * *

Comparison Table

Aspect	Holiday	MarketHistoryJobResult
Primary Purpose	Define non-trading days	Track background job execution
Data Source	Manual entry or calendar API	Generated by job scheduler
Update Frequency	Annually (typically)	Per job execution
Core Fields	`year`, `eventHoliday`, date range	`isSuccess`, `error`, time range
Relationships	None	Many-to-one with User
DTO Generation	Yes (MapStruct)	No
Service Layer	Yes (serviceImpl)	No
Pagination	Yes	No
Primary Consumers	Trading schedulers, availability checks	System administrators, monitoring tools

```

**Sources**:[] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js
[.jhipster/Holiday.json4-44] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhip
.jhipster/MarketHistoryJobResult.json31-45] (https://github.com/nhsv-tradex/nhsv-admin/blob,

* * *

#### Database Schema

##### Holiday Table Structure

```

-- t_holiday table

```

CREATE TABLE t_holiday (
  id BIGSERIAL PRIMARY KEY,
  year INTEGER NOT NULL,
  event_holiday VARCHAR(255) NOT NULL,
  start_date TIMESTAMP WITH TIME ZONE,
  end_date TIMESTAMP WITH TIME ZONE,
  created_at TIMESTAMP WITH TIME ZONE NOT NULL,
  updated_at TIMESTAMP WITH TIME ZONE
);

```

```

##### MarketHistoryJobResult Table Structure

```

-- market_history_job_result table

```

CREATE TABLE market_history_job_result (
  id BIGSERIAL PRIMARY KEY,
  is_success BOOLEAN,
  time_start TIMESTAMP WITH TIME ZONE,
  time_end TIMESTAMP WITH TIME ZONE,
  error TEXT,
  event_id VARCHAR(255),
  symbols TEXT,
  user_id BIGINT,
  FOREIGN KEY (user_id) REFERENCES jhi_user(id)
);

```

```

These schemas are managed by Liquibase changelogs generated from the JHipster entity defin

**Sources**: [] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js
[jhipster/Holiday.json7-35] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhip
.jhipster/MarketHistoryJobResult.json3-43] (https://github.com/nhsv-tradex/nhsv-admin/blob/

* * *

#### Summary

Supporting entities provide essential infrastructure services that enable core business op

- **Holiday**: Enables time-aware business logic by defining non-trading periods, critica
- **MarketHistoryJobResult**: Provides observability into asynchronous data processing, e

Both entities leverage JHipster's code generation to provide consistent REST APIs, database

**Sources**: [] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js
[jhipster/Holiday.json1-45] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhip
.jhipster/MarketHistoryJobResult.json1-46] (https://github.com/nhsv-tradex/nhsv-admin/blob/

---

### Portfolio Management

Relevant source files

- [
.jhipster/Holiday.json] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhips
- [
.jhipster/MarketHistoryJobResult.json] (https://github.com/nhsv-tradex/nhsv-admin/blob/

#### Purpose and Scope

This document describes supporting entities that provide ancillary functionality to the co

The two supporting entities covered in this document are:

- **Holiday**: Manages market holidays and non-trading days
- **MarketHistoryJobResult**: Tracks execution results for market data processing jobs

For information about the core copy trading entities, see [Copy Trading Platform] (#copy-tra

* * *

#### Overview

Supporting entities serve operational and administrative needs that span multiple business

1. **Temporal Business Logic**: The `Holiday` entity enables the system to account for mar
2. **Job Observability**: The `MarketHistoryJobResult` entity provides audit trails and en

```


These entities are defined using JHipster's entity configuration system and follow the same

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
 [.jhipster/Holiday.json1-45] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

* * *

Entity Architecture Diagram

![Chart 1] (nhsv-admin-images/nhsv-admin-section17-1.svg)

****Diagram: Supporting Entities and Their System Integration****

This diagram shows how supporting entities integrate with core business entities. The `Holiday` entity is a core business entity that integrates with the `MarketHistoryJobResult` entity.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
 [.jhipster/Holiday.json1-45] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

* * *

Holiday Entity

Purpose

The `Holiday` entity maintains a calendar of market holidays and non-trading days. This includes:

- Determining valid trading windows
- Scheduling automated copy trading operations
- Calculating business day intervals for performance metrics
- Displaying market availability to users

Entity Definition

The `Holiday` entity is defined in `.jhipster/Holiday.json` and generates a JPA entity mapping.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
 [.jhipster/Holiday.json6] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

Field Structure

Field Name	Type	Constraints	Description
`year`	`Integer`	Required	Calendar year for the holiday
`eventHoliday`	`String`	Required, Max 255 chars	Name or description of the holiday
`startDate`	`ZonedDateTime`	None	Holiday period start timestamp
`endDate`	`ZonedDateTime`	None	Holiday period end timestamp
`createdAt`	`ZonedDateTime`	Required	Record creation timestamp
`updatedAt`	`ZonedDateTime`	None	Last modification timestamp

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
 [.jhipster/Holiday.json7-35] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

Entity Structure Diagram

![Chart 2] (nhsv-admin-images/nhsv-admin-section17-2.svg)

Diagram: Holiday Entity Class Structure

This diagram shows the generated backend components for the `Holiday` entity. The entity fo

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json4] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json4>)

[.jhipster/Holiday.json41-44] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json41-44>)

Configuration Details

JHipster Settings:

Setting	Value	Meaning
dto	mapstruct	Uses MapStruct for entity-DTO mapping
service	serviceImpl	Generates service interface and implementation
pagination	pagination	Enables paginated REST endpoints
jpaMetamodelFiltering	false	Disables JPA Metamodel filtering
incrementalChangelog	true	Uses incremental Liquibase changelogs

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json4-44] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json4-44>)

Time Range Representation

The `Holiday` entity supports flexible holiday duration modeling through `startDate` and `e

- ****Single-day holidays**:** `startDate` set to beginning of day, `endDate` set to end of day
- ****Multi-day closures**:** `startDate` and `endDate` define a continuous non-trading period
- ****Null dates**:** Fields are optional, allowing for year-only holiday records

The use of `ZonedDateTime` ensures proper timezone handling across different market regions

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json20-25] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json20-25>)

* * *

MarketHistoryJobResult Entity

Purpose

The `MarketHistoryJobResult` entity tracks the execution results of background jobs that p

- Audit trail for data processing operations
- Error tracking and debugging information
- Job performance monitoring (start/end times)
- User accountability for job execution

Entity Definition

The `MarketHistoryJobResult` entity is defined in `.jhipster/MarketHistoryJobResult.json` &

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

[.jhipster/MarketHistoryJobResult.json1-46] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>)

Field Structure

Field Name	Type	Constraints	Description
`isSuccess`	`Boolean`	None	Whether the job completed successfully
`timeStart`	`ZonedDateTime`	None	Job execution start timestamp
`timeEnd`	`ZonedDateTime`	None	Job execution end timestamp
`error`	`String`	None	Error message if job failed
`eventId`	`String`	None	Identifier for the specific job event
`symbols`	`String`	None	Market symbols processed by the job

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Relationship Structure

The entity has a **many-to-one** relationship with the ``User`` entity, tracking which user triggered the job.

Relationship Property	Value
Relationship Type	<code>`many-to-one`</code>
Related Entity	<code>`user`</code>
Owner Side	<code>`true`</code>
Display Field	<code>`login`</code>

This relationship enables:

- Tracking job execution by user
- Filtering jobs by user login
- Auditing who triggered specific data processing operations

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Entity Relationship Diagram

! [Chart 3] (nhsv-admin-images/nhsv-admin-section17-3.svg)

****Diagram: MarketHistoryJobResult Entity Relationships****

This ER diagram shows the relationship between ``User`` and ``MarketHistoryJobResult``. Each job is associated with exactly one user.

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Configuration Details

****JHipster Settings**:**

Setting	Value	Meaning
<code>`dto`</code>	Not specified	Uses entity directly (no DTO generation)
<code>`service`</code>	<code>`no`</code>	No service layer generation; direct repository access
<code>`pagination`</code>	<code>`no`</code>	No pagination on REST endpoints
<code>`incrementalChangelog`</code>	<code>`true`</code>	Uses incremental Liquibase changelogs

The absence of DTO and service layer generation suggests this entity is primarily used for data processing or reporting.

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>]

Job Execution Tracking

![Chart 4] (nhsv-admin-images/nhsv-admin-section17-4.svg)

****Diagram: Market History Job Execution Flow****

This sequence diagram illustrates how the `MarketHistoryJobResult` entity tracks job lifecycle.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>)

Symbols Field Usage

The `symbols` field stores market symbols processed during the job, likely as a comma-separated list.

- Understanding job scope (which symbols were included)
- Filtering failed jobs by specific symbols
- Debugging symbol-specific processing issues

****Example storage formats**:**

- Comma-separated: `"AAPL,GOOGL,MSFT"`
- JSON array: `["AAPL","GOOGL","MSFT"]`

The actual format depends on the implementation of the job processing logic.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json27-28>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json27-28>)

* * *

System Integration Pattern

![Chart 5] (nhsv-admin-images/nhsv-admin-section17-5.svg)

****Diagram: Supporting Entities in System Context****

This diagram shows how supporting entities integrate with external dependencies and business logic.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>)

* * *

Comparison Table

Aspect	Holiday	MarketHistoryJobResult
Primary Purpose	Define non-trading days	Track background job execution
Data Source	Manual entry or calendar API	Generated by job scheduler
Update Frequency	Annually (typically)	Per job execution
Core Fields	`year`, `eventHoliday`, date range	`isSuccess`, `error`, time range
Relationships	None	Many-to-one with User
DTO Generation	Yes (MapStruct)	No
Service Layer	Yes (serviceImpl)	No
Pagination	Yes	No
Primary Consumers	Trading schedulers, availability checks	System administrators, monitoring tools

```

**Sources**:[] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js
[.jhipster/Holiday.json4-44] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhip
.jhipster/MarketHistoryJobResult.json31-45] (https://github.com/nhsv-tradex/nhsv-admin/blob,

* * *

#### Database Schema

##### Holiday Table Structure

```

-- t_holiday table

```

CREATE TABLE t_holiday (
  id BIGSERIAL PRIMARY KEY,
  year INTEGER NOT NULL,
  event_holiday VARCHAR(255) NOT NULL,
  start_date TIMESTAMP WITH TIME ZONE,
  end_date TIMESTAMP WITH TIME ZONE,
  created_at TIMESTAMP WITH TIME ZONE NOT NULL,
  updated_at TIMESTAMP WITH TIME ZONE
);

```

```

##### MarketHistoryJobResult Table Structure

```

-- market_history_job_result table

```

CREATE TABLE market_history_job_result (
  id BIGSERIAL PRIMARY KEY,
  is_success BOOLEAN,
  time_start TIMESTAMP WITH TIME ZONE,
  time_end TIMESTAMP WITH TIME ZONE,
  error TEXT,
  event_id VARCHAR(255),
  symbols TEXT,
  user_id BIGINT,
  FOREIGN KEY (user_id) REFERENCES jhi_user(id)
);

```

These schemas are managed by Liquibase changelogs generated from the JHipster entity definition.

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

* * *

Summary

Supporting entities provide essential infrastructure services that enable core business operations.

- **Holiday**: Enables time-aware business logic by defining non-trading periods, critical for accurate market data processing.
- **MarketHistoryJobResult**: Provides observability into asynchronous data processing, ensuring transparency and reliability of market data.

Both entities leverage JHipster's code generation to provide consistent REST APIs, database schemas, and frontend components.

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

Current Portfolio Structure

Relevant source files

- [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json>)
- [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

Purpose and Scope

This document describes supporting entities that provide ancillary functionality to the core trading platform.

The two supporting entities covered in this document are:

- **Holiday**: Manages market holidays and non-trading days
- **MarketHistoryJobResult**: Tracks execution results for market data processing jobs

For information about the core copy trading entities, see [Copy Trading Platform] (#copy-trading-platform).

* * *

Overview

Supporting entities serve operational and administrative needs that span multiple business units.

1. **Temporal Business Logic**: The `Holiday` entity enables the system to account for market holidays and non-trading days.
2. **Job Observability**: The `MarketHistoryJobResult` entity provides audit trails and execution details for asynchronous data processing jobs.

These entities are defined using JHipster's entity configuration system and follow the same

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
[.jhipster/Holiday.json1-45] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

* * *

Entity Architecture Diagram

! [Chart 1] (nhsv-admin-images/nhsv-admin-section18-1.svg)

Diagram: Supporting Entities and Their System Integration

This diagram shows how supporting entities integrate with core business entities. The `Holiday` entity is a core business entity, while `MarketHistoryJobResult` is a supporting entity.

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
[.jhipster/Holiday.json1-45] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

* * *

Holiday Entity

Purpose

The `Holiday` entity maintains a calendar of market holidays and non-trading days. This includes:

- Determining valid trading windows
- Scheduling automated copy trading operations
- Calculating business day intervals for performance metrics
- Displaying market availability to users

Entity Definition

The `Holiday` entity is defined in `.jhipster/Holiday.json` and generates a JPA entity mapping.

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
[.jhipster/Holiday.json6] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

Field Structure

Field Name	Type	Constraints	Description
`year`	`Integer`	Required	Calendar year for the holiday
`eventHoliday`	`String`	Required, Max 255 chars	Name or description of the holiday
`startDate`	`ZonedDateTime`	None	Holiday period start timestamp
`endDate`	`ZonedDateTime`	None	Holiday period end timestamp
`createdAt`	`ZonedDateTime`	Required	Record creation timestamp
`updatedAt`	`ZonedDateTime`	None	Last modification timestamp

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
[.jhipster/Holiday.json7-35] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

Entity Structure Diagram

! [Chart 2] (nhsv-admin-images/nhsv-admin-section18-2.svg)

Diagram: Holiday Entity Class Structure

This diagram shows the generated backend components for the `Holiday` entity. The entity fo

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json4] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json4>)

[.jhipster/Holiday.json41-44] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json41-44>)

Configuration Details

JHipster Settings:

Setting	Value	Meaning
dto	mapstruct	Uses MapStruct for entity-DTO mapping
service	serviceImpl	Generates service interface and implementation
pagination	pagination	Enables paginated REST endpoints
jpaMetamodelFiltering	false	Disables JPA Metamodel filtering
incrementalChangelog	true	Uses incremental Liquibase changelogs

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json4-44] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json4-44>)

Time Range Representation

The `Holiday` entity supports flexible holiday duration modeling through `startDate` and `e

- ****Single-day holidays**:** `startDate` set to beginning of day, `endDate` set to end of day
- ****Multi-day closures**:** `startDate` and `endDate` define a continuous non-trading period
- ****Null dates**:** Fields are optional, allowing for year-only holiday records

The use of `ZonedDateTime` ensures proper timezone handling across different market regions

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json20-25] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json20-25>)

* * *

MarketHistoryJobResult Entity

Purpose

The `MarketHistoryJobResult` entity tracks the execution results of background jobs that p

- Audit trail for data processing operations
- Error tracking and debugging information
- Job performance monitoring (start/end times)
- User accountability for job execution

Entity Definition

The `MarketHistoryJobResult` entity is defined in `.jhipster/MarketHistoryJobResult.json` &

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

[.jhipster/MarketHistoryJobResult.json1-46] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>)

Field Structure

Field Name	Type	Constraints	Description
`isSuccess`	`Boolean`	None	Whether the job completed successfully
`timeStart`	`ZonedDateTime`	None	Job execution start timestamp
`timeEnd`	`ZonedDateTime`	None	Job execution end timestamp
`error`	`String`	None	Error message if job failed
`eventId`	`String`	None	Identifier for the specific job event
`symbols`	`String`	None	Market symbols processed by the job

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Relationship Structure

The entity has a ****many-to-one**** relationship with the ``User`` entity, tracking which user triggered the job.

Relationship Property	Value
Relationship Type	<code>`many-to-one`</code>
Related Entity	<code>`user`</code>
Owner Side	<code>`true`</code>
Display Field	<code>`login`</code>

This relationship enables:

- Tracking job execution by user
- Filtering jobs by user login
- Auditing who triggered specific data processing operations

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Entity Relationship Diagram

! [Chart 3] (nhsv-admin-images/nhsv-admin-section18-3.svg)

****Diagram: MarketHistoryJobResult Entity Relationships****

This ER diagram shows the relationship between ``User`` and ``MarketHistoryJobResult``. Each job is associated with exactly one user.

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Configuration Details

****JHipster Settings**:**

Setting	Value	Meaning
<code>`dto`</code>	Not specified	Uses entity directly (no DTO generation)
<code>`service`</code>	<code>`no`</code>	No service layer generation; direct repository access
<code>`pagination`</code>	<code>`no`</code>	No pagination on REST endpoints
<code>`incrementalChangelog`</code>	<code>`true`</code>	Uses incremental Liquibase changelogs

The absence of DTO and service layer generation suggests this entity is primarily used for data processing or reporting.

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>]

Job Execution Tracking

![Chart 4] (nhsv-admin-images/nhsv-admin-section18-4.svg)

****Diagram: Market History Job Execution Flow****

This sequence diagram illustrates how the `MarketHistoryJobResult` entity tracks job lifecycle.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>)

Symbols Field Usage

The `symbols` field stores market symbols processed during the job, likely as a comma-separated list.

- Understanding job scope (which symbols were included)
- Filtering failed jobs by specific symbols
- Debugging symbol-specific processing issues

****Example storage formats**:**

- Comma-separated: `"AAPL,GOOGL,MSFT"`
- JSON array: `["AAPL","GOOGL","MSFT"]`

The actual format depends on the implementation of the job processing logic.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json27-28>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json27-28>)

* * *

System Integration Pattern

![Chart 5] (nhsv-admin-images/nhsv-admin-section18-5.svg)

****Diagram: Supporting Entities in System Context****

This diagram shows how supporting entities integrate with external dependencies and business logic.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>)

[.jhipster/MarketHistoryJobResult.json1-46] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>)

* * *

Comparison Table

Aspect	Holiday	MarketHistoryJobResult
Primary Purpose	Define non-trading days	Track background job execution
Data Source	Manual entry or calendar API	Generated by job scheduler
Update Frequency	Annually (typically)	Per job execution
Core Fields	`year`, `eventHoliday`, date range	`isSuccess`, `error`, time range
Relationships	None	Many-to-one with User
DTO Generation	Yes (MapStruct)	No
Service Layer	Yes (serviceImpl)	No
Pagination	Yes	No
Primary Consumers	Trading schedulers, availability checks	System administrators, monitoring tools

```

**Sources**:[] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js
[.jhipster/Holiday.json4-44] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhip
.jhipster/MarketHistoryJobResult.json31-45] (https://github.com/nhsv-tradex/nhsv-admin/blob,

* * *

#### Database Schema

##### Holiday Table Structure

```

-- t_holiday table

```

CREATE TABLE t_holiday (
  id BIGSERIAL PRIMARY KEY,
  year INTEGER NOT NULL,
  event_holiday VARCHAR(255) NOT NULL,
  start_date TIMESTAMP WITH TIME ZONE,
  end_date TIMESTAMP WITH TIME ZONE,
  created_at TIMESTAMP WITH TIME ZONE NOT NULL,
  updated_at TIMESTAMP WITH TIME ZONE
);

```

```

##### MarketHistoryJobResult Table Structure

```

-- market_history_job_result table

```

CREATE TABLE market_history_job_result (
  id BIGSERIAL PRIMARY KEY,
  is_success BOOLEAN,
  time_start TIMESTAMP WITH TIME ZONE,
  time_end TIMESTAMP WITH TIME ZONE,
  error TEXT,
  event_id VARCHAR(255),
  symbols TEXT,
  user_id BIGINT,
  FOREIGN KEY (user_id) REFERENCES jhi_user(id)
);

```

These schemas are managed by Liquibase changelogs generated from the JHipster entity definition.

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

* * *

Summary

Supporting entities provide essential infrastructure services that enable core business operations.

- **Holiday**: Enables time-aware business logic by defining non-trading periods, critical for accurate market data processing.
- **MarketHistoryJobResult**: Provides observability into asynchronous data processing, ensuring transparency and reliability of market data.

Both entities leverage JHipster's code generation to provide consistent REST APIs, database schemas, and frontend components.

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

Portfolio History Tracking

Relevant source files

- [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)
- [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

Purpose and Scope

This document describes supporting entities that provide ancillary functionality to the core trading platform.

The two supporting entities covered in this document are:

- **Holiday**: Manages market holidays and non-trading days
- **MarketHistoryJobResult**: Tracks execution results for market data processing jobs

For information about the core copy trading entities, see [Copy Trading Platform] (#copy-trading-platform).

* * *

Overview

Supporting entities serve operational and administrative needs that span multiple business units.

1. **Temporal Business Logic**: The `Holiday` entity enables the system to account for market holidays and non-trading days.
2. **Job Observability**: The `MarketHistoryJobResult` entity provides audit trails and execution details for market data processing jobs.

These entities are defined using JHipster's entity configuration system and follow the same

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
[.jhipster/Holiday.json1-45] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

* * *

Entity Architecture Diagram

! [Chart 1] (nhsv-admin-images/nhsv-admin-section19-1.svg)

Diagram: Supporting Entities and Their System Integration

This diagram shows how supporting entities integrate with core business entities. The `Holiday` entity is a core business entity that integrates with the `MarketHistoryJobResult` entity.

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
[.jhipster/Holiday.json1-45] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

* * *

Holiday Entity

Purpose

The `Holiday` entity maintains a calendar of market holidays and non-trading days. This includes:

- Determining valid trading windows
- Scheduling automated copy trading operations
- Calculating business day intervals for performance metrics
- Displaying market availability to users

Entity Definition

The `Holiday` entity is defined in `.jhipster/Holiday.json` and generates a JPA entity mapping.

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
[.jhipster/Holiday.json6] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

Field Structure

Field Name	Type	Constraints	Description
`year`	`Integer`	Required	Calendar year for the holiday
`eventHoliday`	`String`	Required, Max 255 chars	Name or description of the holiday
`startDate`	`ZonedDateTime`	None	Holiday period start timestamp
`endDate`	`ZonedDateTime`	None	Holiday period end timestamp
`createdAt`	`ZonedDateTime`	Required	Record creation timestamp
`updatedAt`	`ZonedDateTime`	None	Last modification timestamp

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
[.jhipster/Holiday.json7-35] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

Entity Structure Diagram

! [Chart 2] (nhsv-admin-images/nhsv-admin-section19-2.svg)

Diagram: Holiday Entity Class Structure

This diagram shows the generated backend components for the `Holiday` entity. The entity fo

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json4] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json4>)

[.jhipster/Holiday.json41-44] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json41-44>)

Configuration Details

JHipster Settings:

Setting	Value	Meaning
dto	mapstruct	Uses MapStruct for entity-DTO mapping
service	serviceImpl	Generates service interface and implementation
pagination	pagination	Enables paginated REST endpoints
jpaMetamodelFiltering	false	Disables JPA Metamodel filtering
incrementalChangelog	true	Uses incremental Liquibase changelogs

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json4-44] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json4-44>)

Time Range Representation

The `Holiday` entity supports flexible holiday duration modeling through `startDate` and `e

- ****Single-day holidays**:** `startDate` set to beginning of day, `endDate` set to end of day
- ****Multi-day closures**:** `startDate` and `endDate` define a continuous non-trading period
- ****Null dates**:** Fields are optional, allowing for year-only holiday records

The use of `ZonedDateTime` ensures proper timezone handling across different market regions

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json20-25] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json20-25>)

* * *

MarketHistoryJobResult Entity

Purpose

The `MarketHistoryJobResult` entity tracks the execution results of background jobs that p

- Audit trail for data processing operations
- Error tracking and debugging information
- Job performance monitoring (start/end times)
- User accountability for job execution

Entity Definition

The `MarketHistoryJobResult` entity is defined in `.jhipster/MarketHistoryJobResult.json` &

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

[.jhipster/MarketHistoryJobResult.json1-46] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>)

Field Structure

Field Name	Type	Constraints	Description
`isSuccess`	`Boolean`	None	Whether the job completed successfully
`timeStart`	`ZonedDateTime`	None	Job execution start timestamp
`timeEnd`	`ZonedDateTime`	None	Job execution end timestamp
`error`	`String`	None	Error message if job failed
`eventId`	`String`	None	Identifier for the specific job event
`symbols`	`String`	None	Market symbols processed by the job

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Relationship Structure

The entity has a **many-to-one** relationship with the ``User`` entity, tracking which user triggered the job.

Relationship Property	Value
Relationship Type	<code>`many-to-one`</code>
Related Entity	<code>`user`</code>
Owner Side	<code>`true`</code>
Display Field	<code>`login`</code>

This relationship enables:

- Tracking job execution by user
- Filtering jobs by user login
- Auditing who triggered specific data processing operations

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Entity Relationship Diagram

! [Chart 3] (nhsv-admin-images/nhsv-admin-section19-3.svg)

****Diagram: MarketHistoryJobResult Entity Relationships****

This ER diagram shows the relationship between ``User`` and ``MarketHistoryJobResult``. Each job is associated with exactly one user.

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Configuration Details

****JHipster Settings**:**

Setting	Value	Meaning
<code>`dto`</code>	Not specified	Uses entity directly (no DTO generation)
<code>`service`</code>	<code>`no`</code>	No service layer generation; direct repository access
<code>`pagination`</code>	<code>`no`</code>	No pagination on REST endpoints
<code>`incrementalChangelog`</code>	<code>`true`</code>	Uses incremental Liquibase changelogs

The absence of DTO and service layer generation suggests this entity is primarily used for data processing or reporting.

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>]

Job Execution Tracking

![Chart 4] (nhsv-admin-images/nhsv-admin-section19-4.svg)

****Diagram: Market History Job Execution Flow****

This sequence diagram illustrates how the `MarketHistoryJobResult` entity tracks job lifecycle.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>)

Symbols Field Usage

The `symbols` field stores market symbols processed during the job, likely as a comma-separated list.

- Understanding job scope (which symbols were included)
- Filtering failed jobs by specific symbols
- Debugging symbol-specific processing issues

****Example storage formats**:**

- Comma-separated: `"AAPL,GOOGL,MSFT"`
- JSON array: `["AAPL","GOOGL","MSFT"]`

The actual format depends on the implementation of the job processing logic.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json27-28>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json27-28>)

* * *

System Integration Pattern

![Chart 5] (nhsv-admin-images/nhsv-admin-section19-5.svg)

****Diagram: Supporting Entities in System Context****

This diagram shows how supporting entities integrate with external dependencies and business logic.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>)

* * *

Comparison Table

Aspect	Holiday	MarketHistoryJobResult
Primary Purpose	Define non-trading days	Track background job execution
Data Source	Manual entry or calendar API	Generated by job scheduler
Update Frequency	Annually (typically)	Per job execution
Core Fields	`year`, `eventHoliday`, date range	`isSuccess`, `error`, time range
Relationships	None	Many-to-one with User
DTO Generation	Yes (MapStruct)	No
Service Layer	Yes (serviceImpl)	No
Pagination	Yes	No
Primary Consumers	Trading schedulers, availability checks	System administrators, monitoring tools


```

**Sources**:[] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js
[.jhipster/Holiday.json4-44] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhip
.jhipster/MarketHistoryJobResult.json31-45] (https://github.com/nhsv-tradex/nhsv-admin/blob,

* * *

#### Database Schema

##### Holiday Table Structure

```

-- t_holiday table

```

CREATE TABLE t_holiday (
  id BIGSERIAL PRIMARY KEY,
  year INTEGER NOT NULL,
  event_holiday VARCHAR(255) NOT NULL,
  start_date TIMESTAMP WITH TIME ZONE,
  end_date TIMESTAMP WITH TIME ZONE,
  created_at TIMESTAMP WITH TIME ZONE NOT NULL,
  updated_at TIMESTAMP WITH TIME ZONE
);

```

```

##### MarketHistoryJobResult Table Structure

```

-- market_history_job_result table

```

CREATE TABLE market_history_job_result (
  id BIGSERIAL PRIMARY KEY,
  is_success BOOLEAN,
  time_start TIMESTAMP WITH TIME ZONE,
  time_end TIMESTAMP WITH TIME ZONE,
  error TEXT,
  event_id VARCHAR(255),
  symbols TEXT,
  user_id BIGINT,
  FOREIGN KEY (user_id) REFERENCES jhi_user(id)
);

```

These schemas are managed by Liquibase changelogs generated from the JHipster entity definition.

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

* * *

Summary

Supporting entities provide essential infrastructure services that enable core business operations.

- **Holiday**: Enables time-aware business logic by defining non-trading periods, critical for accurate market data processing.
- **MarketHistoryJobResult**: Provides observability into asynchronous data processing, ensuring transparency and reliability of market data.

Both entities leverage JHipster's code generation to provide consistent REST APIs, database schemas, and frontend components.

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

Subscriber Management

Relevant source files

- [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)
- [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

Purpose and Scope

This document describes supporting entities that provide ancillary functionality to the core trading platform.

The two supporting entities covered in this document are:

- **Holiday**: Manages market holidays and non-trading days
- **MarketHistoryJobResult**: Tracks execution results for market data processing jobs

For information about the core copy trading entities, see [Copy Trading Platform] (#copy-trading).

* * *

Overview

Supporting entities serve operational and administrative needs that span multiple business units.

1. **Temporal Business Logic**: The `Holiday` entity enables the system to account for market holidays and non-trading days.
2. **Job Observability**: The `MarketHistoryJobResult` entity provides audit trails and execution details for market data processing jobs.

These entities are defined using JHipster's entity configuration system and follow the same

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
[.jhipster/Holiday.json1-45] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

* * *

Entity Architecture Diagram

! [Chart 1] (nhsv-admin-images/nhsv-admin-section20-1.svg)

Diagram: Supporting Entities and Their System Integration

This diagram shows how supporting entities integrate with core business entities. The `Holiday` entity is a core business entity that integrates with the `MarketHistoryJobResult` entity.

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
[.jhipster/Holiday.json1-45] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

* * *

Holiday Entity

Purpose

The `Holiday` entity maintains a calendar of market holidays and non-trading days. This includes:

- Determining valid trading windows
- Scheduling automated copy trading operations
- Calculating business day intervals for performance metrics
- Displaying market availability to users

Entity Definition

The `Holiday` entity is defined in `.jhipster/Holiday.json` and generates a JPA entity map.

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
[.jhipster/Holiday.json6] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

Field Structure

Field Name	Type	Constraints	Description
`year`	`Integer`	Required	Calendar year for the holiday
`eventHoliday`	`String`	Required, Max 255 chars	Name or description of the holiday
`startDate`	`ZonedDateTime`	None	Holiday period start timestamp
`endDate`	`ZonedDateTime`	None	Holiday period end timestamp
`createdAt`	`ZonedDateTime`	Required	Record creation timestamp
`updatedAt`	`ZonedDateTime`	None	Last modification timestamp

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
[.jhipster/Holiday.json7-35] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

Entity Structure Diagram

! [Chart 2] (nhsv-admin-images/nhsv-admin-section20-2.svg)

Diagram: Holiday Entity Class Structure

This diagram shows the generated backend components for the `Holiday` entity. The entity fo

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json4] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json4>)

[.jhipster/Holiday.json41-44] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json41-44>)

Configuration Details

JHipster Settings:

Setting	Value	Meaning
dto	mapstruct	Uses MapStruct for entity-DTO mapping
service	serviceImpl	Generates service interface and implementation
pagination	pagination	Enables paginated REST endpoints
jpaMetamodelFiltering	false	Disables JPA Metamodel filtering
incrementalChangelog	true	Uses incremental Liquibase changelogs

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json4-44] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json4-44>)

Time Range Representation

The `Holiday` entity supports flexible holiday duration modeling through `startDate` and `e

- ****Single-day holidays**:** `startDate` set to beginning of day, `endDate` set to end of day
- ****Multi-day closures**:** `startDate` and `endDate` define a continuous non-trading period
- ****Null dates**:** Fields are optional, allowing for year-only holiday records

The use of `ZonedDateTime` ensures proper timezone handling across different market regions

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json20-25] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json20-25>)

* * *

MarketHistoryJobResult Entity

Purpose

The `MarketHistoryJobResult` entity tracks the execution results of background jobs that p

- Audit trail for data processing operations
- Error tracking and debugging information
- Job performance monitoring (start/end times)
- User accountability for job execution

Entity Definition

The `MarketHistoryJobResult` entity is defined in `.jhipster/MarketHistoryJobResult.json` &

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

[.jhipster/MarketHistoryJobResult.json1-46] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>)

Field Structure

Field Name	Type	Constraints	Description
`isSuccess`	`Boolean`	None	Whether the job completed successfully
`timeStart`	`ZonedDateTime`	None	Job execution start timestamp
`timeEnd`	`ZonedDateTime`	None	Job execution end timestamp
`error`	`String`	None	Error message if job failed
`eventId`	`String`	None	Identifier for the specific job event
`symbols`	`String`	None	Market symbols processed by the job

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Relationship Structure

The entity has a ****many-to-one**** relationship with the ``User`` entity, tracking which user triggered the job.

Relationship Property	Value
Relationship Type	<code>`many-to-one`</code>
Related Entity	<code>`user`</code>
Owner Side	<code>`true`</code>
Display Field	<code>`login`</code>

This relationship enables:

- Tracking job execution by user
- Filtering jobs by user login
- Auditing who triggered specific data processing operations

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Entity Relationship Diagram

! [Chart 3] (nhsv-admin-images/nhsv-admin-section20-3.svg)

****Diagram: MarketHistoryJobResult Entity Relationships****

This ER diagram shows the relationship between ``User`` and ``MarketHistoryJobResult``. Each job is associated with exactly one user.

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Configuration Details

****JHipster Settings**:**

Setting	Value	Meaning
<code>`dto`</code>	Not specified	Uses entity directly (no DTO generation)
<code>`service`</code>	<code>`no`</code>	No service layer generation; direct repository access
<code>`pagination`</code>	<code>`no`</code>	No pagination on REST endpoints
<code>`incrementalChangelog`</code>	<code>`true`</code>	Uses incremental Liquibase changelogs

The absence of DTO and service layer generation suggests this entity is primarily used for data processing or reporting.

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>]

Job Execution Tracking

![Chart 4] (nhsv-admin-images/nhsv-admin-section20-4.svg)

****Diagram: Market History Job Execution Flow****

This sequence diagram illustrates how the `MarketHistoryJobResult` entity tracks job lifecycle.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>)

[.jhipster/MarketHistoryJobResult.json3-29] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>)

Symbols Field Usage

The `symbols` field stores market symbols processed during the job, likely as a comma-separated list.

- Understanding job scope (which symbols were included)
- Filtering failed jobs by specific symbols
- Debugging symbol-specific processing issues

****Example storage formats**:**

- Comma-separated: `"AAPL,GOOGL,MSFT"`
- JSON array: `["AAPL","GOOGL","MSFT"]`

The actual format depends on the implementation of the job processing logic.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json27-28>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json27-28>)

[.jhipster/MarketHistoryJobResult.json27-28] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json27-28>)

* * *

System Integration Pattern

![Chart 5] (nhsv-admin-images/nhsv-admin-section20-5.svg)

****Diagram: Supporting Entities in System Context****

This diagram shows how supporting entities integrate with external dependencies and business logic.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>)

[.jhipster/Holiday.json1-45] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>)

[.jhipster/MarketHistoryJobResult.json1-46] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>)

* * *

Comparison Table

Aspect	Holiday	MarketHistoryJobResult
Primary Purpose	Define non-trading days	Track background job execution
Data Source	Manual entry or calendar API	Generated by job scheduler
Update Frequency	Annually (typically)	Per job execution
Core Fields	`year`, `eventHoliday`, date range	`isSuccess`, `error`, time range
Relationships	None	Many-to-one with User
DTO Generation	Yes (MapStruct)	No
Service Layer	Yes (serviceImpl)	No
Pagination	Yes	No
Primary Consumers	Trading schedulers, availability checks	System administrators, monitoring tools

```

**Sources**:[] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js
[.jhipster/Holiday.json4-44] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhip
.jhipster/MarketHistoryJobResult.json31-45] (https://github.com/nhsv-tradex/nhsv-admin/blob,

* * *

#### Database Schema

##### Holiday Table Structure

```

-- t_holiday table

```

CREATE TABLE t_holiday (
  id BIGSERIAL PRIMARY KEY,
  year INTEGER NOT NULL,
  event_holiday VARCHAR(255) NOT NULL,
  start_date TIMESTAMP WITH TIME ZONE,
  end_date TIMESTAMP WITH TIME ZONE,
  created_at TIMESTAMP WITH TIME ZONE NOT NULL,
  updated_at TIMESTAMP WITH TIME ZONE
);

```

```

##### MarketHistoryJobResult Table Structure

```

-- market_history_job_result table

```

CREATE TABLE market_history_job_result (
  id BIGSERIAL PRIMARY KEY,
  is_success BOOLEAN,
  time_start TIMESTAMP WITH TIME ZONE,
  time_end TIMESTAMP WITH TIME ZONE,
  error TEXT,
  event_id VARCHAR(255),
  symbols TEXT,
  user_id BIGINT,
  FOREIGN KEY (user_id) REFERENCES jhi_user(id)
);

```

These schemas are managed by Liquibase changelogs generated from the JHipster entity definition.

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

* * *

Summary

Supporting entities provide essential infrastructure services that enable core business operations.

- **Holiday**: Enables time-aware business logic by defining non-trading periods, critical for accurate market data processing.
- **MarketHistoryJobResult**: Provides observability into asynchronous data processing, ensuring transparency and reliability of market data.

Both entities leverage JHipster's code generation to provide consistent REST APIs, database schemas, and frontend components.

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

Subscriber Registration

Relevant source files

- [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)
- [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

Purpose and Scope

This document describes supporting entities that provide ancillary functionality to the core system.

The two supporting entities covered in this document are:

- **Holiday**: Manages market holidays and non-trading days
- **MarketHistoryJobResult**: Tracks execution results for market data processing jobs

For information about the core copy trading entities, see [Copy Trading Platform] (#copy-trading).

* * *

Overview

Supporting entities serve operational and administrative needs that span multiple business units.

1. **Temporal Business Logic**: The `Holiday` entity enables the system to account for market holidays and non-trading days.
2. **Job Observability**: The `MarketHistoryJobResult` entity provides audit trails and execution details for market data processing jobs.

These entities are defined using JHipster's entity configuration system and follow the same

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>
<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json#L45>
<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json#L46>
<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json#L46>

* * *

Entity Architecture Diagram

![[Chart 1] (nhsv-admin-images/nhsv-admin-section21-1.svg)]

Diagram: Supporting Entities and Their System Integration

This diagram shows how supporting entities integrate with core business entities. The `Holiday` entity is a supporting entity that integrates with the core business entities.

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>
<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json#L45>
<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json#L35-43>
<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json#L46>

* * *

Holiday Entity

Purpose

The `Holiday` entity maintains a calendar of market holidays and non-trading days. This includes:

- Determining valid trading windows
- Scheduling automated copy trading operations
- Calculating business day intervals for performance metrics
- Displaying market availability to users

Entity Definition

The `Holiday` entity is defined in `.jhipster/Holiday.json` and generates a JPA entity mapping.

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>
<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json#L6>
<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json#L35-43>
<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json#L46>

Field Structure

Field Name	Type	Constraints	Description
`year`	`Integer`	Required	Calendar year for the holiday
`eventHoliday`	`String`	Required, Max 255 chars	Name or description of the holiday
`startDate`	`ZonedDateTime`	None	Holiday period start timestamp
`endDate`	`ZonedDateTime`	None	Holiday period end timestamp
`createdAt`	`ZonedDateTime`	Required	Record creation timestamp
`updatedAt`	`ZonedDateTime`	None	Last modification timestamp

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>
<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json#L7-35>
<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json#L35-43>
<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json#L46>

Entity Structure Diagram

![[Chart 2] (nhsv-admin-images/nhsv-admin-section21-2.svg)]

Diagram: Holiday Entity Class Structure

This diagram shows the generated backend components for the `Holiday` entity. The entity fo

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json4] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json4>)

[.jhipster/Holiday.json41-44] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json41-44>)

Configuration Details

JHipster Settings:

Setting	Value	Meaning
dto	mapstruct	Uses MapStruct for entity-DTO mapping
service	serviceImpl	Generates service interface and implementation
pagination	pagination	Enables paginated REST endpoints
jpaMetamodelFiltering	false	Disables JPA Metamodel filtering
incrementalChangelog	true	Uses incremental Liquibase changelogs

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json4-44] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json4-44>)

Time Range Representation

The `Holiday` entity supports flexible holiday duration modeling through `startDate` and `e

- ****Single-day holidays**:** `startDate` set to beginning of day, `endDate` set to end of day
- ****Multi-day closures**:** `startDate` and `endDate` define a continuous non-trading period
- ****Null dates**:** Fields are optional, allowing for year-only holiday records

The use of `ZonedDateTime` ensures proper timezone handling across different market regions

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json20-25] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json20-25>)

* * *

MarketHistoryJobResult Entity

Purpose

The `MarketHistoryJobResult` entity tracks the execution results of background jobs that p

- Audit trail for data processing operations
- Error tracking and debugging information
- Job performance monitoring (start/end times)
- User accountability for job execution

Entity Definition

The `MarketHistoryJobResult` entity is defined in `.jhipster/MarketHistoryJobResult.json` &

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

[.jhipster/MarketHistoryJobResult.json1-46] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>)

Field Structure

Field Name	Type	Constraints	Description
`isSuccess`	`Boolean`	None	Whether the job completed successfully
`timeStart`	`ZonedDateTime`	None	Job execution start timestamp
`timeEnd`	`ZonedDateTime`	None	Job execution end timestamp
`error`	`String`	None	Error message if job failed
`eventId`	`String`	None	Identifier for the specific job event
`symbols`	`String`	None	Market symbols processed by the job

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Relationship Structure

The entity has a **many-to-one** relationship with the ``User`` entity, tracking which user triggered the job.

Relationship Property	Value
Relationship Type	<code>`many-to-one`</code>
Related Entity	<code>`user`</code>
Owner Side	<code>`true`</code>
Display Field	<code>`login`</code>

This relationship enables:

- Tracking job execution by user
- Filtering jobs by user login
- Auditing who triggered specific data processing operations

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Entity Relationship Diagram

! [Chart 3] (nhsv-admin-images/nhsv-admin-section21-3.svg)

****Diagram: MarketHistoryJobResult Entity Relationships****

This ER diagram shows the relationship between ``User`` and ``MarketHistoryJobResult``. Each job is associated with exactly one user.

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Configuration Details

****JHipster Settings**:**

Setting	Value	Meaning
<code>`dto`</code>	Not specified	Uses entity directly (no DTO generation)
<code>`service`</code>	<code>`no`</code>	No service layer generation; direct repository access
<code>`pagination`</code>	<code>`no`</code>	No pagination on REST endpoints
<code>`incrementalChangelog`</code>	<code>`true`</code>	Uses incremental Liquibase changelogs

The absence of DTO and service layer generation suggests this entity is primarily used for data processing or reporting.

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>]

Job Execution Tracking

![Chart 4] (nhsv-admin-images/nhsv-admin-section21-4.svg)

****Diagram: Market History Job Execution Flow****

This sequence diagram illustrates how the `MarketHistoryJobResult` entity tracks job lifecycle.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>)

Symbols Field Usage

The `symbols` field stores market symbols processed during the job, likely as a comma-separated list.

- Understanding job scope (which symbols were included)
- Filtering failed jobs by specific symbols
- Debugging symbol-specific processing issues

****Example storage formats**:**

- Comma-separated: `"AAPL,GOOGL,MSFT"`
- JSON array: `["AAPL","GOOGL","MSFT"]`

The actual format depends on the implementation of the job processing logic.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json27-28>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json27-28>)

* * *

System Integration Pattern

![Chart 5] (nhsv-admin-images/nhsv-admin-section21-5.svg)

****Diagram: Supporting Entities in System Context****

This diagram shows how supporting entities integrate with external dependencies and business logic.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>)

* * *

Comparison Table

Aspect	Holiday	MarketHistoryJobResult
Primary Purpose	Define non-trading days	Track background job execution
Data Source	Manual entry or calendar API	Generated by job scheduler
Update Frequency	Annually (typically)	Per job execution
Core Fields	`year`, `eventHoliday`, date range	`isSuccess`, `error`, time range
Relationships	None	Many-to-one with User
DTO Generation	Yes (MapStruct)	No
Service Layer	Yes (serviceImpl)	No
Pagination	Yes	No
Primary Consumers	Trading schedulers, availability checks	System administrators, monitoring tools

```

**Sources**:[] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js
[.jhipster/Holiday.json4-44] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhip
.jhipster/MarketHistoryJobResult.json31-45] (https://github.com/nhsv-tradex/nhsv-admin/blob,

* * *

#### Database Schema

##### Holiday Table Structure

```

-- t_holiday table

```

CREATE TABLE t_holiday (
  id BIGSERIAL PRIMARY KEY,
  year INTEGER NOT NULL,
  event_holiday VARCHAR(255) NOT NULL,
  start_date TIMESTAMP WITH TIME ZONE,
  end_date TIMESTAMP WITH TIME ZONE,
  created_at TIMESTAMP WITH TIME ZONE NOT NULL,
  updated_at TIMESTAMP WITH TIME ZONE
);

```

```

##### MarketHistoryJobResult Table Structure

```

-- market_history_job_result table

```

CREATE TABLE market_history_job_result (
  id BIGSERIAL PRIMARY KEY,
  is_success BOOLEAN,
  time_start TIMESTAMP WITH TIME ZONE,
  time_end TIMESTAMP WITH TIME ZONE,
  error TEXT,
  event_id VARCHAR(255),
  symbols TEXT,
  user_id BIGINT,
  FOREIGN KEY (user_id) REFERENCES jhi_user(id)
);

```

```

These schemas are managed by Liquibase changelogs generated from the JHipster entity definition.

**Sources**: [] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json)
[.jhipster/Holiday.json7-35] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-43] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json4-43] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json5-43]

* * *

#### Summary

Supporting entities provide essential infrastructure services that enable core business operations.

- **Holiday**: Enables time-aware business logic by defining non-trading periods, critical for accurate market data processing.
- **MarketHistoryJobResult**: Provides observability into asynchronous data processing, ensuring transparency and auditability.

Both entities leverage JHipster's code generation to provide consistent REST APIs, database schemas, and frontend components.

**Sources**: [] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json)
[.jhipster/Holiday.json1-45] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json2-46] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-46]

---

### Subscriber Details and History

Relevant source files

- [.jhipster/Holiday.json] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json)
- [.jhipster/MarketHistoryJobResult.json] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json)

#### Purpose and Scope

This document describes supporting entities that provide ancillary functionality to the core trading platform.

The two supporting entities covered in this document are:

- **Holiday**: Manages market holidays and non-trading days
- **MarketHistoryJobResult**: Tracks execution results for market data processing jobs

For information about the core copy trading entities, see [Copy Trading Platform] (#copy-trading-platform).

* * *

#### Overview

Supporting entities serve operational and administrative needs that span multiple business units.

1. **Temporal Business Logic**: The `Holiday` entity enables the system to account for market closures and non-trading periods.
2. **Job Observability**: The `MarketHistoryJobResult` entity provides audit trails and performance metrics for data processing jobs.

```

These entities are defined using JHipster's entity configuration system and follow the same

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
 [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-46>)

* * *

Entity Architecture Diagram

! [Chart 1] (nhsv-admin-images/nhsv-admin-section22-1.svg)

Diagram: Supporting Entities and Their System Integration

This diagram shows how supporting entities integrate with core business entities. The `Holiday` entity is a supporting entity that integrates with the `MarketHistoryJobResult` entity.

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
 [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-46>)

* * *

Holiday Entity

Purpose

The `Holiday` entity maintains a calendar of market holidays and non-trading days. This includes:

- Determining valid trading windows
- Scheduling automated copy trading operations
- Calculating business day intervals for performance metrics
- Displaying market availability to users

Entity Definition

The `Holiday` entity is defined in `.jhipster/Holiday.json` and generates a JPA entity mapping.

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
 [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json6>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-46>)

Field Structure

Field Name	Type	Constraints	Description
`year`	`Integer`	Required	Calendar year for the holiday
`eventHoliday`	`String`	Required, Max 255 chars	Name or description of the holiday
`startDate`	`ZonedDateTime`	None	Holiday period start timestamp
`endDate`	`ZonedDateTime`	None	Holiday period end timestamp
`createdAt`	`ZonedDateTime`	Required	Record creation timestamp
`updatedAt`	`ZonedDateTime`	None	Last modification timestamp

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
 [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json7-35>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-46>)

Entity Structure Diagram

! [Chart 2] (nhsv-admin-images/nhsv-admin-section22-2.svg)

Diagram: Holiday Entity Class Structure

This diagram shows the generated backend components for the `Holiday` entity. The entity fo

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json4] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json4>)

[.jhipster/Holiday.json41-44] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json41-44>)

Configuration Details

JHipster Settings:

Setting	Value	Meaning
dto	mapstruct	Uses MapStruct for entity-DTO mapping
service	serviceImpl	Generates service interface and implementation
pagination	pagination	Enables paginated REST endpoints
jpaMetamodelFiltering	false	Disables JPA Metamodel filtering
incrementalChangelog	true	Uses incremental Liquibase changelogs

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json4-44] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json4-44>)

Time Range Representation

The `Holiday` entity supports flexible holiday duration modeling through `startDate` and `e

- ****Single-day holidays**:** `startDate` set to beginning of day, `endDate` set to end of day
- ****Multi-day closures**:** `startDate` and `endDate` define a continuous non-trading period
- ****Null dates**:** Fields are optional, allowing for year-only holiday records

The use of `ZonedDateTime` ensures proper timezone handling across different market regions

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json20-25] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json20-25>)

* * *

MarketHistoryJobResult Entity

Purpose

The `MarketHistoryJobResult` entity tracks the execution results of background jobs that p

- Audit trail for data processing operations
- Error tracking and debugging information
- Job performance monitoring (start/end times)
- User accountability for job execution

Entity Definition

The `MarketHistoryJobResult` entity is defined in `.jhipster/MarketHistoryJobResult.json` &

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

[.jhipster/MarketHistoryJobResult.json1-46] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>)

Field Structure

Field Name	Type	Constraints	Description
`isSuccess`	`Boolean`	None	Whether the job completed successfully
`timeStart`	`ZonedDateTime`	None	Job execution start timestamp
`timeEnd`	`ZonedDateTime`	None	Job execution end timestamp
`error`	`String`	None	Error message if job failed
`eventId`	`String`	None	Identifier for the specific job event
`symbols`	`String`	None	Market symbols processed by the job

****Sources**:** [(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29)]

Relationship Structure

The entity has a **many-to-one** relationship with the ``User`` entity, tracking which user triggered the job.

Relationship Property	Value
Relationship Type	<code>`many-to-one`</code>
Related Entity	<code>`user`</code>
Owner Side	<code>`true`</code>
Display Field	<code>`login`</code>

This relationship enables:

- Tracking job execution by user
- Filtering jobs by user login
- Auditing who triggered specific data processing operations

****Sources**:** [(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43)]

Entity Relationship Diagram

! [Chart 3] (nhsv-admin-images/nhsv-admin-section22-3.svg)

****Diagram: MarketHistoryJobResult Entity Relationships****

This ER diagram shows the relationship between ``User`` and ``MarketHistoryJobResult``. Each job is associated with exactly one user.

****Sources**:** [(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43)]

Configuration Details

****JHipster Settings**:**

Setting	Value	Meaning
<code>`dto`</code>	Not specified	Uses entity directly (no DTO generation)
<code>`service`</code>	<code>`no`</code>	No service layer generation; direct repository access
<code>`pagination`</code>	<code>`no`</code>	No pagination on REST endpoints
<code>`incrementalChangelog`</code>	<code>`true`</code>	Uses incremental Liquibase changelogs

The absence of DTO and service layer generation suggests this entity is primarily used for data processing or reporting.

****Sources**:** [(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45)]

Job Execution Tracking

![Chart 4] (nhsv-admin-images/nhsv-admin-section22-4.svg)

****Diagram: Market History Job Execution Flow****

This sequence diagram illustrates how the `MarketHistoryJobResult` entity tracks job lifecycle.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>)

Symbols Field Usage

The `symbols` field stores market symbols processed during the job, likely as a comma-separated list.

- Understanding job scope (which symbols were included)
- Filtering failed jobs by specific symbols
- Debugging symbol-specific processing issues

****Example storage formats**:**

- Comma-separated: `"AAPL,GOOGL,MSFT"`
- JSON array: `["AAPL","GOOGL","MSFT"]`

The actual format depends on the implementation of the job processing logic.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json27-28>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json27-28>)

* * *

System Integration Pattern

![Chart 5] (nhsv-admin-images/nhsv-admin-section22-5.svg)

****Diagram: Supporting Entities in System Context****

This diagram shows how supporting entities integrate with external dependencies and business logic.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>)

* * *

Comparison Table

Aspect	Holiday	MarketHistoryJobResult
Primary Purpose	Define non-trading days	Track background job execution
Data Source	Manual entry or calendar API	Generated by job scheduler
Update Frequency	Annually (typically)	Per job execution
Core Fields	`year`, `eventHoliday`, date range	`isSuccess`, `error`, time range
Relationships	None	Many-to-one with User
DTO Generation	Yes (MapStruct)	No
Service Layer	Yes (serviceImpl)	No
Pagination	Yes	No
Primary Consumers	Trading schedulers, availability checks	System administrators, monitoring tools

```

**Sources**:[] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js
[.jhipster/Holiday.json4-44] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhip
.jhipster/MarketHistoryJobResult.json31-45] (https://github.com/nhsv-tradex/nhsv-admin/blob,

* * *

#### Database Schema

##### Holiday Table Structure

```

-- t_holiday table

```

CREATE TABLE t_holiday (
  id BIGSERIAL PRIMARY KEY,
  year INTEGER NOT NULL,
  event_holiday VARCHAR(255) NOT NULL,
  start_date TIMESTAMP WITH TIME ZONE,
  end_date TIMESTAMP WITH TIME ZONE,
  created_at TIMESTAMP WITH TIME ZONE NOT NULL,
  updated_at TIMESTAMP WITH TIME ZONE
);

```

```

##### MarketHistoryJobResult Table Structure

```

-- market_history_job_result table

```

CREATE TABLE market_history_job_result (
  id BIGSERIAL PRIMARY KEY,
  is_success BOOLEAN,
  time_start TIMESTAMP WITH TIME ZONE,
  time_end TIMESTAMP WITH TIME ZONE,
  error TEXT,
  event_id VARCHAR(255),
  symbols TEXT,
  user_id BIGINT,
  FOREIGN KEY (user_id) REFERENCES jhi_user(id)
);

```

```

These schemas are managed by Liquibase changelogs generated from the JHipster entity definition.

**Sources**: [] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json)
[.jhipster/Holiday.json7-35] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-43] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json4-43] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json5-43]

* * *

#### Summary

Supporting entities provide essential infrastructure services that enable core business operations.

- **Holiday**: Enables time-aware business logic by defining non-trading periods, critical for accurate market data processing.
- **MarketHistoryJobResult**: Provides observability into asynchronous data processing, ensuring transparency and auditability.

Both entities leverage JHipster's code generation to provide consistent REST APIs, database schemas, and frontend components.

**Sources**: [] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json)
[.jhipster/Holiday.json1-45] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json2-46] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-46]

---

### Trading Operations

Relevant source files

- [https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json)
- [https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json)

#### Purpose and Scope

This document describes supporting entities that provide ancillary functionality to the core trading platform.

The two supporting entities covered in this document are:

- **Holiday**: Manages market holidays and non-trading days
- **MarketHistoryJobResult**: Tracks execution results for market data processing jobs

For information about the core copy trading entities, see [Copy Trading Platform] (#copy-trading-platform).

* * *

#### Overview

Supporting entities serve operational and administrative needs that span multiple business units.

1. **Temporal Business Logic**: The `Holiday` entity enables the system to account for market holidays and non-trading days.
2. **Job Observability**: The `MarketHistoryJobResult` entity provides audit trails and observability into asynchronous data processing jobs.

```

These entities are defined using JHipster's entity configuration system and follow the same

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>
<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json#L45>
<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json#L46>
<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json#L46>

* * *

Entity Architecture Diagram

![[Chart 1] (nhsv-admin-images/nhsv-admin-section23-1.svg)]

Diagram: Supporting Entities and Their System Integration

This diagram shows how supporting entities integrate with core business entities. The `Holiday` entity is a supporting entity that integrates with the core business entities.

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>
<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json#L45>
<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json#L46>
<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json#L46>

* * *

Holiday Entity

Purpose

The `Holiday` entity maintains a calendar of market holidays and non-trading days. This includes:

- Determining valid trading windows
- Scheduling automated copy trading operations
- Calculating business day intervals for performance metrics
- Displaying market availability to users

Entity Definition

The `Holiday` entity is defined in `.jhipster/Holiday.json` and generates a JPA entity mapping.

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>
<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json#L6>
<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json#L46>

Field Structure

Field Name	Type	Constraints	Description
`year`	`Integer`	Required	Calendar year for the holiday
`eventHoliday`	`String`	Required, Max 255 chars	Name or description of the holiday
`startDate`	`ZonedDateTime`	None	Holiday period start timestamp
`endDate`	`ZonedDateTime`	None	Holiday period end timestamp
`createdAt`	`ZonedDateTime`	Required	Record creation timestamp
`updatedAt`	`ZonedDateTime`	None	Last modification timestamp

Sources: <https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>
<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json#L7-35>
<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json#L46>

Entity Structure Diagram

![[Chart 2] (nhsv-admin-images/nhsv-admin-section23-2.svg)]

Diagram: Holiday Entity Class Structure

This diagram shows the generated backend components for the `Holiday` entity. The entity fo

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json4] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json4>)

[.jhipster/Holiday.json41-44] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json41-44>)

Configuration Details

JHipster Settings:

Setting	Value	Meaning
dto	mapstruct	Uses MapStruct for entity-DTO mapping
service	serviceImpl	Generates service interface and implementation
pagination	pagination	Enables paginated REST endpoints
jpaMetamodelFiltering	false	Disables JPA Metamodel filtering
incrementalChangelog	true	Uses incremental Liquibase changelogs

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json4-44] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json4-44>)

Time Range Representation

The `Holiday` entity supports flexible holiday duration modeling through `startDate` and `e

- ****Single-day holidays**:** `startDate` set to beginning of day, `endDate` set to end of day
- ****Multi-day closures**:** `startDate` and `endDate` define a continuous non-trading period
- ****Null dates**:** Fields are optional, allowing for year-only holiday records

The use of `ZonedDateTime` ensures proper timezone handling across different market regions

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json20-25] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json20-25>)

* * *

MarketHistoryJobResult Entity

Purpose

The `MarketHistoryJobResult` entity tracks the execution results of background jobs that p

- Audit trail for data processing operations
- Error tracking and debugging information
- Job performance monitoring (start/end times)
- User accountability for job execution

Entity Definition

The `MarketHistoryJobResult` entity is defined in `.jhipster/MarketHistoryJobResult.json` &

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

[.jhipster/MarketHistoryJobResult.json1-46] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>)

Field Structure

Field Name	Type	Constraints	Description
`isSuccess`	`Boolean`	None	Whether the job completed successfully
`timeStart`	`ZonedDateTime`	None	Job execution start timestamp
`timeEnd`	`ZonedDateTime`	None	Job execution end timestamp
`error`	`String`	None	Error message if job failed
`eventId`	`String`	None	Identifier for the specific job event
`symbols`	`String`	None	Market symbols processed by the job

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Relationship Structure

The entity has a **many-to-one** relationship with the ``User`` entity, tracking which user triggered the job.

Relationship Property	Value
Relationship Type	<code>`many-to-one`</code>
Related Entity	<code>`user`</code>
Owner Side	<code>`true`</code>
Display Field	<code>`login`</code>

This relationship enables:

- Tracking job execution by user
- Filtering jobs by user login
- Auditing who triggered specific data processing operations

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Entity Relationship Diagram

! [Chart 3] (nhsv-admin-images/nhsv-admin-section23-3.svg)

****Diagram: MarketHistoryJobResult Entity Relationships****

This ER diagram shows the relationship between ``User`` and ``MarketHistoryJobResult``. Each job is associated with exactly one user.

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Configuration Details

****JHipster Settings**:**

Setting	Value	Meaning
<code>`dto`</code>	Not specified	Uses entity directly (no DTO generation)
<code>`service`</code>	<code>`no`</code>	No service layer generation; direct repository access
<code>`pagination`</code>	<code>`no`</code>	No pagination on REST endpoints
<code>`incrementalChangelog`</code>	<code>`true`</code>	Uses incremental Liquibase changelogs

The absence of DTO and service layer generation suggests this entity is primarily used for data processing or reporting.

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>]

Job Execution Tracking

![Chart 4] (nhsv-admin-images/nhsv-admin-section23-4.svg)

****Diagram: Market History Job Execution Flow****

This sequence diagram illustrates how the `MarketHistoryJobResult` entity tracks job lifecycle.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>)

Symbols Field Usage

The `symbols` field stores market symbols processed during the job, likely as a comma-separated list.

- Understanding job scope (which symbols were included)
- Filtering failed jobs by specific symbols
- Debugging symbol-specific processing issues

****Example storage formats**:**

- Comma-separated: `"AAPL,GOOGL,MSFT"`
- JSON array: `["AAPL","GOOGL","MSFT"]`

The actual format depends on the implementation of the job processing logic.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json27-28>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json27-28>)

* * *

System Integration Pattern

![Chart 5] (nhsv-admin-images/nhsv-admin-section23-5.svg)

****Diagram: Supporting Entities in System Context****

This diagram shows how supporting entities integrate with external dependencies and business logic.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>)

* * *

Comparison Table

Aspect	Holiday	MarketHistoryJobResult
Primary Purpose	Define non-trading days	Track background job execution
Data Source	Manual entry or calendar API	Generated by job scheduler
Update Frequency	Annually (typically)	Per job execution
Core Fields	`year`, `eventHoliday`, date range	`isSuccess`, `error`, time range
Relationships	None	Many-to-one with User
DTO Generation	Yes (MapStruct)	No
Service Layer	Yes (serviceImpl)	No
Pagination	Yes	No
Primary Consumers	Trading schedulers, availability checks	System administrators, monitoring tools


```

**Sources**:[] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js
[.jhipster/Holiday.json4-44] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhip
.jhipster/MarketHistoryJobResult.json31-45] (https://github.com/nhsv-tradex/nhsv-admin/blob,

* * *

#### Database Schema

##### Holiday Table Structure

```

-- t_holiday table

```

CREATE TABLE t_holiday (
  id BIGSERIAL PRIMARY KEY,
  year INTEGER NOT NULL,
  event_holiday VARCHAR(255) NOT NULL,
  start_date TIMESTAMP WITH TIME ZONE,
  end_date TIMESTAMP WITH TIME ZONE,
  created_at TIMESTAMP WITH TIME ZONE NOT NULL,
  updated_at TIMESTAMP WITH TIME ZONE
);

```

```

##### MarketHistoryJobResult Table Structure

```

-- market_history_job_result table

```

CREATE TABLE market_history_job_result (
  id BIGSERIAL PRIMARY KEY,
  is_success BOOLEAN,
  time_start TIMESTAMP WITH TIME ZONE,
  time_end TIMESTAMP WITH TIME ZONE,
  error TEXT,
  event_id VARCHAR(255),
  symbols TEXT,
  user_id BIGINT,
  FOREIGN KEY (user_id) REFERENCES jhi_user(id)
);

```

```

These schemas are managed by Liquibase changelogs generated from the JHipster entity definition.

**Sources**: [] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json)
[.jhipster/Holiday.json7-35] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-43] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json4-43] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json5-43]

* * *

#### Summary

Supporting entities provide essential infrastructure services that enable core business operations.

- **Holiday**: Enables time-aware business logic by defining non-trading periods, critical for accurate market data processing.
- **MarketHistoryJobResult**: Provides observability into asynchronous data processing, ensuring transparency and auditability.

Both entities leverage JHipster's code generation to provide consistent REST APIs, database schemas, and frontend components.

**Sources**: [] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json)
[.jhipster/Holiday.json1-45] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json2-46] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-46]

---

### CopyTradingOrder Entity

Relevant source files

- [.jhipster/CopyTradingOrder.json] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/CopyTradingOrder.json)
- [.jhipster/MarketHistoryJobResult.json] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json)

#### Purpose and Scope

This document describes supporting entities that provide ancillary functionality to the core Copy Trading Platform.

The two supporting entities covered in this document are:

- **Holiday**: Manages market holidays and non-trading days
- **MarketHistoryJobResult**: Tracks execution results for market data processing jobs

For information about the core copy trading entities, see [Copy Trading Platform] (#copy-trading-platform).

* * *

#### Overview

Supporting entities serve operational and administrative needs that span multiple business units.

1. **Temporal Business Logic**: The `Holiday` entity enables the system to account for market holidays and non-trading days.
2. **Job Observability**: The `MarketHistoryJobResult` entity provides audit trails and observability into asynchronous data processing jobs.

```

These entities are defined using JHipster's entity configuration system and follow the same

Sources: [1] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
 [2] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

* * *

Entity Architecture Diagram

! [Chart 1] (nhsv-admin-images/nhsv-admin-section24-1.svg)

Diagram: Supporting Entities and Their System Integration

This diagram shows how supporting entities integrate with core business entities. The `Holiday` entity is a supporting entity that integrates with the `MarketHistoryJobResult` entity.

Sources: [1] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
 [2] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

* * *

Holiday Entity

Purpose

The `Holiday` entity maintains a calendar of market holidays and non-trading days. This includes:

- Determining valid trading windows
- Scheduling automated copy trading operations
- Calculating business day intervals for performance metrics
- Displaying market availability to users

Entity Definition

The `Holiday` entity is defined in `.jhipster/Holiday.json` and generates a JPA entity mapping.

Sources: [1] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
 [2] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json6>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

Field Structure

Field Name	Type	Constraints	Description
`year`	`Integer`	Required	Calendar year for the holiday
`eventHoliday`	`String`	Required, Max 255 chars	Name or description of the holiday
`startDate`	`ZonedDateTime`	None	Holiday period start timestamp
`endDate`	`ZonedDateTime`	None	Holiday period end timestamp
`createdAt`	`ZonedDateTime`	Required	Record creation timestamp
`updatedAt`	`ZonedDateTime`	None	Last modification timestamp

Sources: [1] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
 [2] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json7-35>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

Entity Structure Diagram

! [Chart 2] (nhsv-admin-images/nhsv-admin-section24-2.svg)

Diagram: Holiday Entity Class Structure

This diagram shows the generated backend components for the `Holiday` entity. The entity fo

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json4] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json4>)

[.jhipster/Holiday.json41-44] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json41-44>)

Configuration Details

JHipster Settings:

Setting	Value	Meaning
dto	mapstruct	Uses MapStruct for entity-DTO mapping
service	serviceImpl	Generates service interface and implementation
pagination	pagination	Enables paginated REST endpoints
jpaMetamodelFiltering	false	Disables JPA Metamodel filtering
incrementalChangelog	true	Uses incremental Liquibase changelogs

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json4-44] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json4-44>)

Time Range Representation

The `Holiday` entity supports flexible holiday duration modeling through `startDate` and `e

- ****Single-day holidays**:** `startDate` set to beginning of day, `endDate` set to end of day
- ****Multi-day closures**:** `startDate` and `endDate` define a continuous non-trading period
- ****Null dates**:** Fields are optional, allowing for year-only holiday records

The use of `ZonedDateTime` ensures proper timezone handling across different market regions

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json20-25] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json20-25>)

* * *

MarketHistoryJobResult Entity

Purpose

The `MarketHistoryJobResult` entity tracks the execution results of background jobs that p

- Audit trail for data processing operations
- Error tracking and debugging information
- Job performance monitoring (start/end times)
- User accountability for job execution

Entity Definition

The `MarketHistoryJobResult` entity is defined in `.jhipster/MarketHistoryJobResult.json` &

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

[.jhipster/MarketHistoryJobResult.json1-46] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>)

Field Structure

Field Name	Type	Constraints	Description
`isSuccess`	`Boolean`	None	Whether the job completed successfully
`timeStart`	`ZonedDateTime`	None	Job execution start timestamp
`timeEnd`	`ZonedDateTime`	None	Job execution end timestamp
`error`	`String`	None	Error message if job failed
`eventId`	`String`	None	Identifier for the specific job event
`symbols`	`String`	None	Market symbols processed by the job

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Relationship Structure

The entity has a ****many-to-one**** relationship with the ``User`` entity, tracking which user triggered the job.

Relationship Property	Value
Relationship Type	<code>`many-to-one`</code>
Related Entity	<code>`user`</code>
Owner Side	<code>`true`</code>
Display Field	<code>`login`</code>

This relationship enables:

- Tracking job execution by user
- Filtering jobs by user login
- Auditing who triggered specific data processing operations

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Entity Relationship Diagram

! [Chart 3] (nhsv-admin-images/nhsv-admin-section24-3.svg)

****Diagram: MarketHistoryJobResult Entity Relationships****

This ER diagram shows the relationship between ``User`` and ``MarketHistoryJobResult``. Each job is associated with exactly one user.

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Configuration Details

****JHipster Settings**:**

Setting	Value	Meaning
<code>`dto`</code>	Not specified	Uses entity directly (no DTO generation)
<code>`service`</code>	<code>`no`</code>	No service layer generation; direct repository access
<code>`pagination`</code>	<code>`no`</code>	No pagination on REST endpoints
<code>`incrementalChangelog`</code>	<code>`true`</code>	Uses incremental Liquibase changelogs

The absence of DTO and service layer generation suggests this entity is primarily used for data processing or reporting.

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>]

Job Execution Tracking

![Chart 4] (nhsv-admin-images/nhsv-admin-section24-4.svg)

****Diagram: Market History Job Execution Flow****

This sequence diagram illustrates how the `MarketHistoryJobResult` entity tracks job lifecycle.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>)

Symbols Field Usage

The `symbols` field stores market symbols processed during the job, likely as a comma-separated list.

- Understanding job scope (which symbols were included)
- Filtering failed jobs by specific symbols
- Debugging symbol-specific processing issues

****Example storage formats**:**

- Comma-separated: `"AAPL,GOOGL,MSFT"`
- JSON array: `["AAPL","GOOGL","MSFT"]`

The actual format depends on the implementation of the job processing logic.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json27-28>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json27-28>)

* * *

System Integration Pattern

![Chart 5] (nhsv-admin-images/nhsv-admin-section24-5.svg)

****Diagram: Supporting Entities in System Context****

This diagram shows how supporting entities integrate with external dependencies and business logic.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>)

* * *

Comparison Table

Aspect	Holiday	MarketHistoryJobResult
Primary Purpose	Define non-trading days	Track background job execution
Data Source	Manual entry or calendar API	Generated by job scheduler
Update Frequency	Annually (typically)	Per job execution
Core Fields	`year`, `eventHoliday`, date range	`isSuccess`, `error`, time range
Relationships	None	Many-to-one with User
DTO Generation	Yes (MapStruct)	No
Service Layer	Yes (serviceImpl)	No
Pagination	Yes	No
Primary Consumers	Trading schedulers, availability checks	System administrators, monitoring tools

```

**Sources**:[] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js
[.jhipster/Holiday.json4-44] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhip
.jhipster/MarketHistoryJobResult.json31-45] (https://github.com/nhsv-tradex/nhsv-admin/blob,

* * *

#### Database Schema

##### Holiday Table Structure

```

-- t_holiday table

```

CREATE TABLE t_holiday (
  id BIGSERIAL PRIMARY KEY,
  year INTEGER NOT NULL,
  event_holiday VARCHAR(255) NOT NULL,
  start_date TIMESTAMP WITH TIME ZONE,
  end_date TIMESTAMP WITH TIME ZONE,
  created_at TIMESTAMP WITH TIME ZONE NOT NULL,
  updated_at TIMESTAMP WITH TIME ZONE
);

```

```

##### MarketHistoryJobResult Table Structure

```

-- market_history_job_result table

```

CREATE TABLE market_history_job_result (
  id BIGSERIAL PRIMARY KEY,
  is_success BOOLEAN,
  time_start TIMESTAMP WITH TIME ZONE,
  time_end TIMESTAMP WITH TIME ZONE,
  error TEXT,
  event_id VARCHAR(255),
  symbols TEXT,
  user_id BIGINT,
  FOREIGN KEY (user_id) REFERENCES jhi_user(id)
);

```

These schemas are managed by Liquibase changelogs generated from the JHipster entity definition.

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

* * *

Summary

Supporting entities provide essential infrastructure services that enable core business operations.

- **Holiday**: Enables time-aware business logic by defining non-trading periods, critical for accurate market data processing.
- **MarketHistoryJobResult**: Provides observability into asynchronous data processing, ensuring transparency and reliability of market data.

Both entities leverage JHipster's code generation to provide consistent REST APIs, database schemas, and frontend components.

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

CopyTradingRegister Entity

Relevant source files

- [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

Purpose and Scope

This document describes supporting entities that provide ancillary functionality to the core copy trading platform.

The two supporting entities covered in this document are:

- **Holiday**: Manages market holidays and non-trading days
- **MarketHistoryJobResult**: Tracks execution results for market data processing jobs

For information about the core copy trading entities, see [Copy Trading Platform] (#copy-trading-platform).

* * *

Overview

Supporting entities serve operational and administrative needs that span multiple business units.

1. **Temporal Business Logic**: The `Holiday` entity enables the system to account for market holidays and non-trading days.
2. **Job Observability**: The `MarketHistoryJobResult` entity provides audit trails and execution details for asynchronous data processing jobs.

These entities are defined using JHipster's entity configuration system and follow the same

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
 [.jhipster/Holiday.json1-45] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

* * *

Entity Architecture Diagram

![Chart 1] (nhsv-admin-images/nhsv-admin-section25-1.svg)

****Diagram: Supporting Entities and Their System Integration****

This diagram shows how supporting entities integrate with core business entities. The `Holiday` entity is a core business entity, while `MarketHistoryJobResult` is a supporting entity.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
 [.jhipster/Holiday.json1-45] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

* * *

Holiday Entity

Purpose

The `Holiday` entity maintains a calendar of market holidays and non-trading days. This includes:

- Determining valid trading windows
- Scheduling automated copy trading operations
- Calculating business day intervals for performance metrics
- Displaying market availability to users

Entity Definition

The `Holiday` entity is defined in `.jhipster/Holiday.json` and generates a JPA entity mapping.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
 [.jhipster/Holiday.json6] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

Field Structure

Field Name	Type	Constraints	Description
`year`	`Integer`	Required	Calendar year for the holiday
`eventHoliday`	`String`	Required, Max 255 chars	Name or description of the holiday
`startDate`	`ZonedDateTime`	None	Holiday period start timestamp
`endDate`	`ZonedDateTime`	None	Holiday period end timestamp
`createdAt`	`ZonedDateTime`	Required	Record creation timestamp
`updatedAt`	`ZonedDateTime`	None	Last modification timestamp

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
 [.jhipster/Holiday.json7-35] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

Entity Structure Diagram

![Chart 2] (nhsv-admin-images/nhsv-admin-section25-2.svg)

Diagram: Holiday Entity Class Structure

This diagram shows the generated backend components for the `Holiday` entity. The entity fo

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json4] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json4>)

[.jhipster/Holiday.json41-44] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json41-44>)

Configuration Details

JHipster Settings:

Setting	Value	Meaning
dto	mapstruct	Uses MapStruct for entity-DTO mapping
service	serviceImpl	Generates service interface and implementation
pagination	pagination	Enables paginated REST endpoints
jpaMetamodelFiltering	false	Disables JPA Metamodel filtering
incrementalChangelog	true	Uses incremental Liquibase changelogs

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json4-44] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json4-44>)

Time Range Representation

The `Holiday` entity supports flexible holiday duration modeling through `startDate` and `e

- ****Single-day holidays**:** `startDate` set to beginning of day, `endDate` set to end of day
- ****Multi-day closures**:** `startDate` and `endDate` define a continuous non-trading period
- ****Null dates**:** Fields are optional, allowing for year-only holiday records

The use of `ZonedDateTime` ensures proper timezone handling across different market regions

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json20-25] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json20-25>)

* * *

MarketHistoryJobResult Entity

Purpose

The `MarketHistoryJobResult` entity tracks the execution results of background jobs that p

- Audit trail for data processing operations
- Error tracking and debugging information
- Job performance monitoring (start/end times)
- User accountability for job execution

Entity Definition

The `MarketHistoryJobResult` entity is defined in `.jhipster/MarketHistoryJobResult.json` &

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

[.jhipster/MarketHistoryJobResult.json1-46] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>)

Field Structure

Field Name	Type	Constraints	Description
`isSuccess`	`Boolean`	None	Whether the job completed successfully
`timeStart`	`ZonedDateTime`	None	Job execution start timestamp
`timeEnd`	`ZonedDateTime`	None	Job execution end timestamp
`error`	`String`	None	Error message if job failed
`eventId`	`String`	None	Identifier for the specific job event
`symbols`	`String`	None	Market symbols processed by the job

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Relationship Structure

The entity has a ****many-to-one**** relationship with the ``User`` entity, tracking which user triggered the job.

Relationship Property	Value
Relationship Type	<code>`many-to-one`</code>
Related Entity	<code>`user`</code>
Owner Side	<code>`true`</code>
Display Field	<code>`login`</code>

This relationship enables:

- Tracking job execution by user
- Filtering jobs by user login
- Auditing who triggered specific data processing operations

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Entity Relationship Diagram

! [Chart 3] (nhsv-admin-images/nhsv-admin-section25-3.svg)

****Diagram: MarketHistoryJobResult Entity Relationships****

This ER diagram shows the relationship between ``User`` and ``MarketHistoryJobResult``. Each job is associated with exactly one user.

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Configuration Details

****JHipster Settings**:**

Setting	Value	Meaning
<code>`dto`</code>	Not specified	Uses entity directly (no DTO generation)
<code>`service`</code>	<code>`no`</code>	No service layer generation; direct repository access
<code>`pagination`</code>	<code>`no`</code>	No pagination on REST endpoints
<code>`incrementalChangelog`</code>	<code>`true`</code>	Uses incremental Liquibase changelogs

The absence of DTO and service layer generation suggests this entity is primarily used for data processing or reporting.

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>]

Job Execution Tracking

![Chart 4] (nhsv-admin-images/nhsv-admin-section25-4.svg)

****Diagram: Market History Job Execution Flow****

This sequence diagram illustrates how the `MarketHistoryJobResult` entity tracks job lifecycle.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>)

Symbols Field Usage

The `symbols` field stores market symbols processed during the job, likely as a comma-separated list.

- Understanding job scope (which symbols were included)
- Filtering failed jobs by specific symbols
- Debugging symbol-specific processing issues

****Example storage formats**:**

- Comma-separated: `"AAPL,GOOGL,MSFT"`
- JSON array: `["AAPL","GOOGL","MSFT"]`

The actual format depends on the implementation of the job processing logic.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json27-28>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json27-28>)

* * *

System Integration Pattern

![Chart 5] (nhsv-admin-images/nhsv-admin-section25-5.svg)

****Diagram: Supporting Entities in System Context****

This diagram shows how supporting entities integrate with external dependencies and business logic.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>)

* * *

Comparison Table

Aspect	Holiday	MarketHistoryJobResult
Primary Purpose	Define non-trading days	Track background job execution
Data Source	Manual entry or calendar API	Generated by job scheduler
Update Frequency	Annually (typically)	Per job execution
Core Fields	`year`, `eventHoliday`, date range	`isSuccess`, `error`, time range
Relationships	None	Many-to-one with User
DTO Generation	Yes (MapStruct)	No
Service Layer	Yes (serviceImpl)	No
Pagination	Yes	No
Primary Consumers	Trading schedulers, availability checks	System administrators, monitoring tools

```

**Sources**:[] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js
[.jhipster/Holiday.json4-44] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhip
.jhipster/MarketHistoryJobResult.json31-45] (https://github.com/nhsv-tradex/nhsv-admin/blob,

* * *

#### Database Schema

##### Holiday Table Structure

```

-- t_holiday table

```

CREATE TABLE t_holiday (
  id BIGSERIAL PRIMARY KEY,
  year INTEGER NOT NULL,
  event_holiday VARCHAR(255) NOT NULL,
  start_date TIMESTAMP WITH TIME ZONE,
  end_date TIMESTAMP WITH TIME ZONE,
  created_at TIMESTAMP WITH TIME ZONE NOT NULL,
  updated_at TIMESTAMP WITH TIME ZONE
);

```

```

##### MarketHistoryJobResult Table Structure

```

-- market_history_job_result table

```

CREATE TABLE market_history_job_result (
  id BIGSERIAL PRIMARY KEY,
  is_success BOOLEAN,
  time_start TIMESTAMP WITH TIME ZONE,
  time_end TIMESTAMP WITH TIME ZONE,
  error TEXT,
  event_id VARCHAR(255),
  symbols TEXT,
  user_id BIGINT,
  FOREIGN KEY (user_id) REFERENCES jhi_user(id)
);

```


These entities are defined using JHipster's entity configuration system and follow the same

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
 [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-46>)

* * *

Entity Architecture Diagram

! [Chart 1] (nhsv-admin-images/nhsv-admin-section26-1.svg)

Diagram: Supporting Entities and Their System Integration

This diagram shows how supporting entities integrate with core business entities. The `Holiday` entity is a supporting entity that integrates with the `MarketHistoryJobResult` entity.

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
 [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-46>)

* * *

Holiday Entity

Purpose

The `Holiday` entity maintains a calendar of market holidays and non-trading days. This includes:

- Determining valid trading windows
- Scheduling automated copy trading operations
- Calculating business day intervals for performance metrics
- Displaying market availability to users

Entity Definition

The `Holiday` entity is defined in `.jhipster/Holiday.json` and generates a JPA entity mapping.

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
 [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json6>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-46>)

Field Structure

Field Name	Type	Constraints	Description
`year`	`Integer`	Required	Calendar year for the holiday
`eventHoliday`	`String`	Required, Max 255 chars	Name or description of the holiday
`startDate`	`ZonedDateTime`	None	Holiday period start timestamp
`endDate`	`ZonedDateTime`	None	Holiday period end timestamp
`createdAt`	`ZonedDateTime`	Required	Record creation timestamp
`updatedAt`	`ZonedDateTime`	None	Last modification timestamp

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
 [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json7-35>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-46>)

Entity Structure Diagram

! [Chart 2] (nhsv-admin-images/nhsv-admin-section26-2.svg)

Diagram: Holiday Entity Class Structure

This diagram shows the generated backend components for the `Holiday` entity. The entity fo

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json4] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json4>)

[.jhipster/Holiday.json41-44] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json41-44>)

Configuration Details

JHipster Settings:

Setting	Value	Meaning
dto	mapstruct	Uses MapStruct for entity-DTO mapping
service	serviceImpl	Generates service interface and implementation
pagination	pagination	Enables paginated REST endpoints
jpaMetamodelFiltering	false	Disables JPA Metamodel filtering
incrementalChangelog	true	Uses incremental Liquibase changelogs

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json4-44] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json4-44>)

Time Range Representation

The `Holiday` entity supports flexible holiday duration modeling through `startDate` and `e

- ****Single-day holidays**:** `startDate` set to beginning of day, `endDate` set to end of day
- ****Multi-day closures**:** `startDate` and `endDate` define a continuous non-trading period
- ****Null dates**:** Fields are optional, allowing for year-only holiday records

The use of `ZonedDateTime` ensures proper timezone handling across different market regions

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json20-25] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json20-25>)

* * *

MarketHistoryJobResult Entity

Purpose

The `MarketHistoryJobResult` entity tracks the execution results of background jobs that p

- Audit trail for data processing operations
- Error tracking and debugging information
- Job performance monitoring (start/end times)
- User accountability for job execution

Entity Definition

The `MarketHistoryJobResult` entity is defined in `.jhipster/MarketHistoryJobResult.json` &

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

[.jhipster/MarketHistoryJobResult.json1-46] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>)

Field Structure

Field Name	Type	Constraints	Description
`isSuccess`	`Boolean`	None	Whether the job completed successfully
`timeStart`	`ZonedDateTime`	None	Job execution start timestamp
`timeEnd`	`ZonedDateTime`	None	Job execution end timestamp
`error`	`String`	None	Error message if job failed
`eventId`	`String`	None	Identifier for the specific job event
`symbols`	`String`	None	Market symbols processed by the job

****Sources**:** [(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29)] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29)

Relationship Structure

The entity has a **many-to-one** relationship with the ``User`` entity, tracking which user triggered the job.

Relationship Property	Value
Relationship Type	<code>`many-to-one`</code>
Related Entity	<code>`user`</code>
Owner Side	<code>`true`</code>
Display Field	<code>`login`</code>

This relationship enables:

- Tracking job execution by user
- Filtering jobs by user login
- Auditing who triggered specific data processing operations

****Sources**:** [(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43)] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43)

Entity Relationship Diagram

! [Chart 3] (nhsv-admin-images/nhsv-admin-section26-3.svg)

****Diagram: MarketHistoryJobResult Entity Relationships****

This ER diagram shows the relationship between ``User`` and ``MarketHistoryJobResult``. Each job is associated with exactly one user.

****Sources**:** [(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43)] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43)

Configuration Details

****JHipster Settings**:**

Setting	Value	Meaning
<code>`dto`</code>	Not specified	Uses entity directly (no DTO generation)
<code>`service`</code>	<code>`no`</code>	No service layer generation; direct repository access
<code>`pagination`</code>	<code>`no`</code>	No pagination on REST endpoints
<code>`incrementalChangelog`</code>	<code>`true`</code>	Uses incremental Liquibase changelogs

The absence of DTO and service layer generation suggests this entity is primarily used for data processing or reporting.

****Sources**:** [(https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45) (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45)] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45)

Job Execution Tracking

![Chart 4] (nhsv-admin-images/nhsv-admin-section26-4.svg)

****Diagram: Market History Job Execution Flow****

This sequence diagram illustrates how the `MarketHistoryJobResult` entity tracks job lifecycle.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>)

Symbols Field Usage

The `symbols` field stores market symbols processed during the job, likely as a comma-separated list.

- Understanding job scope (which symbols were included)
- Filtering failed jobs by specific symbols
- Debugging symbol-specific processing issues

****Example storage formats**:**

- Comma-separated: `"AAPL,GOOGL,MSFT"`
- JSON array: `["AAPL","GOOGL","MSFT"]`

The actual format depends on the implementation of the job processing logic.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json27-28>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json27-28>)

* * *

System Integration Pattern

![Chart 5] (nhsv-admin-images/nhsv-admin-section26-5.svg)

****Diagram: Supporting Entities in System Context****

This diagram shows how supporting entities integrate with external dependencies and business logic.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>)

* * *

Comparison Table

Aspect	Holiday	MarketHistoryJobResult
Primary Purpose	Define non-trading days	Track background job execution
Data Source	Manual entry or calendar API	Generated by job scheduler
Update Frequency	Annually (typically)	Per job execution
Core Fields	`year`, `eventHoliday`, date range	`isSuccess`, `error`, time range
Relationships	None	Many-to-one with User
DTO Generation	Yes (MapStruct)	No
Service Layer	Yes (serviceImpl)	No
Pagination	Yes	No
Primary Consumers	Trading schedulers, availability checks	System administrators, monitoring tools

```

**Sources**:[] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js
[.jhipster/Holiday.json4-44] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhip
.jhipster/MarketHistoryJobResult.json31-45] (https://github.com/nhsv-tradex/nhsv-admin/blob,

* * *

#### Database Schema

##### Holiday Table Structure

```

-- t_holiday table

```

CREATE TABLE t_holiday (
  id BIGSERIAL PRIMARY KEY,
  year INTEGER NOT NULL,
  event_holiday VARCHAR(255) NOT NULL,
  start_date TIMESTAMP WITH TIME ZONE,
  end_date TIMESTAMP WITH TIME ZONE,
  created_at TIMESTAMP WITH TIME ZONE NOT NULL,
  updated_at TIMESTAMP WITH TIME ZONE
);

```

```

##### MarketHistoryJobResult Table Structure

```

-- market_history_job_result table

```

CREATE TABLE market_history_job_result (
  id BIGSERIAL PRIMARY KEY,
  is_success BOOLEAN,
  time_start TIMESTAMP WITH TIME ZONE,
  time_end TIMESTAMP WITH TIME ZONE,
  error TEXT,
  event_id VARCHAR(255),
  symbols TEXT,
  user_id BIGINT,
  FOREIGN KEY (user_id) REFERENCES jhi_user(id)
);

```

These schemas are managed by Liquibase changelogs generated from the JHipster entity definition.

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

* * *

Summary

Supporting entities provide essential infrastructure services that enable core business operations.

- **Holiday**: Enables time-aware business logic by defining non-trading periods, critical for accurate market data processing.
- **MarketHistoryJobResult**: Provides observability into asynchronous data processing, ensuring transparency and reliability of market data.

Both entities leverage JHipster's code generation to provide consistent REST APIs, database schemas, and frontend components.

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

Holiday Entity

Relevant source files

- [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json>)
- [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

Purpose and Scope

This document describes supporting entities that provide ancillary functionality to the core trading platform.

The two supporting entities covered in this document are:

- **Holiday**: Manages market holidays and non-trading days
- **MarketHistoryJobResult**: Tracks execution results for market data processing jobs

For information about the core copy trading entities, see [Copy Trading Platform] (#copy-trading-platform).

* * *

Overview

Supporting entities serve operational and administrative needs that span multiple business units.

1. **Temporal Business Logic**: The `Holiday` entity enables the system to account for market holidays and non-trading days.
2. **Job Observability**: The `MarketHistoryJobResult` entity provides audit trails and execution details for asynchronous data processing jobs.

These entities are defined using JHipster's entity configuration system and follow the same

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
[.jhipster/Holiday.json1-45] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

* * *

Entity Architecture Diagram

![Chart 1] (nhsv-admin-images/nhsv-admin-section27-1.svg)

Diagram: Supporting Entities and Their System Integration

This diagram shows how supporting entities integrate with core business entities. The `Holiday` entity is a supporting entity that integrates with the `MarketHistoryJobResult` entity.

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
[.jhipster/Holiday.json1-45] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

* * *

Holiday Entity

Purpose

The `Holiday` entity maintains a calendar of market holidays and non-trading days. This includes:

- Determining valid trading windows
- Scheduling automated copy trading operations
- Calculating business day intervals for performance metrics
- Displaying market availability to users

Entity Definition

The `Holiday` entity is defined in `.jhipster/Holiday.json` and generates a JPA entity map.

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
[.jhipster/Holiday.json6] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

Field Structure

Field Name	Type	Constraints	Description
`year`	`Integer`	Required	Calendar year for the holiday
`eventHoliday`	`String`	Required, Max 255 chars	Name or description of the holiday
`startDate`	`ZonedDateTime`	None	Holiday period start timestamp
`endDate`	`ZonedDateTime`	None	Holiday period end timestamp
`createdAt`	`ZonedDateTime`	Required	Record creation timestamp
`updatedAt`	`ZonedDateTime`	None	Last modification timestamp

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
[.jhipster/Holiday.json7-35] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

Entity Structure Diagram

![Chart 2] (nhsv-admin-images/nhsv-admin-section27-2.svg)

Diagram: Holiday Entity Class Structure

This diagram shows the generated backend components for the `Holiday` entity. The entity fo

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json4] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json4>)

[.jhipster/Holiday.json41-44] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json41-44>)

Configuration Details

JHipster Settings:

Setting	Value	Meaning
dto	mapstruct	Uses MapStruct for entity-DTO mapping
service	serviceImpl	Generates service interface and implementation
pagination	pagination	Enables paginated REST endpoints
jpaMetamodelFiltering	false	Disables JPA Metamodel filtering
incrementalChangelog	true	Uses incremental Liquibase changelogs

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json4-44] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json4-44>)

Time Range Representation

The `Holiday` entity supports flexible holiday duration modeling through `startDate` and `e

- ****Single-day holidays**:** `startDate` set to beginning of day, `endDate` set to end of day
- ****Multi-day closures**:** `startDate` and `endDate` define a continuous non-trading period
- ****Null dates**:** Fields are optional, allowing for year-only holiday records

The use of `ZonedDateTime` ensures proper timezone handling across different market regions

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json20-25] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json20-25>)

* * *

MarketHistoryJobResult Entity

Purpose

The `MarketHistoryJobResult` entity tracks the execution results of background jobs that p

- Audit trail for data processing operations
- Error tracking and debugging information
- Job performance monitoring (start/end times)
- User accountability for job execution

Entity Definition

The `MarketHistoryJobResult` entity is defined in `.jhipster/MarketHistoryJobResult.json` &

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

[.jhipster/MarketHistoryJobResult.json1-46] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>)

Field Structure

Field Name	Type	Constraints	Description
`isSuccess`	`Boolean`	None	Whether the job completed successfully
`timeStart`	`ZonedDateTime`	None	Job execution start timestamp
`timeEnd`	`ZonedDateTime`	None	Job execution end timestamp
`error`	`String`	None	Error message if job failed
`eventId`	`String`	None	Identifier for the specific job event
`symbols`	`String`	None	Market symbols processed by the job

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Relationship Structure

The entity has a **many-to-one** relationship with the ``User`` entity, tracking which user triggered the job.

Relationship Property	Value
Relationship Type	<code>`many-to-one`</code>
Related Entity	<code>`user`</code>
Owner Side	<code>`true`</code>
Display Field	<code>`login`</code>

This relationship enables:

- Tracking job execution by user
- Filtering jobs by user login
- Auditing who triggered specific data processing operations

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Entity Relationship Diagram

! [Chart 3] (nhsv-admin-images/nhsv-admin-section27-3.svg)

****Diagram: MarketHistoryJobResult Entity Relationships****

This ER diagram shows the relationship between ``User`` and ``MarketHistoryJobResult``. Each job is associated with exactly one user.

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Configuration Details

****JHipster Settings**:**

Setting	Value	Meaning
<code>`dto`</code>	Not specified	Uses entity directly (no DTO generation)
<code>`service`</code>	<code>`no`</code>	No service layer generation; direct repository access
<code>`pagination`</code>	<code>`no`</code>	No pagination on REST endpoints
<code>`incrementalChangelog`</code>	<code>`true`</code>	Uses incremental Liquibase changelogs

The absence of DTO and service layer generation suggests this entity is primarily used for data processing or reporting.

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>]

Job Execution Tracking

![Chart 4] (nhsv-admin-images/nhsv-admin-section27-4.svg)

****Diagram: Market History Job Execution Flow****

This sequence diagram illustrates how the `MarketHistoryJobResult` entity tracks job lifecycle.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>)

Symbols Field Usage

The `symbols` field stores market symbols processed during the job, likely as a comma-separated list.

- Understanding job scope (which symbols were included)
- Filtering failed jobs by specific symbols
- Debugging symbol-specific processing issues

****Example storage formats**:**

- Comma-separated: `"AAPL,GOOGL,MSFT"`
- JSON array: `["AAPL","GOOGL","MSFT"]`

The actual format depends on the implementation of the job processing logic.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json27-28>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json27-28>)

* * *

System Integration Pattern

![Chart 5] (nhsv-admin-images/nhsv-admin-section27-5.svg)

****Diagram: Supporting Entities in System Context****

This diagram shows how supporting entities integrate with external dependencies and business logic.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>)

* * *

Comparison Table

Aspect	Holiday	MarketHistoryJobResult
Primary Purpose	Define non-trading days	Track background job execution
Data Source	Manual entry or calendar API	Generated by job scheduler
Update Frequency	Annually (typically)	Per job execution
Core Fields	`year`, `eventHoliday`, date range	`isSuccess`, `error`, time range
Relationships	None	Many-to-one with User
DTO Generation	Yes (MapStruct)	No
Service Layer	Yes (serviceImpl)	No
Pagination	Yes	No
Primary Consumers	Trading schedulers, availability checks	System administrators, monitoring tools


```

**Sources**:[] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js
[.jhipster/Holiday.json4-44] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhip
.jhipster/MarketHistoryJobResult.json31-45] (https://github.com/nhsv-tradex/nhsv-admin/blob,

* * *

#### Database Schema

##### Holiday Table Structure

```

-- t_holiday table

```

CREATE TABLE t_holiday (
  id BIGSERIAL PRIMARY KEY,
  year INTEGER NOT NULL,
  event_holiday VARCHAR(255) NOT NULL,
  start_date TIMESTAMP WITH TIME ZONE,
  end_date TIMESTAMP WITH TIME ZONE,
  created_at TIMESTAMP WITH TIME ZONE NOT NULL,
  updated_at TIMESTAMP WITH TIME ZONE
);

```

```

##### MarketHistoryJobResult Table Structure

```

-- market_history_job_result table

```

CREATE TABLE market_history_job_result (
  id BIGSERIAL PRIMARY KEY,
  is_success BOOLEAN,
  time_start TIMESTAMP WITH TIME ZONE,
  time_end TIMESTAMP WITH TIME ZONE,
  error TEXT,
  event_id VARCHAR(255),
  symbols TEXT,
  user_id BIGINT,
  FOREIGN KEY (user_id) REFERENCES jhi_user(id)
);

```

```

These schemas are managed by Liquibase changelogs generated from the JHipster entity definition.

**Sources**: [] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json)
[.jhipster/Holiday.json7-35] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-43] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json4-43] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json5-43]

* * *

#### Summary

Supporting entities provide essential infrastructure services that enable core business operations.

- **Holiday**: Enables time-aware business logic by defining non-trading periods, critical for accurate market data processing.
- **MarketHistoryJobResult**: Provides observability into asynchronous data processing, ensuring transparency and auditability.

Both entities leverage JHipster's code generation to provide consistent REST APIs, database schemas, and frontend components.

**Sources**: [] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json)
[.jhipster/Holiday.json1-45] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json2-46] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-46]

---

### MarketHistoryJobResult Entity

Relevant source files

- [.jhipster/MarketHistoryJobResult.json] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json)
- [.jhipster/MarketHistoryJobResult.json] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json)

#### Purpose and Scope

This document describes supporting entities that provide ancillary functionality to the core trading platform.

The two supporting entities covered in this document are:

- **Holiday**: Manages market holidays and non-trading days
- **MarketHistoryJobResult**: Tracks execution results for market data processing jobs

For information about the core copy trading entities, see [Copy Trading Platform] (#copy-trading-platform).

* * *

#### Overview

Supporting entities serve operational and administrative needs that span multiple business units.

1. **Temporal Business Logic**: The `Holiday` entity enables the system to account for market holidays and non-trading days.
2. **Job Observability**: The `MarketHistoryJobResult` entity provides audit trails and observability into asynchronous data processing jobs.

```

These entities are defined using JHipster's entity configuration system and follow the same

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
[.jhipster/Holiday.json1-45] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

* * *

Entity Architecture Diagram

! [Chart 1] (nhsv-admin-images/nhsv-admin-section28-1.svg)

Diagram: Supporting Entities and Their System Integration

This diagram shows how supporting entities integrate with core business entities. The `Holiday` entity is a core business entity that integrates with the `MarketHistoryJobResult` entity.

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
[.jhipster/Holiday.json1-45] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

* * *

Holiday Entity

Purpose

The `Holiday` entity maintains a calendar of market holidays and non-trading days. This includes:

- Determining valid trading windows
- Scheduling automated copy trading operations
- Calculating business day intervals for performance metrics
- Displaying market availability to users

Entity Definition

The `Holiday` entity is defined in `.jhipster/Holiday.json` and generates a JPA entity map.

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
[.jhipster/Holiday.json6] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

Field Structure

Field Name	Type	Constraints	Description
`year`	`Integer`	Required	Calendar year for the holiday
`eventHoliday`	`String`	Required, Max 255 chars	Name or description of the holiday
`startDate`	`ZonedDateTime`	None	Holiday period start timestamp
`endDate`	`ZonedDateTime`	None	Holiday period end timestamp
`createdAt`	`ZonedDateTime`	Required	Record creation timestamp
`updatedAt`	`ZonedDateTime`	None	Last modification timestamp

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)
[.jhipster/Holiday.json7-35] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json44-45>)

Entity Structure Diagram

! [Chart 2] (nhsv-admin-images/nhsv-admin-section28-2.svg)

Diagram: Holiday Entity Class Structure

This diagram shows the generated backend components for the `Holiday` entity. The entity fo

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json4] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json4>)

[.jhipster/Holiday.json41-44] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json41-44>)

Configuration Details

JHipster Settings:

Setting	Value	Meaning
dto	mapstruct	Uses MapStruct for entity-DTO mapping
service	serviceImpl	Generates service interface and implementation
pagination	pagination	Enables paginated REST endpoints
jpaMetamodelFiltering	false	Disables JPA Metamodel filtering
incrementalChangelog	true	Uses incremental Liquibase changelogs

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json4-44] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json4-44>)

Time Range Representation

The `Holiday` entity supports flexible holiday duration modeling through `startDate` and `e

- ****Single-day holidays**:** `startDate` set to beginning of day, `endDate` set to end of day
- ****Multi-day closures**:** `startDate` and `endDate` define a continuous non-trading period
- ****Null dates**:** Fields are optional, allowing for year-only holiday records

The use of `ZonedDateTime` ensures proper timezone handling across different market regions

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js>)

[.jhipster/Holiday.json20-25] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json20-25>)

* * *

MarketHistoryJobResult Entity

Purpose

The `MarketHistoryJobResult` entity tracks the execution results of background jobs that p

- Audit trail for data processing operations
- Error tracking and debugging information
- Job performance monitoring (start/end times)
- User accountability for job execution

Entity Definition

The `MarketHistoryJobResult` entity is defined in `.jhipster/MarketHistoryJobResult.json` &

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json>)

[.jhipster/MarketHistoryJobResult.json1-46] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>)

Field Structure

Field Name	Type	Constraints	Description
`isSuccess`	`Boolean`	None	Whether the job completed successfully
`timeStart`	`ZonedDateTime`	None	Job execution start timestamp
`timeEnd`	`ZonedDateTime`	None	Job execution end timestamp
`error`	`String`	None	Error message if job failed
`eventId`	`String`	None	Identifier for the specific job event
`symbols`	`String`	None	Market symbols processed by the job

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Relationship Structure

The entity has a **many-to-one** relationship with the ``User`` entity, tracking which user triggered the job.

Relationship Property	Value
Relationship Type	<code>`many-to-one`</code>
Related Entity	<code>`user`</code>
Owner Side	<code>`true`</code>
Display Field	<code>`login`</code>

This relationship enables:

- Tracking job execution by user
- Filtering jobs by user login
- Auditing who triggered specific data processing operations

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Entity Relationship Diagram

! [Chart 3] (nhsv-admin-images/nhsv-admin-section28-3.svg)

****Diagram: MarketHistoryJobResult Entity Relationships****

This ER diagram shows the relationship between ``User`` and ``MarketHistoryJobResult``. Each job is associated with exactly one user.

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>]

Configuration Details

****JHipster Settings**:**

Setting	Value	Meaning
<code>`dto`</code>	Not specified	Uses entity directly (no DTO generation)
<code>`service`</code>	<code>`no`</code>	No service layer generation; direct repository access
<code>`pagination`</code>	<code>`no`</code>	No pagination on REST endpoints
<code>`incrementalChangelog`</code>	<code>`true`</code>	Uses incremental Liquibase changelogs

The absence of DTO and service layer generation suggests this entity is primarily used for data processing or reporting.

****Sources**:** [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json31-45>] [<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json35-43>]

Job Execution Tracking

![Chart 4] (nhsv-admin-images/nhsv-admin-section28-4.svg)

****Diagram: Market History Job Execution Flow****

This sequence diagram illustrates how the `MarketHistoryJobResult` entity tracks job lifecycle.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json3-29>)

Symbols Field Usage

The `symbols` field stores market symbols processed during the job, likely as a comma-separated list.

- Understanding job scope (which symbols were included)
- Filtering failed jobs by specific symbols
- Debugging symbol-specific processing issues

****Example storage formats**:**

- Comma-separated: `"AAPL,GOOGL,MSFT"`
- JSON array: `["AAPL","GOOGL","MSFT"]`

The actual format depends on the implementation of the job processing logic.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json27-28>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json27-28>)

* * *

System Integration Pattern

![Chart 5] (nhsv-admin-images/nhsv-admin-section28-5.svg)

****Diagram: Supporting Entities in System Context****

This diagram shows how supporting entities integrate with external dependencies and business logic.

****Sources**:** [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json1-45>)

[.jhipster/MarketHistoryJobResult.json1-46] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json1-46>)

* * *

Comparison Table

Aspect	Holiday	MarketHistoryJobResult
Primary Purpose	Define non-trading days	Track background job execution
Data Source	Manual entry or calendar API	Generated by job scheduler
Update Frequency	Annually (typically)	Per job execution
Core Fields	`year`, `eventHoliday`, date range	`isSuccess`, `error`, time range
Relationships	None	Many-to-one with User
DTO Generation	Yes (MapStruct)	No
Service Layer	Yes (serviceImpl)	No
Pagination	Yes	No
Primary Consumers	Trading schedulers, availability checks	System administrators, monitoring tools

```

**Sources**:[ ] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.js
[.jhipster/Holiday.json4-44] (https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhip
.jhipster/MarketHistoryJobResult.json31-45] (https://github.com/nhsv-tradex/nhsv-admin/blob,

* * *

#### Database Schema

##### Holiday Table Structure

```

```

-- t_holiday table
CREATE TABLE t_holiday (
  id BIGSERIAL PRIMARY KEY,
  year INTEGER NOT NULL,
  event_holiday VARCHAR(255) NOT NULL,
  start_date TIMESTAMP WITH TIME ZONE,
  end_date TIMESTAMP WITH TIME ZONE,
  created_at TIMESTAMP WITH TIME ZONE NOT NULL,
  updated_at TIMESTAMP WITH TIME ZONE
);

```

```

##### MarketHistoryJobResult Table Structure

```

```

-- market_history_job_result table
CREATE TABLE market_history_job_result (
  id BIGSERIAL PRIMARY KEY,
  is_success BOOLEAN,
  time_start TIMESTAMP WITH TIME ZONE,
  time_end TIMESTAMP WITH TIME ZONE,
  error TEXT,
  event_id VARCHAR(255),
  symbols TEXT,
  user_id BIGINT,
  FOREIGN KEY (user_id) REFERENCES jhi_user(id)
);

```

These schemas are managed by Liquibase changelogs generated from the JHipster entity definition.

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json#L7-35>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json#L3-43>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json#L3-43>)

* * *

Summary

Supporting entities provide essential infrastructure services that enable core business operations.

- **Holiday**: Enables time-aware business logic by defining non-trading periods, critical for accurate market data processing.
- **MarketHistoryJobResult**: Provides observability into asynchronous data processing, enabling monitoring and troubleshooting of long-running jobs.

Both entities leverage JHipster's code generation to provide consistent REST APIs, database schemas, and frontend components.

Sources: [] (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/Holiday.json#L1-45>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json#L1-46>) (<https://github.com/nhsv-tradex/nhsv-admin/blob/2cf90647/.jhipster/MarketHistoryJobResult.json#L1-46>)
